

Data Science Tools (Winter Term 2025/26)

Preliminary and incomplete draft

Ulrich Küsters

2025-11-04 19:55:56

Contents

About	7
1 Frontmatter	9
Preface	9
Notes	9
2 Overview	11
2.1 Course information	11
2.2 Course content	12
2.3 Organization of the course	15
2.4 Preliminary detailed overview of course content	17
3 Software	19
3.1 Reference sources	20
4 Selected Literature	23
4.1 Primary literature	23
4.2 Secondary literature	24
4.3 Literature about tools	24
4.4 Literature about R and Python	25
4.5 Literature on R as a programming language	25
4.6 Literature about R and SQL	25

4.7	Operating systems	26
4.8	Reference works	26
4.9	Personal recommendation for purchase	26
4.10	Course accompanying materials	26
4.11	Online sources	27
4.12	Online help and further information	28
5	RGui	29
5.1	Important R commands for help (within R)	31
6	RStudio	35
6.1	RStudio screen	35
6.2	RStudio elements	37
6.3	RStudio – the most important panels (panes) and keyboard shortcuts	38
6.4	Project Management with RStudio (tbd)	40
6.5	Version Control with RStudio and Github (tbd)	40
6.6	Documentation using RMarkdown and Quarto (tbd)	41
7	Data sets	43
7.1	Overview of four example data sets	43
7.2	Data set <code>german</code> (or <code>German Credit</code>)	44
7.3	Analysis with the R method <code>summary</code>	48
7.4	Analysis with the R package <code>mosaic</code>	51
7.5	Analysis with the R package <code>summarytools</code>	54
7.6	Summary analysis (summary statistics)	60
7.7	Exemplary graphs with ‘ <code>ggplot2</code> ’	70
7.8	Data set <code>nycflights13</code>	79
8	Overview about of ‘graphics’, <code>lattice</code> and <code>ggplot2</code>	85
9	Base graphics in R {<code>r-base-graphics</code>}	87
9.1	Data	87
9.2	Histogram	94
9.3	Density plot	96
9.4	Barplot	97

9.5	Pie chart	99
9.6	Dotchart	100
9.7	Boxplots	101
9.8	Index plot	102
9.9	X-Y plot	106
9.10	X-Y plot extended and annotated	107
9.11	Tile or panel plot with graphics	109
9.12	Global graphic parameters with par	110
9.13	Overview of some important functions of the R package graphics	114
9.14	X-Y plot with regression line	114
9.15	Plotting functions in graphics	119
9.16	Excursus: Generating functions with R	120
9.17	Notes on the genesis of the R graphics functions	125
9.18	Excursus: attach and detach	125
10	Lattice graphics in R	129
10.1	Overview	129
10.2	Basic structure of the lattice functions	129
10.3	lattice -examples ‘univariate’	130
10.4	lattice -examples ‘bivariat’	134
10.5	lattice examples with conditioning variables	135
10.6	lattice examples for three variables	138
11	Dynamic 3D graphs with plotly	141
12	Appendix A: List of links for R software, libraries, help, and introductions	143
12.1	Basic software	143
12.2	Manuals and help	145
12.3	Supplementary online information on textbooks and libraries	148
12.4	Documentation of R programs and data analyses	153
12.5	Relevant IT standards	154
13	References	159

About

Lecture notes for the course **Data Science Tools** in the Winter Term (WS 2025/26) mainly for master students of *Business Administration and Operations Research* at the *Department of Economics* and master students of *Data Science* at the *Department of Mathematics and Geography* of the **Catholic University of Eichstätt-Ingolstadt**. These notes are based on a translated and modified version of German lecture notes for several courses about *Data Science Werkzeuge* and *Data Science Methoden* taught since 2016.

Chapter 1

Frontmatter

Preface

tbd

Notes

The creation and correction of this manuscript has not yet been completed. Please use only within the course. Distribution is not permitted.

This document is constantly being corrected and expanded. Please note that links may change at any time. It may also be the case that the specified websites no longer exist or are even redirected to harmful sites, which must then be intercepted by your firewall, your virus scanner, your browser or by yourself.

Please note that the URLs behind the links are not visible in the HTML and Word outputs generated from this R Markdown file. In the PDF output files, the links are sometimes placed in footnotes. If you click on links with a blue background, the corresponding URL usually opens. In some cases, however, only the file behind the URL is saved in the download directory.

When checking the links selectively, it was found that a few URLs that referred to helpful information (especially on R and the Tidyverse) at the time of creation now contain problematic content or link to problematic pages. These were then - as far as known - eliminated or

replaced. Please note, however, that I cannot accept any responsibility for the accuracy of the links and in particular for the content of the links.

Please send any comments and suggestions for corrections to me by Email¹. Sometimes there are still links to German URLs. It would be great if you would inform me about these.

No responsibility is taken for the contents of the linked pages.

Some additional notes: When generating graphics with the R package **GGally** in chapter 3, a positioning problem occurred in December 2020 with R version 4.0.3, which cannot be traced back to the versions of the R packages used. Therefore, we **temporarily** used the last R version from the main version 3 (R version 3.6.3 under Windows²) in the course in WS 2020/21. These problems have been fixed since October 30, 2023 at the latest, but you should always bear in mind when developing production software that there are sometimes version problems that you can only avoid when using an R program for a longer period of time by fixing to an R version and fixing the R packages using the `renv`³ environment. The R packages used and the R version used can be read out and documented using the R function `sessionInfo()`; see also the appendix of this document.

In **most cases**, the PDF files generated with RMarkdown via **knitr** are readable with *Adobe Reader*. However, this is not always possible. If you want to create a 100% Adobe-compatible PDF file, you can generate it using the Sumatra PDF viewer integrated in RStudio by selecting the ‘Microsoft Print to PDF’ printer under Windows 10 and generating a new PDF file. However, this is significantly larger than the original PDF file created with **knitr** by a factor of around 100.

This problem occurs with all graphics that are (1) created with `dfSummary` from the `summarytools` package, (2) embedded in RMarkdown and (3) when ‘knitr’ and ‘pandoc’ are used to generate a PDF file. When the corresponding **knitr** option is activated some versions of Adobe PDF reader (a) generates an error message and (b) omits the graphics embedded in the tables. In the Sumatra PDF reader, which is used internally in the Windows version of RStudio, however, the graphics are displayed. Likewise, the generated PDF files can be displayed without problems with the Microsoft Edge browser under Microsoft Windows 10/11, for example. If you activate the **knitr** option to create an HTML or Word `docx` file, the embedded graphics are displayed correctly.

Otherwise: If you want to create comparable documents like this **NEW**, I recommend using the more modern variant **Quarto** instead of **RMarkdown**.

¹<mailto:ulrich.kuesters@ku.de>

²<https://cran.r-project.org/bin/windows/base/old/3.6.3/R-3.6.3-win.exe>

³<https://rstudio.github.io/renv/>

Chapter 2

Overview

2.1 Course information

2.1.1 Note

It may be necessary to teach the statistical programming environment R as part of the course. In this case, the planned material cannot be taught in its entirety, so that the parts planned for January/February 2026 may only be covered in other courses such as *Data Science Methods*.

2.1.2 Regular course slots (subject to change)

- Winter semester 2025/26 from October 13, 2025, to February 6, 2026, net = 15 weeks
 - Net $15 \times 4 = 60$ hours, each lasting 45 minutes
 - Spread over 10-11 sessions, mostly 6 hours on Wednesdays starting October 15, between 1:00 p.m. and 6:00 p.m. with variable slots, usually
 - * 1:00 p.m. to 2:30 p.m. [2]
 - * 2:45 p.m. to 4:15 p.m. [2]
 - * 4:30 p.m. to 6:00 p.m. [2]
 - In addition, there will be a preliminary discussion of the exam, also on a Wednesday in the same time slot as above.

- Some Wednesday sessions may be moved to Fridays.

Important Note: Participants should have a solid knowledge of **R** and **RStudio** and (to a much lesser extent) **SQL** in some cases. If these prerequisites are not met, the missing knowledge will be taught ‘on the fly’ and other topics will be shortened or omitted. However, the minimum requirement is participation in the R crash course, which will be held on - October 25, 2025 from 10:00 am to 6:00 pm, - November 7, 2025 from 10:00 am to 2:00 pm and - November 14, 2025 from 10:00 am to 2:00 pm.

The aim is to teach the central tools by December 2025, followed by a preliminary overview of selected data science methods in January 2026. This plan is (very) optimistic; it is likely that some parts and methodological aspects can only be explored in greater depth in the other data science courses.

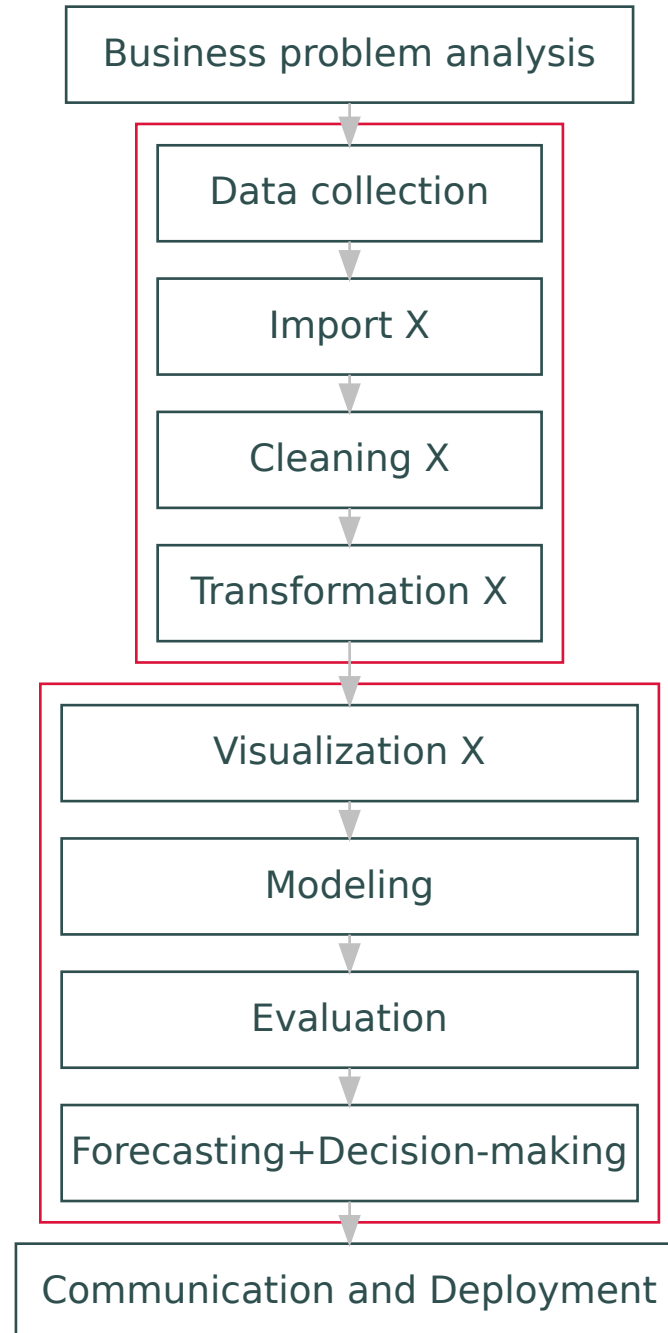
2.1.3 Location

- HB-U03, subject to change.
- Exceptions will be announced on the KU campus.
- Zoom only if the instructor is ill but able to conduct the course online.

2.2 Course content

Using examples, the following elements are used to explain preparation, exploration, analysis, and graphical presentation:

1. The statistical programming environment **R**.
2. The integrated development environment **RStudio**.
3. Numerous R libraries (mainly from the **tidyverse**) such as
 - **ggplot2** for graphics,
 - **dplyr** and **tidyr** for data preparation, and
 - **MASS** for data analysis ¹.
4. Publicly available data sets such as
 - **mtcars**, a data set describing the features of 32 cars describing *Motor Trend Car Road Tests*)
 - **nycflights13** of domestic flights from New York in 2013 and
 - **german** (“The German Credit Data Set”), a data set of 1,000 southern German borrowers from the 1970s. (the R system library contains these dataset in the library ‘datasets’)



2.2.1 Classification of this course in the data science process

The course, *Data Science Tools (DST)* focuses on the steps marked with **X**. Modeling, evaluation, and prediction are primarily covered in the courses *Data Science Concepts (DSC)* or *Data Science Methods (DSM)*; whether these courses will still be offered in one of the following semesters is currently (September 19, 2025) unclear.

The topics *Business Problem Analysis*, *Data Collection*, and *Communication* are best learned in a seminar (e.g., *Data Science Challenge*) in which a practical problem is solved using **R** and appropriate data science methods.

The correct way to learn these steps, together with the steps *decision* and *deployment*, is in a company or government agency, or, with a great deal of luck, during an internship.

2.2.2 Course content

1. Tools (80%)
 - Basics of the statistical software environment **R** and **RStudio**
 - Data visualization
 - Data preparation and data structures
 - Functional data analysis
2. Statistical methods (15%)
 - Basic statistical tools
 - Supervised learning, illustrated using Naive Bayes
 - Evaluation and receiver operating characteristic (ROC) (This may change)
3. Supplements (5%): In WS 2025/26, possibly basics of relational databases or the use of LLM with R.

2.2.3 Learning objectives of the course

Upon completion of the course, each participant should be able to

- a. using **R**, **RStudio**, and relevant **R** libraries from the *tidyverse*, as well as
- b. using simple statistical methods
- c. statistically evaluate business-relevant information from data sets, SQL databases, and the Internet, and
- d. interpret the results in a meaningful way.

¹Due to the numerous libraries, there may be name conflicts; their resolution will be discussed on a case-by-case basis.

2.2.4 Prerequisites for this course

2.2.4.1 Mandatory requirements for this course

- Mathematics for Economists
- Descriptive Statistics and Probability Theory (Statistics I) and
- Inductive and Multivariate Statistics (Statistics II)
- Computer-assisted statistics with R or, alternatively, the R crash course at the beginning of the winter semester 2025/26

2.2.5 Structure and learning format of the course

Lectures and exercises are integrated into this course. Unlike in previous years, this course does not offer an installation date for **R** and **RStudio**, as this was covered in the *R crash course* at the beginning of the winter semester. Accordingly, this course will only cover the installation of course-relevant **R** libraries (primarily from the *tidyverse* (max. 5%).

The timing of lectures and exercises will be arranged as needed, i.e., lectures (teaching of material) and exercises (with R) will mainly take place on Wednesdays, and possibly also on a some few Fridays.

2.3 Organization of the course

2.3.1 Times and organization (*NON-BINDING*)

- Wednesday in the 1:00 p.m. to 6:00 p.m. slot in the form of an integrated lecture and exercise. If necessary, some canceled Tuesday slots will be replaced by Friday appointments.
- Changes will be announced via the **KU Campus System** (campus.ku.de²), where all specific dates are also stored.
- Registration in the **KU Campus System** (campus.ku.de³) is **REQUIRED**.
- Registration in the **KU ILIAS system** (KU ILIAS⁴) is **REQUIRED**. The link can be found in the KU Campus System.

IMPORTANT: In the KU Campus, no distinction is made between lectures and exercises. In fact, (very few) exercises are interspersed as needed.

²<https://campus.ku.de/>

³<https://campus.ku.de/studienangebot/>

⁴<https://elearn.ku.de/>

2.3.2 Examination

The exam form is still open: 1. It might be a written exam, probably lasting 60 or 90 minutes. It will be held in person at the end of the winter semester; only entries in the KU Campus system are binding. There will be only **one** exam in the first exam period of the winter semester.

2.3.3 ILIAS group

The course [Data Science Tools (WS 2025/26)] has been set up on the **Ilias-elarn-KU learning platform of KU Eichstätt-Ingolstadt**⁵(https://elarn.ku.de/TODO&client_id=elarnKU) has been set up on the **Ilias-elarn-KU learning platform of KU Eichstätt-Ingolstadt**⁶, where supplementary documents and materials are stored.

2.3.4 Attendance

Not required for examination purposes! I recommend regular attendance.

Important note: Read, learn, recap, practice (independently), apply!

Participating in the course **only makes sense** if you learn as follows:

1. Type in and work through all the examples yourself on your computer at home.
2. Actively solve exercises from the specified literature.
3. Defines and solves their own analysis tasks.
4. Intensively study the literature on the subject.
5. intensively engage with
 - the software tools,
 - the issues and
 - statistical methods.

2.3.5 Actual workload in the winter semester

- 45 hours = course preparation (careful reading of the specified literature and programming)
- 45 hours = attendance time for lectures and exercises
- 30 hours = preparation and follow-up work
- 30 hours = exam preparation

⁵<https://elarn.ku.de/>

⁶<https://elarn.ku.de/>

TOTAL: 150 hours = total workload in hours (1 hour = 60 minutes)

2.4 Preliminary detailed overview of course content

- Overview of the course, organization, and exam
 - Focus: data- and tool-oriented
 - Sequence: data, graphics, data preparation, methods (concept and application)
- Introduction to R, RStudio, and help systems
- Introduction to R libraries
- Relevant IT standards
- Installation of R libraries and introduction to R and RStudio
- Introduction to RMarkdown
- Presentation of two data sets
 - Airlines data set in variant `nycflights13`
 - Credit scoring Germany data set `GermanCredit`
- Basic ideas of R-based graphics in `graphics` and `lattice`
- Basic concepts of `ggplot2` and the *Grammar of Graphics*
 - Simple `ggplot2` examples
 - `ggplot2`: Layers, aesthetics, geoms, scales, coordinate systems, facets, legends, and themes
- The *Tidyverse*, especially `tidyr` and `dplyr`
 - `tibble`, `select`, `filter`, `mutate`, `transmute`, `rename`, and `arrange`
 - `summarise`, `group_by`
 - `gather`, `spread`, and their successors `pivot_longer` and `pivot_wider`
 - `unite`, `separate` or `separate_rows`
 - `inner_join`, `left_join`, `right_join`, `full_join`, `anti_join`, and `by`
 - Exercise with data set `nycflights13`
 - Iteration, especially `base-R` and `purrr`
 - `apply`, `lapply`, `sapply`, `tapply`, and `mapply`
 - `map*`
 - `(dplyr::do` and `mosaic::do)`
- Model analysis with `broom`, among others
 - `tidy`
 - `glance`
 - `augment`

- Reading data
 - CSV and Excel files
 - `read_csv2` and `write_csv2`
 - `read_xlsx` and related functions
 - Variable definitions
 - Variable types including extensions for factors/characters and date/time/POSIXct/POSIXlt; *tidyverse* packages
 - * `forcats`
 - * `lubridate`
 - * `stringr`
- Accessing `mySQL` databases from R
- Technical exercises:
 - Data import and export, linear analysis (descriptive univariate and bivariate statistics)
 - Exercise for `german_credit`
 - Exercise using advanced examples: combining commands from the *Tidyverse*
 - Exercise with `nycflights13` (including comparison with `airlines`)
- Discrimination and classification
 - Confusion matrix, evaluation measures, and ROC
 - Recapitulation
 - Naive Bayes
 - Cross-validation (subject to change)
- Component methods
 - Eigenvalues
 - Principal components
 - Singular value decomposition (subject to change)
- Repetition
- Exam briefing

Chapter 3

Software

Preliminary note: Additional links are provided in the link list in the appendix A1.

The **R** interpreter is used

- for data preparation,
- to carry out statistical data analyses,
- to create statistical graphics and
- for programming mathematical-statistical algorithms.

In addition to command line tools, the **R** interpreter also contains an **RGui**, a simple graphical user interface (GUI). The **RGui** is only suitable for small data analyses.

RStudio, on the other hand, is used as a central development environment

- for carrying out extensive data analyses,
- for the creation of **R** programs,
- for creating documentation (texts, slides etc.) e.g. with **RMarkdown** or **Quarto**
- for version management, e.g. with GitHub and
- for freezing environments using **renv** similar to virtual environments in Python.

In this course we will mainly use the comfortable and stable **RStudio**. In exceptional cases we use the **RGui** *additionally* as GUI when

the **RGui** has advantages.

There are numerous **R** libraries (called *packages*) available with which a very large range of complex statistical problems can be solved. On **CRAN**¹ you can find an organized classification of the libraries under **Task Views**² and a list of all available packages under **Packages**³.

On September 18, 2025, according to R Package Documentation⁴, there were 28,465 packages of considerably varying quality on CRAN, with a further 84,000 or so packages available on other platforms. A whole series of packages are excellent and need not shy away from comparison with commercial software. These include, in particular, those for *R-base*, *MASS* and all libraries belonging to the Tidyverse⁵, Tidymodels⁶ and the Tidyverts⁷.

A side note: R⁸ is based on the LISP dialect Scheme⁹, which was mainly used in computer science education in the 1980s, in addition to the S¹⁰ system developed at Bell Labs¹¹ since 1976. The book by Abelson et al. (1979), which is still worth reading and is now available in a second edition from 1996 as a PDF version¹², contains many fundamentally important procedures and concepts of procedural, list-oriented and object-oriented programming. Additionally R uses many features of APL with field orientation (matrices and arrays = tensors, see Iverson (1962) and the Wikipedia entry on APL¹³) and nested arrays of APL2 (see Brown et al. (1988)). An overview of programming paradigms can be found in Wikipedia in the entry Programming paradigm¹⁴.

3.1 Reference sources

R and **RStudio** are available for *Windows*, *macOS* and *Linux*. The most flexible environment is under *Linux*. In this course we primarily work with R version 4.5.1 under Windows¹⁵.

¹<https://cran.r-project.org>

²<https://cran.r-project.org/web/views/>

³<https://cran.r-project.org/web/packages/index.html>

⁴<https://rdrr.io/>

⁵<https://www.tidyverse.org/>

⁶<https://www.tidymodels.org/>

⁷<https://tidyverts.org/>

⁸[https://de.wikipedia.org/wiki/R_\(programming%20language\)](https://de.wikipedia.org/wiki/R_(programming%20language))

⁹[https://en.wikipedia.org/wiki/Scheme_\(programming_language\)](https://en.wikipedia.org/wiki/Scheme_(programming_language))

¹⁰[https://en.wikipedia.org/wiki/S_\(programming_language\)](https://en.wikipedia.org/wiki/S_(programming_language))

¹¹https://en.wikipedia.org/wiki/Bell_Labs

¹²<https://web.mit.edu/6.001/6.037/sicp.pdf>

¹³[https://en.wikipedia.org/wiki/APL_\(programming_language\)](https://en.wikipedia.org/wiki/APL_(programming_language))

¹⁴https://en.wikipedia.org/wiki/Programming_paradigm

¹⁵<https://cran.r-project.org/bin/windows/base/R-4.5.1-win.exe>

R is an open source and freely available software; essential information can be found at R-Project for Statistical Computing¹⁶. The software can be downloaded from the **Comprehensive R Archive Network** (CRAN)¹⁷.

RStudio is offered in various versions by the company Posit¹⁸, the successor company of RStudio.com. The basic version, which is distributed under the AGPL v3 license, can be downloaded from RStudio Desktop¹⁹ in the version RStudio Desktop 2024.12.1-563 - Windows 10/11 (64-Bit)²⁰.

The company *Posit* was formerly named *RStudio*. The name change reflects that Posit also includes Python services beyond R services. Therefore a successor of RStudio (still under development) named Positron²¹ has been developed by Posit which simplifies the fast change between Python and R. While promising, Positron has still inferior features compared to RStudio. Therefore we will stay with RStudio in this course.

There was also a version *Microsoft R Open* created by the company **Microsoft** and also available as open source, which was characterized by some additions. Among other things, the *Math Kernel Libraries (MKL)* are included, which often calculate faster, especially with algorithms operating in parallel. However, the last Microsoft R Open version is no longer current, as Microsoft announced²² on June 30, 2021 that it will only support the regular R version of CRAN with libraries in the future. The last Microsoft R Open version has the number 4.0.2, while R on CRAN is now already available in version 4.5.1.

¹⁶<https://www.r-project.org>

¹⁷<https://cran.r-project.org/>

¹⁸<https://posit.co/>

¹⁹<https://posit.co/download/rstudio-desktop/>

²⁰<https://posit.co/download/rstudio-desktop/>

²¹<https://posit.co/products/ide/positron/>

²²<https://cloudblogs.microsoft.com/sqlserver/2021/06/30/looking-to-the-future-for-r-in-azure-sql-and-sql-server/>

Chapter 4

Selected Literature

4.1 Primary literature

- Baumer, B.S., Kaplan, D.T. and Horton, N.J. (2021): *Modern Data Science with R*. 2nd edition. CRC Press.

IMPORTANT NOTICE FROM OCTOBER 11, 2021:

Possibly only applicable to used books.

On March 31, 2021, the second edition of the above book was published (see Baumer et al. (2021)). The book has been revised, especially in the back part. In the front part you will mainly find updates to the so-called functional **apply** functions, which have been replaced to a considerable extent by functions from the Tidyverse libraries **purrr** and **map**. In this respect, the second edition is significantly more up-to-date than the first edition.

The problem with the second edition, however, is that there are misprints in circulation in which the color-printed illustrations and the R source texts reproduced with syntax coloring are only reproduced in grayscale or in extremely poor color quality – in fact, unreadable. This applies not only to the prints, but also to individual eBooks. All copies purchased in 2011 by the Eichstätt-Ingolstadt University Library as well as the copy I bought from Amazon.de¹ on September 27, 2021 and returned show these deficits. Only the copy I received directly from CRC was legible. In the meantime, most copies, whether print or eBook, should be flawless. But you still have to be careful with antiquarian copies.

¹<https://www.amazon.de/gp/product/0367191490>

The second edition is also available on-line at Modern Data Science with R², but there is no guarantee of persistence for on-line sources. And the exact citation of on-line sources is also more difficult. For advanced applications (e.g. flex dashboards, geospatial computations), the second edition is definitely recommended.

In addition, a preliminary draft of the third edition³ has been available since summer 2023, but a number of sections are still missing or still need to be revised. Therefore we still using the second edition for this course in the winter term 2025/26. However, there are R codes in the second edition that are no longer up to date. This concerns, for example, the reading of Wikipedia entries on Japanese nuclear reactors, as the Wikipedia format has changed. This has not yet been corrected in the preliminary draft of the third edition.

4.2 Secondary literature

- Wickham, H. and Golemund, G. (2023): *R for Data Science - Import, tidy, transform, visualize, and model data*. O'Reilly. Second edition. Also available online⁴.
- Wickham, H. and Golemund, G. (2018): *R für Data Science: Daten importieren, bereinigen, umformen, modellieren und visualisieren*. O'Reilly Germany. (English original version of the *first* edition by Wickham, H. and Golemund, G. (2016), outdated in several sections).
- Chang, W. (2019): *R Graphics Cookbook - Practical Recipes for Visualizing Data*. O'Reilly. Second edition. Available online an an updated version⁵ dated 2025-06.26

4.3 Literature about tools

- Gandrud, Ch. (2020): *Reproducible Research with R and RStudio*. CRC. Third edition.
- Hillebrand, J. and Nierhoff, M.H. (2015): *Mastering RStudio - Develop, Communicate, and Collaborate with R*. Packt.
- Wickham, H. (2016): *ggplot2 - Elegant Graphics for Data Analysis*. 2nd ed, Springer. Now somewhat outdated. A forthcoming third edition is available online as an incomplete ggplot2: Elegant Graphics for Data Analysis (3e)⁶ draft.

Note: Figures from the books cited above were occasionally taken from the corresponding PDF files.

²<https://mdsr-book.github.io/mdsr2e/>

³<https://mdsr-book.github.io/mdsr3e/>

⁴<https://r4ds.hadley.nz/>

⁵<https://r-graphics.org/>

⁶<https://ggplot2-book.org/>

4.4 Literature about R and Python

- Ohri, A. (2018): *Python for R Users - A Data Science Approach*. Wiley.
- Zhang, N. (2021): *A Tour of Data Science - Learn R and Python in Parallel*. CRC Press.
- Scavetta, R.J. & Angelov, B. (2021): *Python and R for the Modern Data Scientist – The Best of Both Worlds*. O'Reilly.

These books are particularly interesting for those participants who already know Python or (have to) learn it as part of another course. Statisticians often prefer R to Python for various reasons (see e.g. Quora article [What would it take to make academic statisticians switch from R to Python⁷](https://www.quora.com/What-would-it-take-to-make-academic-statisticians-switch-from-R-to-Python)), while computer scientists with a focus on AI (e.g. deeply nested neural networks, image processing by convolution networks, speech processing by recurrent nets and quantum computing) tend to prefer the other way around. Most professional data scientists know R as well as Python and SQL. For computational intensive tasks some other programming languages like C++ or Fortran are used. R offers some simple interfaces to integrate C++, Fortran and Python codes for parts of the computation.

4.5 Literature on R as a programming language

The following two books delve very deeply into the R programming language and are therefore recommended for anyone who (wants to or has to) use R as a primary programming language:

- Wickham, H. (2019): *Advanced R*. CRC Press, 2nd edition.
- Grosser, M., Bumann, H. and Wickham, H. (2022): *Advanced R Solutions*. CRC Press.

Grosser et al. (2022) contains all solutions to the sometimes complicated tasks in Wickham (2019).

4.6 Literature about R and SQL

- Badia, A. (2020): *SQL for Data Science - Data Cleaning, Wrangling and Analytics with Relational Databases*. Springer.

There are many other excellent texts for SQL available but the book cited above focuses on Data Science issues.

⁷<https://www.quora.com/What-would-it-take-to-make-academic-statisticians-switch-from-R-to-Python/answer/Attila-Marossy>

4.7 Operating systems

Sometimes you need additional information about operating systems. Linux is an ideal environment for **R** and is also used on the Linux parallel cluster of the Leibniz Supercomputing Center (LRZ) of the Bavarian Academy of Science⁸. If you use the *RStudio interface* available in the highly secure LRZ-AI system in the Datalab as an LRZ-RStudio web service, you can get by with very rudimentary Linux knowledge.

This also applies to the various RStudio cloud services available, such as the EC2 RStudio servers in the Amazon Marketplace⁹.

For batch applications, however, there is no getting around Linux, for which I recommend **the** standard work:

- Kofler, M. (2023): *Linux: Das umfassende Handbuch*. 18th updated edition, Rheinwerk Computing. (there is an english translation available)

4.8 Reference works

- R documentation (accessible through `help()` and `help.start()`)
- Horton, N.J. and Kleinman, K. (2015): *Using R and RStudio for Data Management, Statistical Analysis and Graphics*. CRC Press. Second edition. Mainly useful for traditional, i.e. base R

4.9 Personal recommendation for purchase

Baumer et al. (2021), Chang (2023) and Gandrud (2020). Possibly Kofler (2023), the latter especially if you are switching to Linux anyway.

4.10 Course accompanying materials

In addition to the above-mentioned literature, course notes (such as this text) will also be made available. These course notes were created with *RMarkdown*¹⁰ and represent a mixture of texts, R code segments embedded via R chunks¹¹, tables, R output and graphics.

⁸<https://www.lrz.de>

⁹<https://aws.amazon.com/marketplace>

¹⁰<https://rmarkdown.rstudio.com/>

¹¹The English term *chunk* means something like a block of data, a short statement or, in psycholinguistics, a block of linguistic information for more effective use of short-term memory. In the context of R and R markdown, R chunks are (usually) short R code segments that are used to perform a brief calculation or to create a graphic. However, since there is no limit to the size of the R code segments in R markdown, you can also write extremely long R code in a single R

4.11 Online sources

- R-Project¹², especially R-Manuals¹³ for base R.
- CRAN¹⁴ for a wide range of packages and tasks views.
- Posit (formerly RStudio)¹⁵, especially Cheatsheets¹⁶ *cheat sheets*, meaning reference cards.
- Tidyverse¹⁷ for data science tools.
- Tidymodels¹⁸ for data science methods.
- Tidyverts¹⁹ for time series forecasting.
- MLR3²⁰ for machine learning (as well as its predecessor MLR²¹)
- RDocumentation²²
- R Package Documentation²³

In general: *Google* for a command or an error message (augment with [R]). And obviously the large range of chat engines for Large Language Models (LLM) like Copilot²⁴.

Warning: There are numerous alternative and different solution paths in **R** for most statistical problems, which are supported by various libraries, some of which vary greatly in quality, stability and continuity. We will concentrate on the very mature *Tidyverse*, but will also work with *older* or *competing* concepts and commands mainly from Base-R.

By the way, for many R packages you can find complementary scientific books and papers explaining their use and interpretation. This is also addressed in the R vs. Python debate, see the Quora posts What would it take to make academic statisticians switch from R to Python²⁵ and Why I Prefer Python to R for Data Work²⁶, among others, the latter of which contains a number of criticisms that

chunk – but of course these chunks are not readable.

¹²<https://www.r-project.org/>

¹³<https://cran.r-project.org/manuals.html>

¹⁴<https://cran.r-project.org/>

¹⁵<https://posit.co/>

¹⁶<https://posit.co/resources/cheatsheets/>

¹⁷<https://www.tidyverse.org>

¹⁸<https://www.tidymodels.org/>

¹⁹<https://tidyverts.org/>

²⁰<https://mlr3.mlr-org.com/>

²¹<https://mlr.mlr-org.com/>

²²<https://www.rdocumentation.org>

²³<https://rdr.io/>

²⁴<https://copilot.microsoft.com/>

²⁵<https://www.quora.com/What-would-it-take-to-make-academic-statisticians-switch-from-R-to-Python/answer/Attila-Marossy>

²⁶<https://zpz.github.io/blog/prefer-python-for-data-work/>

I disagree with²⁷. The focus of Python is also not on statistics, as you can see from the Python Package Index³¹ and the Python Packages³² page of the *Geeks for Geeks*³³ homepage, which is interesting for computing anyway (beyond R and Python).

4.12 Online help and further information

Help is available within **R** and **RStudio** through the internal help pages. Also helpful are:

- The above mentioned *Cheat Sheets* from Posit³⁴ for the GUI, libraries and the **R** language in general.
- R-bloggers³⁵, a site with numerous informative articles.
- Stackoverflow³⁶ with numerous questions and answers about **R**.
- R-Seek³⁷, a search engine for **R**.

See also the link list in appendix A.

²⁷The wildest argument is to call the R developers of the R Software Foundation²⁸ amateur programmers because they largely come from statistics and have thus created a FUBU (for-users-by-users) system. This core group includes people such as John Chambers²⁹ (winner of the ACM Software System Award) and Hadley Wickham³⁰, who is the main developer of **ggplot2** and the **tidyverse** as well as chief scientist of Posit.

³¹<https://pypi.org/>

³²<https://www.geeksforgeeks.org/python-packages/>

³³<https://www.geeksforgeeks.org/>

³⁴<https://posit.co/resources/cheatsheets/>

³⁵<https://www.r-bloggers.com/>

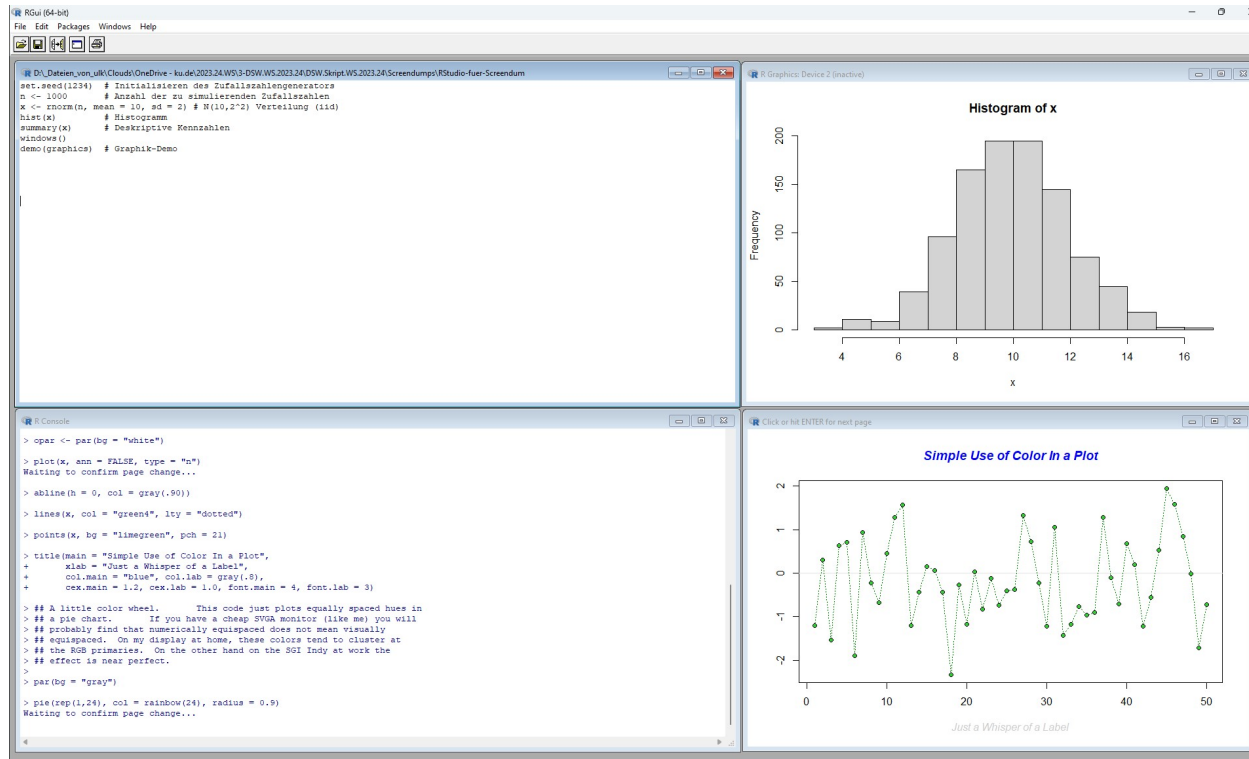
³⁶<https://stackoverflow.com/questions/tagged/r>

³⁷<https://rseek.org/>

Chapter 5

RGui

The **R** installed under *Windows* (similar under *MacOS* and *Linux*) contains a simple *Graphical User Interface* (GUI) for elementary data analysis and programming; the following



The GUI known as *RGui* consists of the following components:

Permanent

1. Header bar (with common and partly panel-specific elements such as File, Edit, View, Help, Miscellaneous, Packages, Windows, Help) and context-sensitive buttons (e.g. **STOP** in console mode).
2. Console (bottom left) for program input and output.

Optional

3. Simple editor (create new script with file →), R commands are transferred to the console with Ctrl-R and executed.

4. Graphics windows (are opened implicitly with graphics commands, explicitly with R command `windows()`).

The R interpreter can be installed in English or German; in the English version, of course, the labels change, e.g. *File* instead of *Datei*.

5.1 Important R commands for help (within R)

R is a very complex piece of software in which you can only find your way around if you use the various help functions.

In the following list, only some of the function calls that can be entered in the console are shown as examples. The corresponding result is always suppressed and in some cases (e.g. with `help.search("lm")`) cannot be embedded in R Markdown at all, as a dedicated panel (for `help.search("lm")`, for example, the *Help* panel in RStudio) is used for output.

The console is available in R command line mode, in *RGui* as well as in the *RStudio* development environment described below.

The `library` function is used to display all installed R packages and localize all library paths:

```
> library()
```

If you use the function `library` with a package name as the first argument, the corresponding library is loaded and placed in the search path directly after the global environment. For example, the following function call loads the R package `mlr` (a very extensive collection of machine learning algorithms, and even more in the successor package `mlr3`,).

```
> library(mlr)
```

Calling `help` with a function name as argument returns a function-specific help, here for `lm`:

```
> help("lm")
```

The function `help.search` is used to search the very extensive R help for a term. The following function call to search for `print` functions returns a very comprehensive result:

```
> help.search("print")
```

The arguments of a function¹ are displayed with `args`, here for `matrix`:

```
> args("matrix")
```

And the body of a function is displayed with `body`, here for `matrix`:

¹The central elements belonging to a function can be extracted with `formals`, `body` and `environment`.

```
> body("matrix")
```

With `example` the examples given in the documentation for a function are executed; here for `mean`:

```
> example("mean")
```

The quotation marks can often be omitted, e.g. `example(mean)` returns the same result as `example("mean")` .

With `vignette` all vignettes of the packages installed on the running system (with detailed explanations) are displayed:

```
> vignette()
```

If you also enter the vignette name (sometimes, but not always the library name²) in `vignette`, you will receive the corresponding vignette directly. Here is an example for the Tidyverse library `dplyr`:

```
> vignette("dplyr")
```

If no vignette exists for a library, a warning message indicating the lack of availability is displayed in the console, e.g. with `vignette("tidyverse")`.

With `apropos`, all available and *visible* objects *within the search path* that use the object name specified in the argument or contain it as a character string are listed. Here as an example `lm`:

```
> apropos("lm")
```

The `methods` function is used both to identify class-specific functions (methods) of a specific class and to identify the class-specific functions associated with a generic function.

When specifying the `class` argument, the `methods` function outputs the names of all class-specific methods that are available for the object class named in the argument. Here is an example for `lm`:

```
> methods(class="lm")
```

If you call `methods` with a generic function name as default (`generic.function`), all methods belonging to the function argument are listed. The following call lists all methods that can be called generically via the `print` function:

```
> methods("print")
```

²There are no vignettes for most packages. In addition, some packages contain several vignettes, but not a single vignette name matches the library name. This is the case with `ggplot2`, for example, where the three very informative vignettes are named `ggplot2-specs`, `extending-ggplot2` and `ggplot2-in-packages`.

The function `where` from the R library `pryr` can be used to find both the environment and the installation directory of the library in which the symbol name specified for the default argument `name` can be found. Example: The following function call outputs both the library and the installation location of the function `tally`. Note that only the loaded packages are taken into account in the search³:

```
> pryr::where("tally")
```

The function `find` from the R package `utils` finds the package in the search path in which the symbol named as argument is available.

```
> find("tally")
```

The function `sessionInfo` from the R base package `utils` returns the encoding used locally in the current session, all libraries used, etc:

```
> sessionInfo()
```

The `session_info` function from the `devtools` package is a variant of `utils::sessionInfo` and outputs the same information in a somewhat clearer form – however, within RStudio (not within Rgui) an uncritical collision currently occurs with a parallel installation of *Quarto*:

```
> devtools::session_info()
```

If a library name (of an installed library) is used as an argument in the `session_info` function, all associated dependencies are output. Here is an example for `ggplot2`:

```
> devtools::session_info("ggplot2")
```

`library` loads a package that is placed immediately after the global environment (`.GlobalEnv`) in the search path (diagnosable with `search()`) when the library name alone is mentioned without any additional arguments. The following example loads the string processing package `stringr`:

```
> library(stringr)
```

`ls()` alone lists all symbols stored in the workspace `.GlobalEnv`; this corresponds to `ls(".GlobalEnv")`.

```
> ls()
```

If you want to list all exported symbols of a loaded package contained in the search path, you must enter the corresponding package name with the prefix “`package:`” (a colon) as an argument. For example, the following function call lists all (exported) functions

³In the context of this course, two functions with the same name `tally` are used, one from the *Tidyverse* package `dplyr`, the other from the package `mosaic`. To avoid naming conflicts, we will only ever load the *Tidyverse* libraries and use the prefix form `mosaic::tally` when using the `tally` function from `mosaic`.

available in the `stringr` package *after* loading the package with `library(stringr)`:

```
> ls("package:stringr")
```

The *RGui* IDE is rarely used, but there are some useful cases. An example is the function call `help.start()` , which invokes the HTML help with the preferred browser (in my case, Mozilla's Firefox), which is sometimes more flexible compared to the internal *RStudio* help.

Chapter 6

RStudio

6.1 RStudio screen

RStudio¹ is an extremely powerful Integrated Development Environment² (IDE) developed by Posit³, with which you develop software and carry out data analyses. Beyond this RStudio also contains tools and interfaces for documentation (e.g. RMarkdown⁴ and Quarto⁵), project management⁶, environment control⁷, version control⁸ (e.g. Github⁹), databases (via connections and libraries) and package development¹⁰. R support many tools including dashboards (Shiny¹¹¹²), interfaces to other programming languages such as Python

¹<https://docs.posit.co/ide/user/>

²https://en.wikipedia.org/wiki/Integrated_development_environment

³posit.co

⁴<https://rmarkdown.rstudio.com/>

⁵<https://quarto.org/>

⁶<https://support.posit.co/hc/en-us/articles/200526207-Using-RStudio-Projects>

⁷<https://docs.posit.co/ide/user/ide/guide/environments/r/renv.html>

⁸<https://docs.posit.co/ide/user/ide/guide/tools/version-control.html>

⁹<https://github.com/>

¹⁰<https://docs.posit.co/ide/user/ide/guide/pkg-devel/writing-packages.html>

¹¹<https://shiny.posit.co/>

¹²Recently Posit has also developed Shiny tools for Python.

using reticulate¹³, C++ using Rcpp¹⁴ and others, formatting systems like LaTeX¹⁵ and Markdown¹⁶ and Large Language Models (LLM)¹⁷ like Github Copilot¹⁸. All these tools are well integrated into RStudio.

Please note: Posit has recently developed an IDE called Positron¹⁹, which makes it easier to work with both R and Python simultaneously. However, a whole range of tools that are already available in a mature and stable form in RStudio are still missing, while Positron is still in its infancy (beta testing). For this reason, RStudio will still be used in this course in the 2025/26 winter semester. In the medium term, however, a switch to Positron might be a good option.

Positron emphasizes data analysis. Contrary to Positron the RStudio IDE in its current state is additionally also a very capable for the development of statistical software.

The following screen dump shows the standard elements:

¹³<https://rstudio.github.io/reticulate/>

¹⁴<https://www.rcpp.org/>

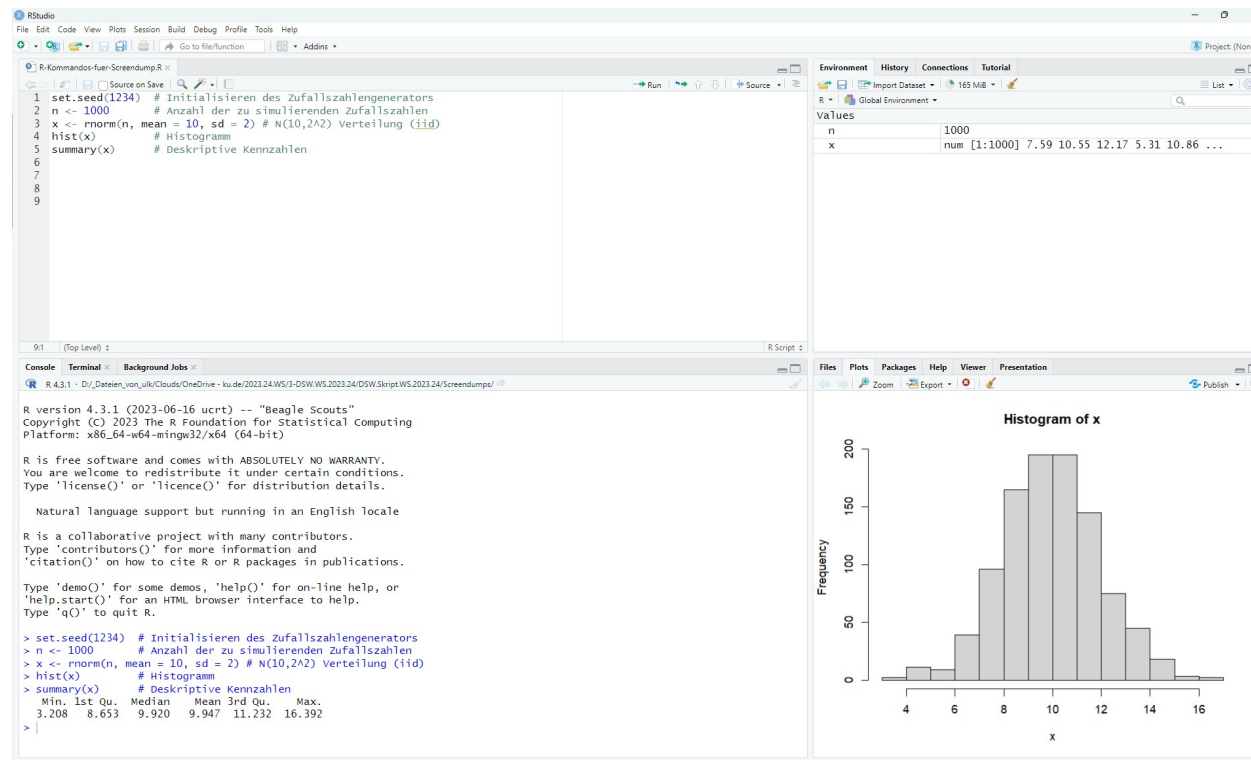
¹⁵<https://en.wikipedia.org/wiki/LaTeX>

¹⁶<https://en.wikipedia.org/wiki/Markdown>

¹⁷https://en.wikipedia.org/wiki/Large_language_model

¹⁸<https://docs.posit.co/ide/user/ide/guide/tools/copilot.html>

¹⁹<https://positron.posit.co/>



6.2 RStudio elements

RStudio has numerous shortcuts. However, a small subset of all shortcuts and commands is sufficient for meaningful use.

- *Header bar* (with elements such as File, Edit, Code, View, Plots, Session, Build, Debug, Profile, Tools, Help) as well as partially context-sensitive buttons under the header bar and in subwindows (e.g. the **STOP** button in the console, which is only active during computing).
- *Configurable panel fields* (Panels, can be set under *Tools* → *Global Options* → *Pane Layout*). The most important are the following

Panes, between which you can quickly switch using keyboard sequences.

1. Source (**Ctrl**+1)
2. Console (**Ctrl**+2)
3. Help (**Ctrl**+3)
4. Show history (**Ctrl**+4)
5. Show files (**Ctrl**+5)
6. Show plots (**Ctrl**+6)
7. Show packages (**Ctrl**+7)
8. Show environment (**Ctrl**+8)
9. Show viewer (**Ctrl**+9)

Attention:

Ctrl+digit means that the **Ctrl** key (**Ctrl** key) and the associated key with the corresponding digit (but **not** the function key) are pressed together.

If you want to *maximize* the corresponding window, you *usually* add **Shift**. However, this does not always work, e.g. **Ctrl**+**Shift**+1 maximizes the source code panel, but **Ctrl**+**Shift**+2 does not maximize the console, but restores the configuration with the usual fourfold window division – unless the cursor is already in the console. According to the help, the shortcut **Ctrl**+**Shift**+**Alt**+0 should actually be used to restore this window division, but this does not always work in a German-speaking Windows 10 environment. In any case, you can create the desired window division or window expansion by clicking on the four button in the header or by selecting View → Panes → Show all Panes.

Caution: *RStudio* contains a lot of keyboard shortcuts, so you can quickly cause mischief by mistake. It is therefore advisable to save the source file often and to undo any subsequent incorrect changes successively with **Ctrl**+Z. K means pressing the K key, i.e. not pressing the capital letter using **Shift**+k.

6.3 RStudio – the most important panels (panes) and keyboard shortcuts

Preliminary remark:

1. For the definition of *panes* see the Wikipedia entry List of graphical user interface elements²⁰.
2. Depending on the keyboard driver and key mapping, some keys may not work. In this case, the corresponding functions must be entered via menus.

²⁰https://en.wikipedia.org/wiki/List_of_graphical_user_interface_elements

3. The keyboard shortcuts in the list below are only a small, subjective selection of the most useful shortcuts *in my opinion*. By clicking on the field **See All Shortcuts** in the help via **Help**→**Keyboard Shortcuts Help** you will get a complete and very extensive list.

Keyboard shortcuts (selection):

- Everywhere (in all panes)
 - Shortcut Help (**Alt+Shift+K**)
 - Show all Panes (**Ctrl+Shift+Alt+0**, does not always work under German Windows 10/11)
- In the panes source code and view
 - Next tab in file or view (**Ctrl+Tab**)
 - Previous tab in file or view (**Ctrl+Shift+Tab**)
- Source (**Ctrl+1**)
 - New file with **File** → **New File** → **R Script** (**Ctrl+Shift+N**)
 - Save file under **File** → **Save** (**Ctrl+s**) or **Save as**
 - Open file under **File** → **Open File** (**Ctrl+O**)
 - Current help for the selected command (**F1**)
 - Execute current line and move forward (**Ctrl+Return**)²¹
 - Execute start to current line (**Ctrl+Alt+B**)
 - Execute current chunk in RMarkdown (**Ctrl+Shift+Return**)
 - Restart R, i.e. reinitialize R; incl. **detach** all loaded libraries (**Ctrl+Shift+F10**)
 - Assignment symbol (**Alt+-**) **<-** (Note: with RMarkdown, the equal sign **=** is inserted in the text with some RStudio versions)
 - The pipe operator **%>%** (**Ctrl+Shift+M**) frequently used in the *Tidyverse*, explanation follows later. **IMPORTANT NOTE:** Since R 4.1, the pipe operator **|>** has been introduced in **R**, which - with very few exceptions²² - has the same function as the pipe operator **%>%** defined in the tidyverse in the R package **magrittr**. If you want to generate the pipe operator **|>** implemented in **R** in *RStudio* with **Ctrl+Shift+M**, this must be set under **Tools** → **Global Options** → **Code** → **Editing**.
 - Partly as usual under Windows cursor and key control (e.g. **Ctrl+C**, **Ctrl+X**, **Ctrl+V**, **↑**, i.e. arrow-up etc.).
 - Generate HTML report from source with results (**Ctrl+Shift+K**, only for Rmd and HTML sources)
 - Execute the running script (**Ctrl+Shift+S**)
 - To gain an overview:
 - * Collapse fold (**Alt+L**)
 - * Expand fold (**Alt+Shift+L**)
 - * Collapse all folds (**Alt+0**)

²¹Usually the **Return** key is labeled **HIERALT**

²²See the blog entry Differences between the base R and magrittr pipes²³ by Wickham (2023).

- * Expand all folds (**Alt+Shift+0**). Note: Letter ‘O’, not digit ‘0’.
- Console (bottom left) for program input and output (**Ctrl+2**)
 - Direct input of R commands possible; execute with **Return**.
 - Empty console (**Ctrl+L**)
 - Scroll through the last commands (**Ctrl+↑**, **Ctrl+↓**) and embed the selected code for post-editing with **Return**.
- Help (**Ctrl+3**) allows you to view and search the help; clicking on the home icon takes you to the main window.
 - Entering `help.start()` in **RGui** (*not RStudio*) calls up more convenient help in the external web browser; *RStudio* uses the internal help window.
 - Entering `help.start()` in the **RStudio** console opens the help window and displays the URL at the same time. If you enter this URL in the browser, you will get the same help as calling `help.start()` in **RGui**.
- History (**Ctrl+4**) allows you to view previous commands and copy them to Source or Console (select and click either **To Source** or **To Console**)
- Show Files (**Ctrl+5**) allows you to browse and select files and change directories.
- Show Plots (**Ctrl+6**) allows you to view graphics, zoom and export.
- Show Packages (**Ctrl+7**) allows you to install and update packages and view the documentation by clicking on the name of an R package to call up the corresponding help pages directly.
- Environment (**Ctrl+8**) allows you to view variables and functions without entering a console.

Please make sure that you are using the correct R version (currently R 4.4.3) for this course under RStudio. You can set this under *Tools → Global Options → Basic → R-Version*.

On Posit’s cheat sheet page²⁴ you will find a scrolable RStudio IDE cheatsheet²⁵ with the most important commands as well as its PDF version²⁶.

6.4 Project Management with RStudio (tbd)

tbd

6.5 Version Control with RStudio and Github (tbd)

tbd

²⁴<https://posit.co/resources/cheatsheets/>

²⁵https://rstudio.github.io/cheatsheets/html/rstudio-ide.html?_gl=1*151v0rq*_ga*NTEzMzQyNjE3LjE3NTc3NjIyMTQ.*_ga_2C0WZ1JHG0*czE3NTg5MDcwNjkkbzgkZzEkdDE3NTg5MDcwNzQkajU1JGwwJGgw

²⁶<https://rstudio.github.io/cheatsheets/rstudio-ide.pdf>

6.6 Documentation using RMarkdown and Quarto (tbd)

tbd

Chapter 7

Data sets

7.1 Overview of four example data sets

In this course *Data Science Tools* we will only work with very few exemplary data sets:

1. **mtcars**: This is a very small dataset, so you can manually check how **tidyverse** functions work. A description can be obtained from the help function, which can be called using `help("mtcars")`.
2. **german**: This data set, dating from the 1950s and publicly available in different variants, comprises the data of 1000 borrowers and is used in this course to illustrate an elementary classification procedure (Naive Bayes) and to illustrate evaluation methods. Note that several variants of this dataset are inconsistent.
3. **nycflights13**: This R package contains five separate tables that can be used to illustrate SQL-like **tidyverse** functions very well. A first overview can be found in the help via `help(package = "nycflights13")`. Further additions can be found below. Numerous examples of this data set are covered in Wickham et al. (2023), the **standard textbook** on the *Tidyverse*.
4. A dataset on public households extracted from the GENESIS¹ database system of the Federal Statistical Office of Germany², which illustrates some principles of data preparation and data cleaning. Details are described in chapter 32.

¹<https://www-genesis.destatis.de/genesis/online>

²<https://www.destatis.de>

TBD: Example 4 might be complemented or substituted by a data sets provide by EUROSTAT.

Numerous other examples can be found in Baumer et al. (2021) and in Wickham & Grolemund (2018) and Wickham et al. (2023). Furthermore, numerous R libraries contain datasets, including the R package `datasets` (*The R Datasets Package*) of the basic installation. There is also a huge amount of publicly available and more or less well documented datasets on the Internet, e.g. at Kaggle³.

In the following, the second data set `german` is characterized and explained as an example. Subsequently, some special features of `nycflights13` are briefly outlined.

7.2 Data set `german` (or German Credit)

The dataset **German Credit** by Prof. Hans Hofmann (University of Hamburg) was deposited in 1994 on the UCI Machine Learning Repository⁴ as a CSV file⁵ together with documentation⁶. The original dataset uses non-speaking and cryptic variable names (e.g. `A3` for *credit history*) and variable values (e.g. digit 3 for *delay in paying off in the past* for variable `A3`) and therefore requires intensive processing.

The date of entry for the data record is unknown, but based on the variable values, it can be assumed that it was in the 1970s.

The Hofmann dataset has been made available by numerous authors in the past in several versions, some with considerably different variable names and variable characteristics, so that the use of an already prepared version is recommended for teaching purposes⁷. Examples are:

1. Under Freakonomics⁸ you will find the data as a CSV file, which has already been used in this form by several authors (including Charpentier⁹). This version can be read¹⁰ and saved¹¹ directly from the URL using the following commands:

³<https://www.kaggle.com/>

⁴[https://archive.ics.uci.edu/ml/datasets/statlog+\(german+credit+data\)](https://archive.ics.uci.edu/ml/datasets/statlog+(german+credit+data))

⁵<https://archive.ics.uci.edu/ml/machine-learning-databases/statlog/german/german.data-numeric>

⁶<https://archive.ics.uci.edu/ml/machine-learning-databases/statlog/german/german.doc>

⁷However, data and variable formatting can be learned very well using this data set. You can also use regular expressions from the `tidyverse` package `stringr` for the semi-automatic creation of variable names and variable values from the above-mentioned original data set description `german.doc`. However, it is not easy to find meaningful acronyms for variable values (labels) such as “no credits taken/all credits paid back duly”.

⁸http://freakonometrics.free.fr/german_credit.csv

⁹<https://www.r-bloggers.com/classification-on-the-german-credit-database/>

¹⁰*Caution:* The URL can only be read under three conditions: (a) You are connected to the Internet. (b) The Freakonomics server is active. (c) The file is still available at the URL.

¹¹Data and especially URLs change frequently. It is therefore recommended that all information downloaded from the Internet, especially data and its description, is backed up locally and protected against overwriting. This backup is not yet guaranteed by the R code below. Often it makes sense to separate the import of data and basic processing from the actual data analysis, which is then based on locally stored files.

```

> url <- "http://freakonometrics.free.fr/german_credit.csv"
> credit <- read.csv(url, header = TRUE, sep = ",")
> #write.csv(credit, file = "./data/german_credit/freakonometrics/credit.csv")
> #credit <- read.csv(file = "./data/german_credit/freakonometrics/credit.csv")
> dplyr::glimpse(credit)
Rows: 1,000
Columns: 21
$ Creditability      <int> 1, 1, 1, 1, 1, 1~
$ Account.Balance    <int> 1, 1, 2, 1, 1, 1~
$ Duration.of.Credit.month. <int> 18, 9, 12, 12, 1~
$ Payment.Status.of.Previous.Credit <int> 4, 4, 2, 4, 4, 4~
$ Purpose            <int> 2, 0, 9, 0, 0, 0~
$ Credit.Amount      <int> 1049, 2799, 841, ~
$ Value.Savings.Stocks <int> 1, 1, 2, 1, 1, 1~
$ Length.of.current.employment <int> 2, 3, 4, 3, 3, 2~
$ Instalment.per.cent <int> 4, 2, 2, 3, 4, 1~
$ Sex...Marital.Status <int> 2, 3, 2, 3, 3, 3~
$ Guarantors         <int> 1, 1, 1, 1, 1, 1~
$ Duration.in.Current.address <int> 4, 2, 4, 2, 4, 3~
$ Most.valuable.available.asset <int> 2, 1, 1, 1, 2, 1~
$ Age..years.        <int> 21, 36, 23, 39, ~
$ Concurrent.Credits <int> 3, 3, 3, 3, 1, 3~
$ Type.of.apartment  <int> 1, 1, 1, 1, 2, 1~
$ No.of.Credits.at.this.Bank <int> 1, 2, 1, 2, 2, 2~
$ Occupation         <int> 3, 3, 2, 2, 2, 2~
$ No.of.dependents   <int> 1, 2, 1, 2, 1, 2~
$ Telephone          <int> 1, 1, 1, 1, 1, 1~
$ Foreign.Worker     <int> 1, 1, 1, 2, 2, 2~

```

2. Under Kaggle¹² you will find several variants of this data set with different descriptions. The associated data description¹³ contains a spreadsheet presentation of the data with the most important key figures and graphs. Note that the data on Kaggle includes the variable `foreign_worker`, where the majority is coded as `yes`. This is unreasonable, as can be seen from the analysis under point

¹²<https://www.kaggle.com>

¹³<https://www.kaggle.com/guilhermepires/credit>

3.

Downloading the data from Kaggle data is tedious for two reasons:

- a. Registration with Kaggle is required.
 - b. As far as I know (October 27, 2025), there is no official Kaggle API available under CRAN¹⁴.
3. In the library `rchallenge` the data set is also available as dataframe `german` in the R package `rchallenge`. This prepared data set has two essential and decisive advantages:
- a. Nominal scaled variables are available in the data set as unordered factor variables (`factor`). This facilitates the initial preparation¹⁶.
 - b. The dataset `rchallenge::german` contains important corrections by Grömping (2019)¹⁷, who proved that several English-language variants of the original dataset from the 1950s were incorrectly coded due to misinterpretations of the German-language variable names and variable characteristics. These errors were eliminated in the dataframe `german` in the R package `rchallenge`. However, there are numerous analyses in circulation based on other data formats that were created on the basis of incorrect variants. These analyses are therefore also incorrect.

```
> german <- rchallenge::german # imports german without loading rchallenge
> names(german)
[1] "status"           "duration"
[3] "credit_history"    "purpose"
[5] "amount"           "savings"
[7] "employment_duration" "installment_rate"
[9] "personal_status_sex" "other_debtors"
[11] "present_residence" "property"
[13] "age"              "other_installment_plans"
[15] "housing"          "number_credits"
[17] "job"              "people_liable"
```

¹⁴Under Github you can find an R client for Kaggle's API that I have not yet tested, see <https://github.com/mkearney/kaggler>. Another, unofficial and also untested solution can be found at R API for Kaggle Dataset¹⁵.

¹⁶Caution is advised with factor variables. Classic statistical R functions such as `lm` and `anova`, which use the formula interface, use the properties of the `factor` class in a positive sense, as suitable design matrices are constructed automatically. However, problems repeatedly occur, especially when analyzing extensive data records that are linked via keys (*keys*) – particularly when date and time information is included. For this reason, factor variables are nowadays only used in the modelling phase when analyzing large data sets; in data administration and exploratory data analysis, on the other hand, nominally scaled variables are often left in their original state as character strings (`character`), although this requires more storage space.

¹⁷http://www1.beuth-hochschule.de/FB_II/reports/Report-2019-004.pdf

```

[19] "telephone"          "foreign_worker"
[21] "credit_risk"
> glimpse(german)
Rows: 1,000
Columns: 21
$ status          <fct> no checking account, no ch~
$ duration        <int> 18, 9, 12, 12, 12, 10, 8, ~
$ credit_history  <fct> all credits at this bank p~
$ purpose         <fct> car (used), others, retrain~
$ amount         <int> 1049, 2799, 841, 2122, 217~
$ savings         <fct> unknown/no savings account~
$ employment_duration <fct> < 1 yr, 1 <= ... < 4 yrs, ~
$ installment_rate <ord> < 20, 25 <= ... < 35, 25 <~
$ personal_status_sex <fct> female : non-single or mal~
$ other_debtors    <fct> none, none, none, none, no~
$ present_residence <ord> >= 7 yrs, 1 <= ... < 4 yrs~
$ property        <fct> car or other, unknown / no~
$ age             <int> 21, 36, 23, 39, 38, 48, 39~
$ other_installment_plans <fct> none, none, none, none, ba~
$ housing         <fct> for free, for free, for fr~
$ number_credits  <ord> 1, 2-3, 1, 2-3, 2-3, 2-3, ~
$ job            <fct> skilled employee/official,~
$ people_liable   <fct> 0 to 2, 3 or more, 0 to 2,~
$ telephone       <fct> no, no, no, no, no, no, no~
$ foreign_worker  <fct> no, no, no, yes, yes, yes,~
$ credit_risk     <fct> good, good, good, good, go~

```

Only the data set `german` from the R package `rchallenge` is required, but not the other functions and variables. The assignment `german <- rchallenge::german` avoids having to load the `rchallenge` package. Loading the `rchallenge` package with `library(rchallenge)`, on the other hand, would result in all symbols from `rchallenge` being visible, including those that are not required.

The dataframe `german` has 1000 observations and 21 variables.

7.3 Analysis with the R method summary

An initial overview can be obtained with the `summary` function, which is part of the R base system:

```
> summary(german)
```

status	
no checking account	:274
... < 0 DM	:269
0<= ... < 200 DM	: 63
... >= 200 DM / salary for at least 1 year	:394

duration	
Min.	: 4.0
1st Qu.	:12.0
Median	:18.0
Mean	:20.9
3rd Qu.	:24.0
Max.	:72.0

credit_history	
delay in paying off in the past	: 40
critical account/other credits elsewhere	: 49
no credits taken/all credits paid back duly	:530
existing credits paid back duly till now	: 88
all credits at this bank paid back duly	:293

purpose		amount	
furniture/equipment	:280	Min.	: 250
others	:234	1st Qu.	: 1366
car (used)	:181	Median	: 2320
car (new)	:103	Mean	: 3271
retraining	: 97	3rd Qu.	: 3972


```

repairs          : 50   Max.   :18424
(Other)          : 55

      savings      employment_duration
unknown/no savings account:603  unemployed      : 62
... < 100 DM      :103   < 1 yr      :172
100 <= ... < 500 DM      : 63   1 <= ... < 4 yrs:339
500 <= ... < 1000 DM     : 48   4 <= ... < 7 yrs:174
... >= 1000 DM         :183   >= 7 yrs      :253

      installment_rate
>= 35      :136
25 <= ... < 35:231
20 <= ... < 25:157
< 20      :476

      personal_status_sex
male : divorced/separated      : 50
female : non-single or male : single:310
male : married/widowed        :548
female : single                : 92

      other_debtors      present_residence
none      :907   < 1 yr      :130
co-applicant: 41   1 <= ... < 4 yrs:308
guarantor  : 52   4 <= ... < 7 yrs:149
              >= 7 yrs      :413

```

```

                                property
unknown / no property          :282
car or other                    :232
building soc. savings agr./life insurance:332
real estate                    :154

    age      other_installment_plans      housing
Min.   :19.00  bank   :139                for free:179
1st Qu.:27.00  stores: 47                rent   :714
Median :33.00  none   :814                own    :107
Mean   :35.54
3rd Qu.:42.00
Max.   :75.00

number_credits
1      :633
2-3    :333
4-5    : 28
>= 6   : 6

                                job
unemployed/unskilled - non-resident : 22
unskilled - resident                :200
skilled employee/official            :630
manager/self-empl./highly qualif. employee:148

    people_liable      telephone
3 or more:155    no      :596

```

```
0 to 2 :845 yes (under customer name):404
```

```
foreign_worker credit_risk
yes: 37        bad :300
no :963        good:700
```

Some variables (`savings`, `employment_duration`, `status`) are strictly speaking ordinal variables that can be converted into ordinal-scaled variables using `ordered` or `as.ordered`. As we do not use any special methods for ordinal-scaled variables in this course, this transformation is not used here.

The variable values can also be extracted from the documentation using the string functions available in the R base system and in the Tidyverse-library `stringr`. Many of these string functions are based on regular expressions¹⁸.

7.4 Analysis with the R package `mosaic`

Alternatively, you can use the `tally` function for discrete variables and the `favstats` function from the R package `mosaic`¹⁹ for continuous variables; `tally` also allows several options, some of which are illustrated with the following calls²⁰. The main textbook for this course (Baumer et al. (2021)) mainly uses functions from `mosaic` for description. Therefore, these functions are also used occasionally in the following chapters.

¹⁸https://en.wikipedia.org/wiki/Regular_expression

¹⁹Without loading the library, this is done by calling `mosaic::tally` or `mosaic::favstats`. Alternatively, the functions `tally` and `favstats` can be used directly after first loading `mosaic` using `library(mosaic)` or `require(mosaic)`.

²⁰Both the widely used `dplyr` library from the `tidyverse` and `mosaic` sometimes use identical names for different functions, e.g. the symbol `tally`. Therefore, the `mosaic` library is mostly not loaded, but used by combining the package name followed by two colons (`::`) as a prefix to the function name (e.g. by calling `mosaic::tally`).

The R package `janitor`²¹, which is much more closely linked to the *Tidyverse* libraries, contains the function `taby1` (callable e.g. with `janitor::taby1`), which can be used to prepare up to three-dimensional tables. Supplementary functions such as `adorn_totals` and `adorn_percentages` enable the flexible extension of tables with marginal totals, percentage values and other variables.

The direct use of the `mosaic` functions with the `tidyverse` is more complex, as the first argument of most `mosaic` functions usually does not match the data set.

A note: To use a function with the name `<fkt>` from an installed R library (R package) `<rpkg>`, you can load the library using `require(<rpkg>)` or `library(<rpkg>)` and then call it directly using the function call `<fkt>(<args>)` with arguments `<args>`. Alternatively, you can bypass loading the library by using `::` and call the function directly using `rpkg::fkt(<args>)`.

```
> mosaic::tally(~savings, data = german)
Registered S3 method overwritten by 'mosaic':
  method      from
  fortify.SpatialPolygonsDataFrame ggplot2
savings
unknown/no savings account      ... < 100 DM
                                603      103
                                100 <= ... < 500 DM      500 <= ... < 1000 DM
                                63      48
                                ... >= 1000 DM
                                183
```

```
> mosaic::tally(~savings, data = german, format = "percent")
savings
unknown/no savings account      ... < 100 DM
                                60.3      10.3
                                100 <= ... < 500 DM      500 <= ... < 1000 DM
                                6.3      4.8
                                ... >= 1000 DM
                                18.3
```

²¹A side note: The `janitor` library contains numerous auxiliary functions for data preparation and data exploration. This also includes functions that can be used to clean up variable names. Excel tables often use column names which, within the world of programming languages, are non-syntactic names (including the % sign) and can therefore only be used with difficulty (in R, for example, with backticks). The `janitor` function `janitor::clean_names` can be used to convert these variable names very quickly into syntactically valid names.

```
> mosaic::tally(~credit_risk, data = german, format = "count")
credit_risk
  bad good
  300 700

> mosaic::favstats(~duration, data = german)
min Q1 median Q3 max  mean    sd    n missing
  4 12    18 24  72 20.903 12.05881 1000      0
```

tally can also be used for multidimensional or conditional tables.

Examples are:

```
> mosaic::tally(~credit_history + credit_risk, data = german)
credit_history
  delay in paying off in the past
critical account/other credits elsewhere
no credits taken/all credits paid back duly
existing credits paid back duly till now
all credits at this bank paid back duly
credit_risk
  bad good
  25 15
  28 21
 169 361
  28 60
  50 243

> round(mosaic::tally(credit_risk ~ credit_history, data = german, format = "percent"),2)
credit_history
credit_risk delay in paying off in the past
  bad 62.50
  good 37.50
credit_history
credit_risk critical account/other credits elsewhere
  bad 57.14
  good 42.86
credit_history
credit_risk no credits taken/all credits paid back duly
  bad 31.89
  good 68.11
credit_history
```

```

credit_risk existing credits paid back duly till now
      bad      31.82
      good     68.18
      credit_history
credit_risk all credits at this bank paid back duly
      bad      17.06
      good     82.94

> t(round(mosaic::tally(credit_risk~credit_history, data = german, format = "percent"),2))
      credit_risk
credit_history      bad  good
delay in paying off in the past    62.50 37.50
critical account/other credits elsewhere    57.14 42.86
no credits taken/all credits paid back duly    31.89 68.11
existing credits paid back duly till now    31.82 68.18
all credits at this bank paid back duly    17.06 82.94

```

The first table generates the joint distribution. The second table, on the other hand, shows the conditional percentage distribution of credit risks in *good* and *bad* for a given credit history. And the third table shows the second table in transposed form.

These examples also clearly show that it makes sense to use short character strings for categorical characteristic values. Finding these without simultaneously omitting essential information can be very difficult, especially for data sets with numerous variables (e.g. for the Income and Consumer Survey Einkommens- und Verbraucherstichprobe (EVS)²² (Survey on income, consumption and living conditions²³).

7.5 Analysis with the R package `summarytools`

An alternative to the functions used above are the functions available in the `summarytools` package. These are demonstrated here as examples for some variables. However, you can only obtain readable output with the functions of this library if you format the results with `knitr` and *RMarkdown* using the knitr option `results = 'asis'` **and** if you explicitly include the library with `library(summarytools)`. Calling `summarytools::freq()`, on the other hand, often leads to incorrect output. Under Github²⁴ you will always find the current

²²https://www.destatis.de/DE/Themen/Gesellschaft-Umwelt/Einkommen-Konsum-Lebensbedingungen/Konsumausgaben-Lebenshaltungskosten/Methode_n/einkommens-verbrauchsstichprobe.html

²³https://www.destatis.de/EN/Themes/Society-Environment/Income-Consumption-Living-Conditions/Consumption-Expenditure/_node.html

²⁴<https://github.com/dcomtois/summarytools>

R package `summarytools`, which is still under development. Regardless of the installation source, the `summarytools` package is loaded with `library`.

```
> library(summarytools)
```

The following examples show the variety of descriptive statistics in **R**. Depending on the problem and the formatting quality requirements, you will decide on an approach or a package.

7.5.1 Descriptive statistics for metric variables

7.5.1.1 Descriptive statistics for amount

The function `descr` from the package `summarytools` generates a series of descriptive statistics for metric variables; when applied to non-metric variables (`class(<var>) != "numeric"`) such as `savings`, however, you will receive an error message.

```
> descr(german$amount, style = "simple")
```

Descriptive Statistics

german\$amount

N: 1000

	amount
-----	-----
Mean	3271.25
Std.Dev	2822.75
Min	250.00
Q1	1365.00
Median	2319.50
Q3	3972.50
Max	18424.00
MAD	1627.15
IQR	2606.75
CV	0.86
Skewness	1.94
SE.Skewness	0.08
Kurtosis	4.25
N.Valid	1000.00

N	1000.00
Pct.Valid	100.00

Incidentally, warnings occurred occasionally in earlier versions (e.g. the warning that occurred in 2020: `funcs()` is deprecated as of `dplyr` 0.8.0.).

Warnings indicating that *deprecated* functions are being used usually occur when developers do not update their libraries immediately after version changes in R or the Tidyverse. Such warnings can often be ignored temporarily, as *outdated* functions are often still available in the libraries at the same time as the newer versions²⁵. Only after more extensive version changes (e.g. change of the main version) are the *outdated* functions removed from a library. However, the developers of the `summarytools` library adapted the corresponding functions relatively quickly²⁶.

7.5.1.2 Descriptive statistics for all numeric variables

```
> names(german)[sapply(german, is.numeric)]
[1] "duration" "amount"   "age"
> germanNumeric <- german[,sapply(german, is.numeric)]
> descr(germanNumeric, style = "grid")
Descriptive Statistics
germanNumeric
N: 1000
```

	age	amount	duration
Mean	35.54	3271.25	20.90
Std.Dev	11.35	2822.75	12.06
Min	19.00	250.00	4.00
Q1	27.00	1365.00	12.00

²⁵You cannot rely on this.

²⁶Sometimes perfectly useful libraries are eliminated from CRAN, for example because developers do not adapt their libraries to new R interpreters, do not maintain them for all R platforms (Windows, Mac, Linux) or because substantial errors are not eliminated.

Median	33.00	2319.50	18.00
Q3	42.00	3972.50	24.00
Max	75.00	18424.00	72.00
MAD	10.38	1627.15	8.90
IQR	15.00	2606.75	12.00
CV	0.32	0.86	0.58
Skewness	1.02	1.94	1.09
SE.Skewness	0.08	0.08	0.08
Kurtosis	0.60	4.25	0.90
N.Valid	1000.00	1000.00	1000.00
N	1000.00	1000.00	1000.00
Pct.Valid	100.00	100.00	100.00

Within this RMarkdown document, the output format can be displayed in a much more appealing way using the argument `style = "rmarkdown"` and the R chunk option `results = 'asis'`. However, the heading “Descriptive Statistics” becomes quite large - unless you leave it out by using the argument `headings = FALSE` in the function call of `descr`.

```
> descr(germanNumeric, style = "rmarkdown")
```

7.5.2 Descriptive Statistics

7.5.2.1 germanNumeric

N: 1000

	age	amount	duration
Mean	35.54	3271.25	20.90
Std.Dev	11.35	2822.75	12.06
Min	19.00	250.00	4.00
Q1	27.00	1365.00	12.00
Median	33.00	2319.50	18.00
Q3	42.00	3972.50	24.00
Max	75.00	18424.00	72.00
MAD	10.38	1627.15	8.90
IQR	15.00	2606.75	12.00
CV	0.32	0.86	0.58
Skewness	1.02	1.94	1.09
SE.Skewness	0.08	0.08	0.08
Kurtosis	0.60	4.25	0.90
N.Valid	1000.00	1000.00	1000.00
N	1000.00	1000.00	1000.00
Pct.Valid	100.00	100.00	100.00

7.5.3 Descriptive statistics for qualitative variables

7.5.3.1 Frequency table credit_history

```
> freq(german$credit_history, style = "rmarkdown", headings = FALSE)
setting plain.ascii to FALSE
```

	Freq	% Valid	% Valid Cum.	% Total	% Total Cum.
delay in paying off in the past	40	4.00	4.00	4.00	4.00
critical account/other credits elsewhere	49	4.90	8.90	4.90	8.90

	Freq	% Valid	% Valid Cum.	% Total	% Total Cum.
no credits taken/all credits paid back duly	530	53.00	61.90	53.00	61.90
existing credits paid back duly till now	88	8.80	70.70	8.80	70.70
all credits at this bank paid back duly	293	29.30	100.00	29.30	100.00
<NA>	0			0.00	100.00
Total	1000	100.00	100.00	100.00	100.00

7.5.3.2 Frequency table credit_risk

```
> freq(german$credit_risk, style = "rmarkdown", headings = FALSE)
setting plain.ascii to FALSE
```

	Freq	% Valid	% Valid Cum.	% Total	% Total Cum.
bad	300	30.00	30.00	30.00	30.00
good	700	70.00	100.00	70.00	100.00
<NA>	0			0.00	100.00
Total	1000	100.00	100.00	100.00	100.00

7.5.3.3 Conditional distribution credit_risk|credit_history

```
> with(german,
+   ctable(credit_history, credit_risk,
+     prop = "r", # <== is default and can therefore be omitted
+     method = "render",
+     style = "rmarkdown",
+     headings = FALSE))
```

	credit_risk	bad	good	Total
credit_history				
delay in paying off in the past		25 (62.5%)	15 (37.5%)	40 (100.0%)
critical account/other credits elsewhere		28 (57.1%)	21 (42.9%)	49 (100.0%)

no credits taken/all credits paid back duly	169 (31.9%)	361 (68.1%)	530 (100.0%)
existing credits paid back duly till now	28 (31.8%)	60 (68.2%)	88 (100.0%)
all credits at this bank paid back duly	50 (17.1%)	243 (82.9%)	293 (100.0%)
Total	300 (30.0%)	700 (70.0%)	1000 (100.0%)

7.5.3.4 Contingency table `credit_history * credit_risk`

```
> with(german,
+   ctable(credit_history, credit_risk,
+         prop = "t", # common distribution ("r" would be default)
+         method = "render",
+         style = "rmarkdown",
+         headings = FALSE))
```

	credit_risk	bad	good	Total
credit_history				
delay in paying off in the past		25 (2.5%)	15 (1.5%)	40 (4.0%)
critical account/other credits elsewhere		28 (2.8%)	21 (2.1%)	49 (4.9%)
no credits taken/all credits paid back duly		169 (16.9%)	361 (36.1%)	530 (53.0%)
existing credits paid back duly till now		28 (2.8%)	60 (6.0%)	88 (8.8%)
all credits at this bank paid back duly		50 (5.0%)	243 (24.3%)	293 (29.3%)
Total		300 (30.0%)	700 (70.0%)	1000 (100.0%)

7.6 Summary analysis (summary statistics)

It is also possible to generate a summarized description of all variables of a `data.frame`. Within RStudio, you can use the `dfSummary` function to create simple and clear, albeit low-detail displays. Notes on the output:

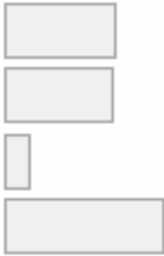
1. if `view(dfSummary(german))` is executed **within** RMarkdown, a *Data Frame Summary* is written to a temporary `html` file.
2. if `view(dfSummary(german))` is executed **within** RStudio (via source or console), the corresponding output appears in the RStudio viewer window.

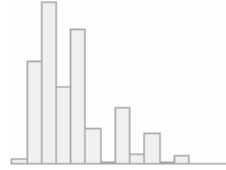
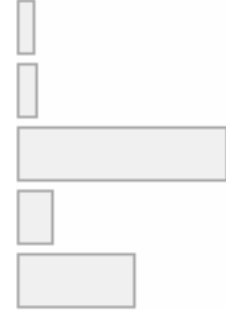
3. if `dfSummary(german)` is executed directly in the console, an object with the class memberships `summarytools` and `data.frame` is generated, which contains a rough representation with alphanumeric graphics, among other things, in the print output.

The function `dfSummary` returns a text representation of the summary in all cases, while `view` returns the temporary HTML file including path as `invisible`.

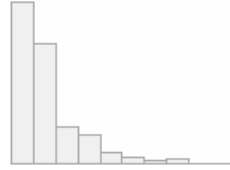
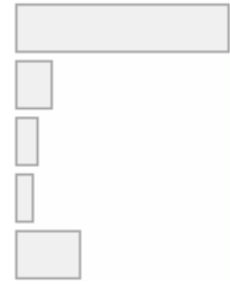
Within RMarkdown you can generate a similar, but usually better representation with the following command:

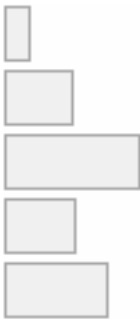
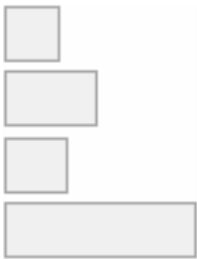
```
> dfSummary(german,
+           plain.ascii = FALSE,
+           style = "grid",
+           headings = FALSE,
+           tmp.img.dir = "./tmp",
+           silent = TRUE)
```

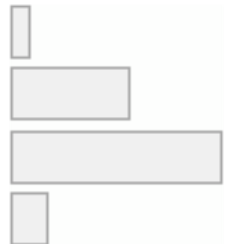
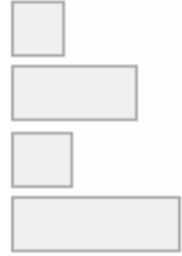
No	Variable	Stats / Values	Freqs (% of Valid)	Graph	Valid	Missing
1	status [factor]	1. no checking account 2. ... < 0 DM 3. 0<= ... < 200 DM 4. ... >= 200 DM / salary fo	274 (27.4%) 269 (26.9%) 63 (6.3%) 394 (39.4%)		1000 (100.0%)	0 (0.0%)

No	Variable	Stats / Values	Freqs (% of Valid)	Graph	Valid	Missing
2	duration [integer]	Mean (sd) : 20.9 (12.1) min < med < max: 4 < 18 < 72 IQR (CV) : 12 (0.6)	33 distinct values		1000 (100.0%)	0 (0.0%)
3	credit_history [factor]	1. delay in paying off in th 2. critical account/other cr 3. no credits taken/all cred 4. existing credits paid bac 5. all credits at this bank	40 (4.0%) 49 (4.9%) 530 (53.0%) 88 (8.8%) 293 (29.3%)		1000 (100.0%)	0 (0.0%)

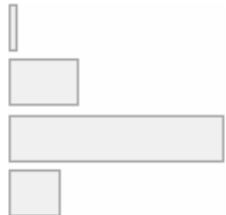
No	Variable	Stats / Values	Freqs (% of Valid)	Graph	Valid	Missing
4	purpose [factor]	1. others	234 (23.4%)		1000 (100.0%)	0 (0.0%)
		2. car (new)	103 (10.3%)			
		3. car (used)	181 (18.1%)			
		4. furniture/equipment	280 (28.0%)			
		5. radio/television	12 (1.2%)			
		6. domestic appliances	22 (2.2%)			
		7. repairs	50 (5.0%)			
		8. education	0 (0.0%)			
		9. vacation	9 (0.9%)			
		10. retraining	97 (9.7%)			
		11. business	12 (1.2%)			

No	Variable	Stats / Values	Freqs (% of Valid)	Graph	Valid	Missing
5	amount [integer]	Mean (sd) : 3271.2 (2822.8) min < med < max: 250 < 2319.5 < 18424 IQR (CV) : 2606.8 (0.9)	923 distinct values		1000 (100.0%)	0 (0.0%)
6	savings [factor]	1. unknown/no savings accoun 2. ... < 100 DM 3. 100 <= ... < 500 DM 4. 500 <= ... < 1000 DM 5. ... >= 1000 DM	603 (60.3%) 103 (10.3%) 63 (6.3%) 48 (4.8%) 183 (18.3%)		1000 (100.0%)	0 (0.0%)

No	Variable	Stats / Values	Freqs (% of Valid)	Graph	Valid	Missing
7	employment_duration [factor]	1. unemployed	62 (6.2%)		1000 (100.0%)	0 (0.0%)
		2. < 1 yr	172 (17.2%)			
		3. 1 <= ... < 4 yrs	339 (33.9%)			
		4. 4 <= ... < 7 yrs	174 (17.4%)			
		5. >= 7 yrs	253 (25.3%)			
8	installment_rate [ordered, factor]	1. >= 35	136 (13.6%)		1000 (100.0%)	0 (0.0%)
		2. 25 <= ... < 35	231 (23.1%)			
		3. 20 <= ... < 25	157 (15.7%)			
		4. < 20	476 (47.6%)			

No	Variable	Stats / Values	Freqs (% of Valid)	Graph	Valid	Missing
9	personal_status_sex [factor]	1. male : divorced/separated 2. female : non-single or ma 3. male : married/widowed 4. female : single	50 (5.0%) 310 (31.0%) 548 (54.8%) 92 (9.2%)		1000 (100.0%)	0 (0.0%)
10	other_debtors [factor]	1. none 2. co-applicant 3. guarantor	907 (90.7%) 41 (4.1%) 52 (5.2%)		1000 (100.0%)	0 (0.0%)
11	present_residence [ordered, factor]	1. < 1 yr 2. 1 <= ... < 4 yrs 3. 4 <= ... < 7 yrs 4. >= 7 yrs	130 (13.0%) 308 (30.8%) 149 (14.9%) 413 (41.3%)		1000 (100.0%)	0 (0.0%)

No	Variable	Stats / Values	Freqs (% of Valid)	Graph	Valid	Missing
12	property [factor]	1. unknown / no property 2. car or other 3. building soc. savings agr 4. real estate	282 (28.2%) 232 (23.2%) 332 (33.2%) 154 (15.4%)		1000 (100.0%)	0 (0.0%)
13	age [integer]	Mean (sd) : 35.5 (11.4) min < med < max: 19 < 33 < 75 IQR (CV) : 15 (0.3)	53 distinct values		1000 (100.0%)	0 (0.0%)
14	other_installment_plans [factor]	1. bank 2. stores 3. none	139 (13.9%) 47 (4.7%) 814 (81.4%)		1000 (100.0%)	0 (0.0%)

No	Variable	Stats / Values	Freqs (% of Valid)	Graph	Valid	Missing
15	housing [factor]	1. for free 2. rent 3. own	179 (17.9%) 714 (71.4%) 107 (10.7%)		1000 (100.0%)	0 (0.0%)
16	number_credits [ordered, factor]	1. 1 2. 2-3 3. 4-5 4. >= 6	633 (63.3%) 333 (33.3%) 28 (2.8%) 6 (0.6%)		1000 (100.0%)	0 (0.0%)
17	job [factor]	1. unemployed/unskilled - no 2. unskilled - resident 3. skilled employee/official 4. manager/self-empl./highly	22 (2.2%) 200 (20.0%) 630 (63.0%) 148 (14.8%)		1000 (100.0%)	0 (0.0%)

No	Variable	Stats / Values	Freqs (% of Valid)	Graph	Valid	Missing
18	people_liable [factor]	1. 3 or more 2. 0 to 2	155 (15.5%) 845 (84.5%)		1000 (100.0%)	0 (0.0%)
19	telephone [factor]	1. no 2. yes (under customer name)	596 (59.6%) 404 (40.4%)		1000 (100.0%)	0 (0.0%)
20	foreign_worker [factor]	1. yes 2. no	37 (3.7%) 963 (96.3%)		1000 (100.0%)	0 (0.0%)
21	credit_risk [factor]	1. bad 2. good	300 (30.0%) 700 (70.0%)		1000 (100.0%)	0 (0.0%)

Generating visually appealing tables is generally an art, so you can find numerous tips on the Internet such as [How to Make Beautiful Tables in R](https://rfortherestofus.com/2019/11/how-to-make-beautiful-tables-in-r)²⁷ and [Top 7 Packages for Making Beautiful Tables in R](https://towardsdatascience.com/top-7-packages-for-making-beautiful-tables-in-r-7683d054e541)²⁸ as well as several R packages such as the package `r2spss`²⁹ on CRAN³⁰. These R packages are particularly useful when you need to analyze extensive survey data.

Incidentally, there is a whole range of other packages in **R** that can be used to calculate summarized statistics. Worth mentioning here is also the package `skimr`, with which you can save summary statistics in the `tibble`'s shown in the third chapter and thus feed them into a further evaluation. This package can also be used to generate tables that can be used to create dynamic reports using the functions in **RMarkdown** and `knitr`.

²⁷<https://rfortherestofus.com/2019/11/how-to-make-beautiful-tables-in-r>

²⁸<https://towardsdatascience.com/top-7-packages-for-making-beautiful-tables-in-r-7683d054e541>

²⁹<https://cran.r-project.org/web/packages/r2spss/index.html>

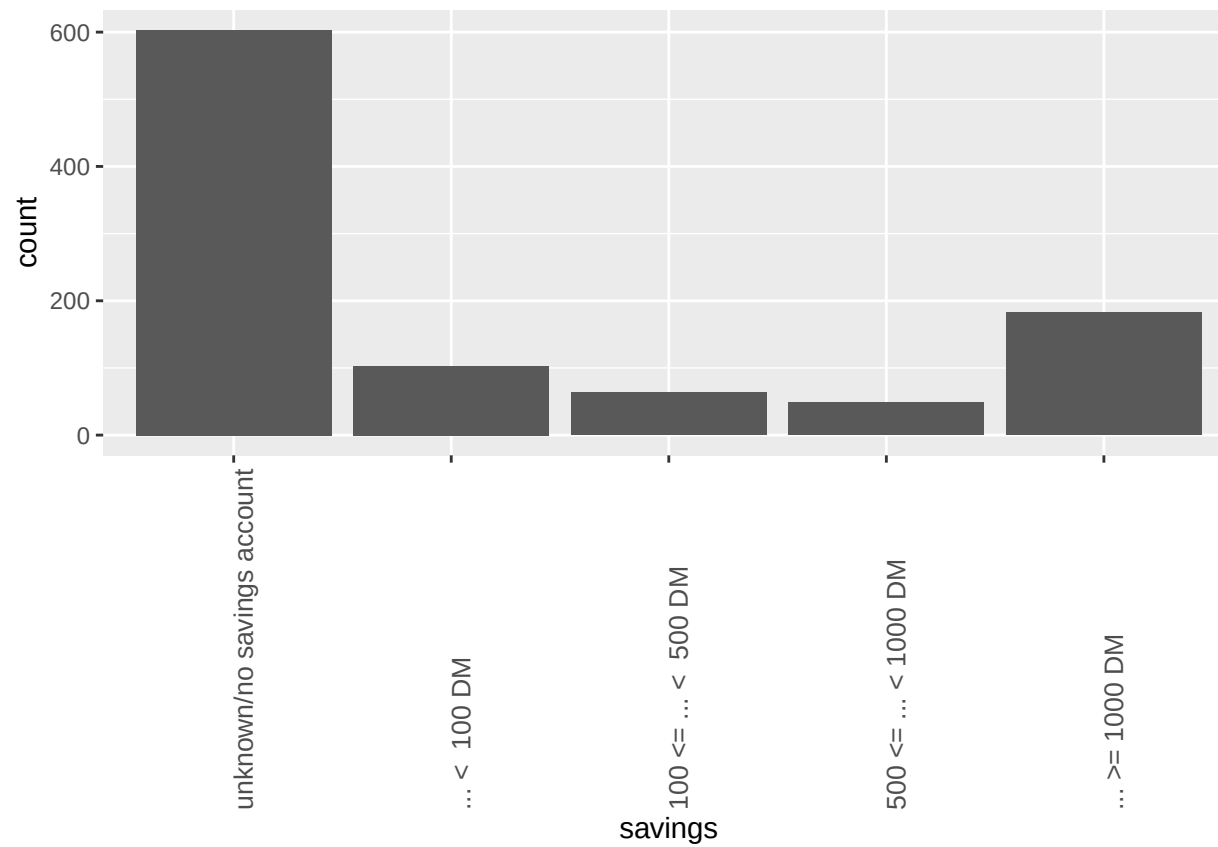
³⁰<https://cran.r-project.org/>

Another package is `dataMaid`. Unfortunately the last revisions of the R package `dataMaid` are no longer as stable as the versions from previous years. This is why I am increasingly using the R package `summarytools`, which was described in the previous section.

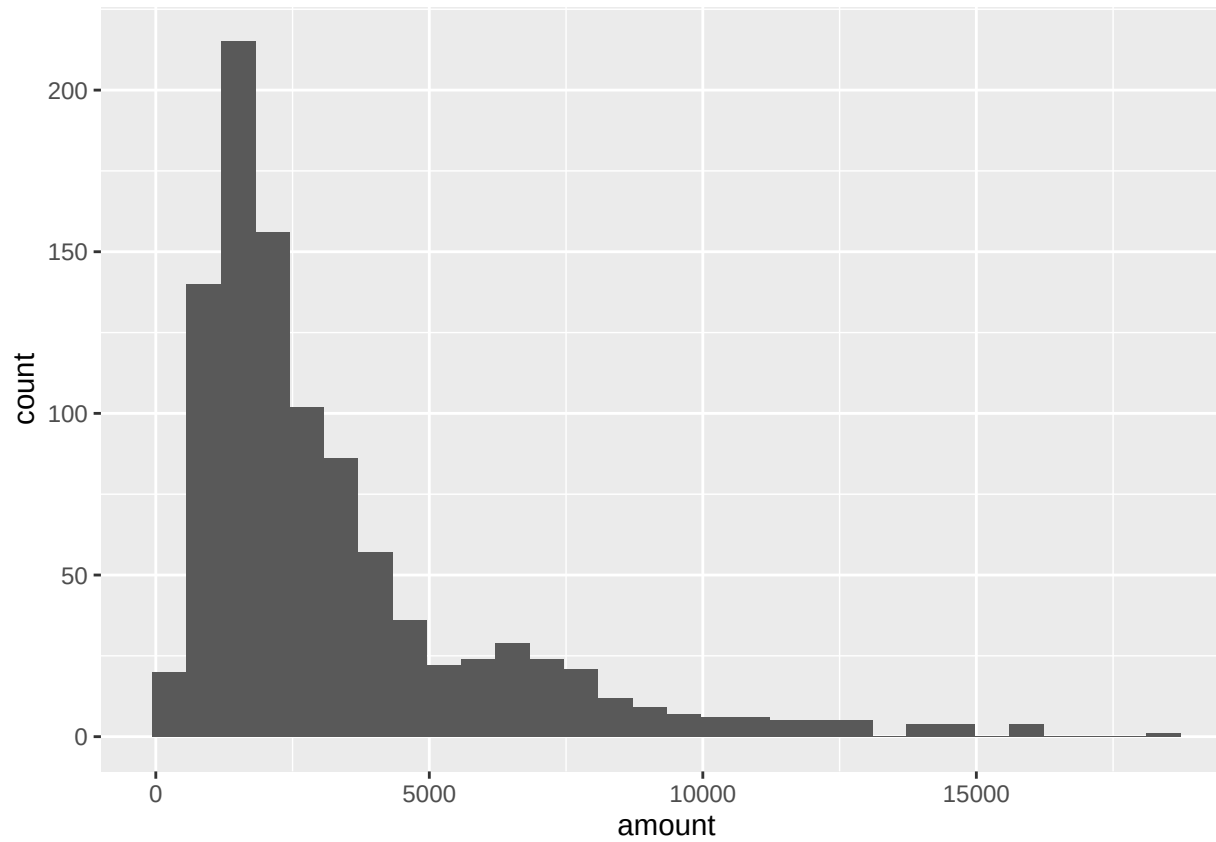
7.7 Exemplary graphs with ‘ggplot2’

As a preview of the highly practical and, above all, very powerful R library `ggplot2`, several variables (`status`, `credit_history` and `amount`) are presented graphically as examples. However, the logic behind the functions will be described in the next chapter.

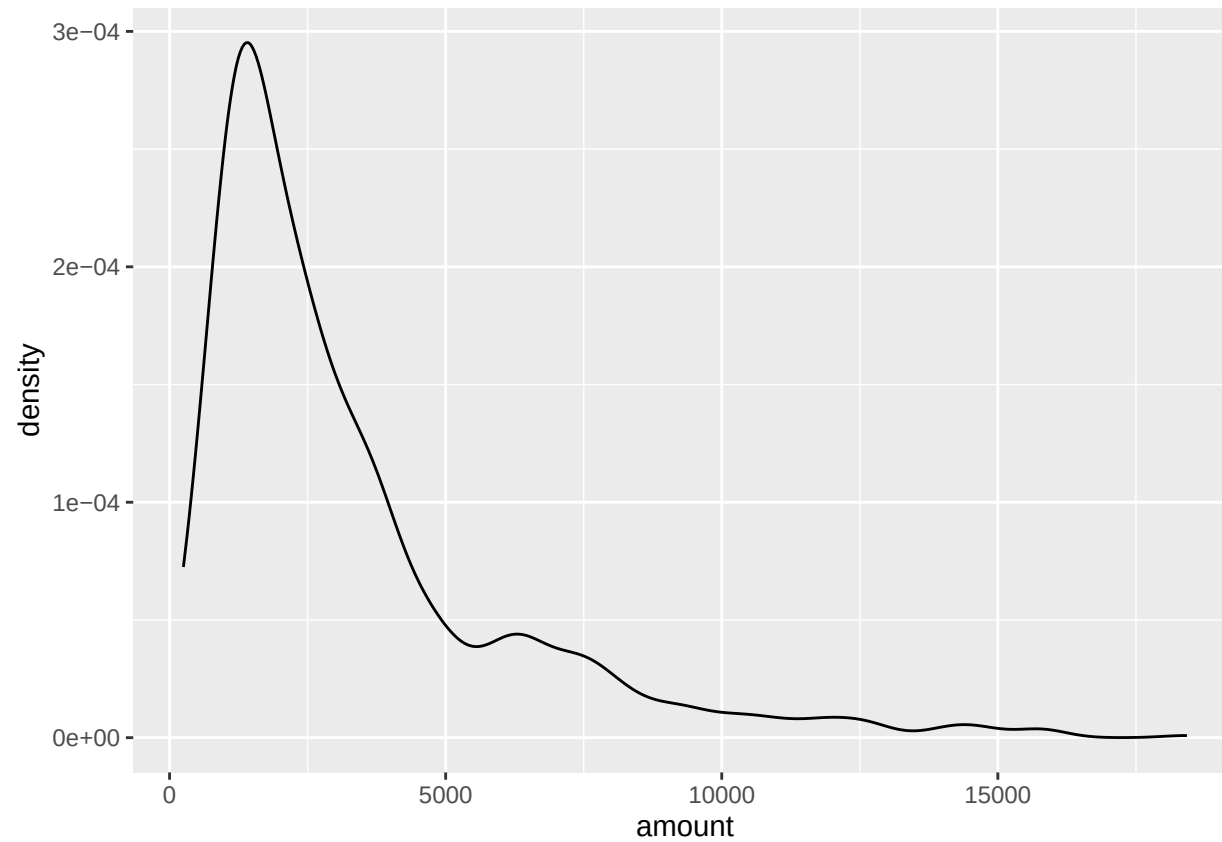
```
> german |>
+   ggplot(aes(x = savings)) +
+   geom_bar() +
+   theme(axis.text.x = element_text(
+     size = 10,
+     angle = 90,
+     vjust = 1,
+     hjust = 0))
```



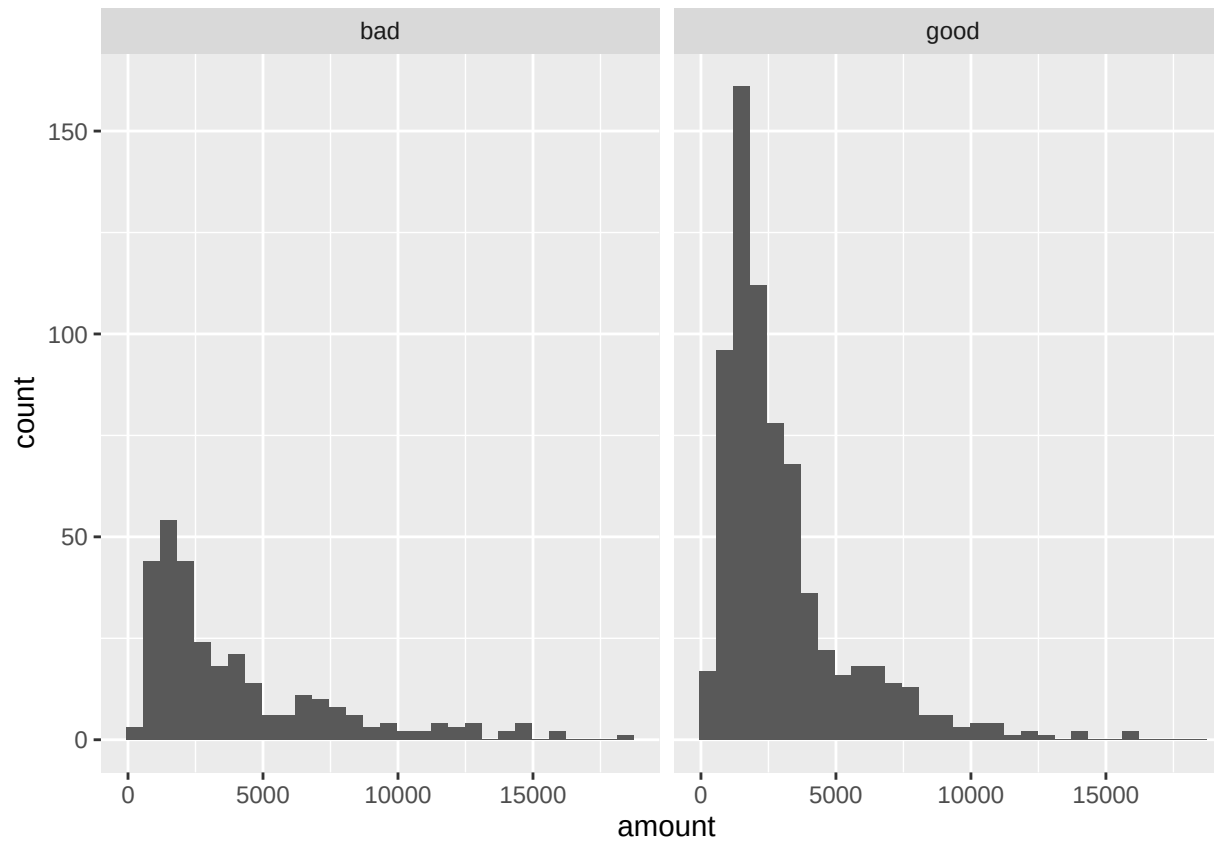
```
> german |>
+   ggplot(aes(x = amount)) +
+   geom_histogram(
+     stat_bin() using `bins = 30`. Pick better value
+     `binwidth`.
+   )
```



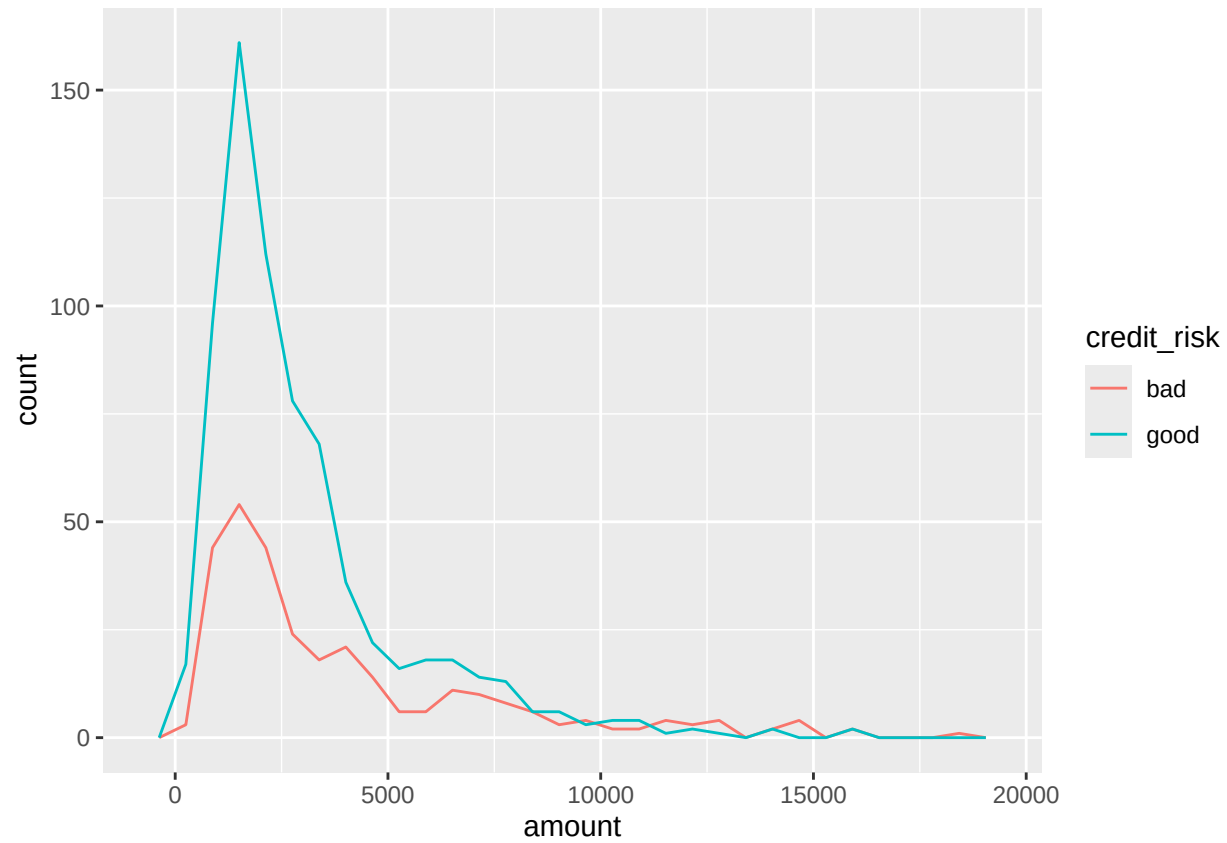
```
> german |>  
+   ggplot(aes(x = amount)) +  
+   geom_density()
```

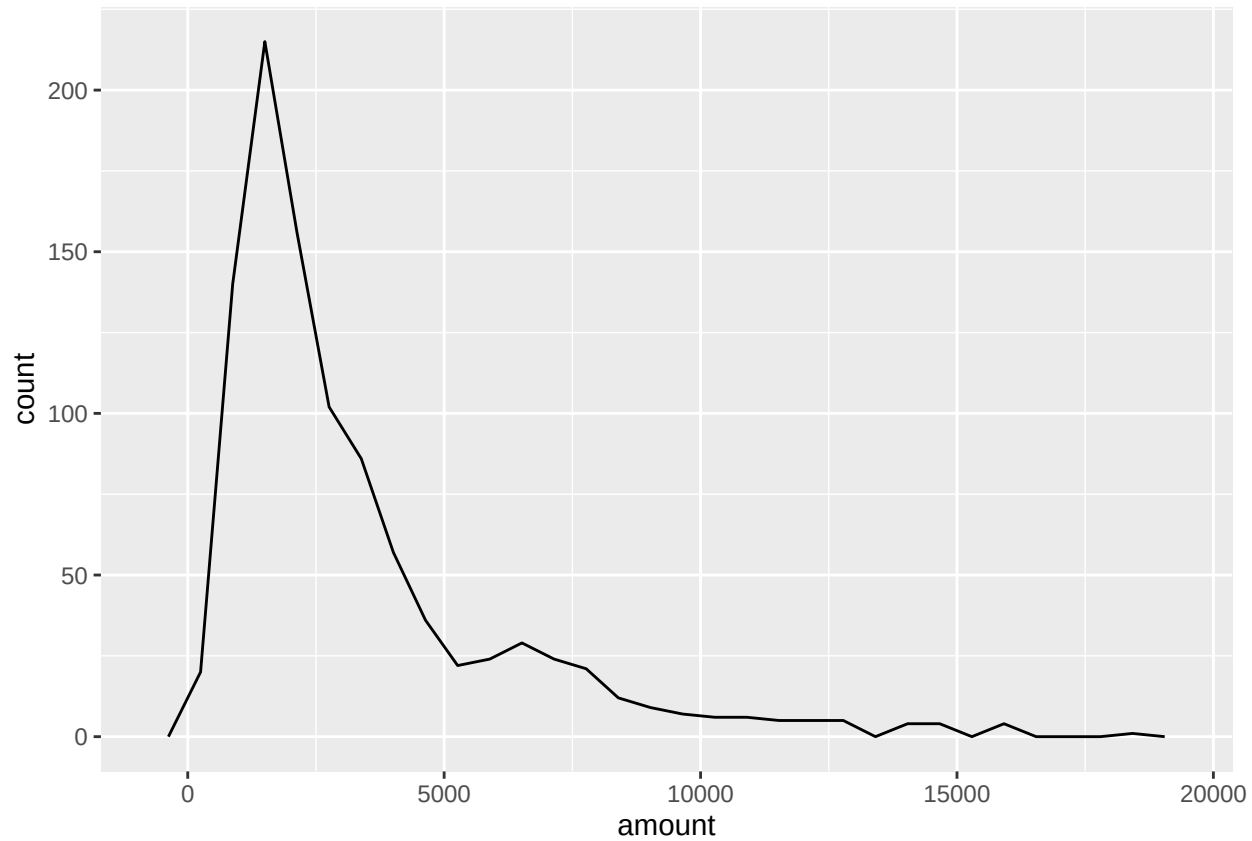
```
> german |>  
+   ggplot(aes(x = amount)) +  
+   geom_histogram() +  
+   facet_wrap(~credit_risk)  
`stat_bin()` using `bins = 30`. Pick better value  
`binwidth`.
```



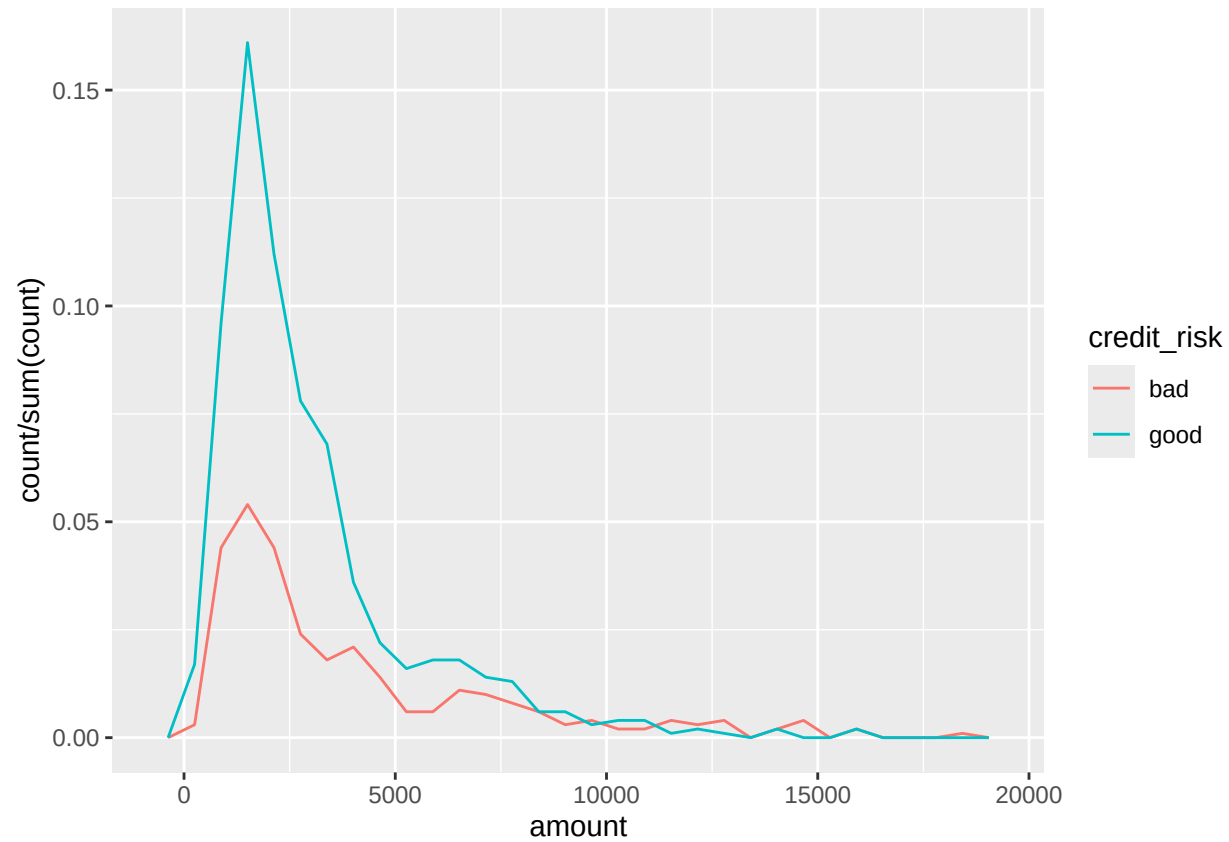
```
> german |>
+   ggplot(aes(x = amount, color = credit_risk)) +
+   geom_freqpoly()
`stat_bin()` using `bins = 30`. Pick better value
`binwidth`.
```



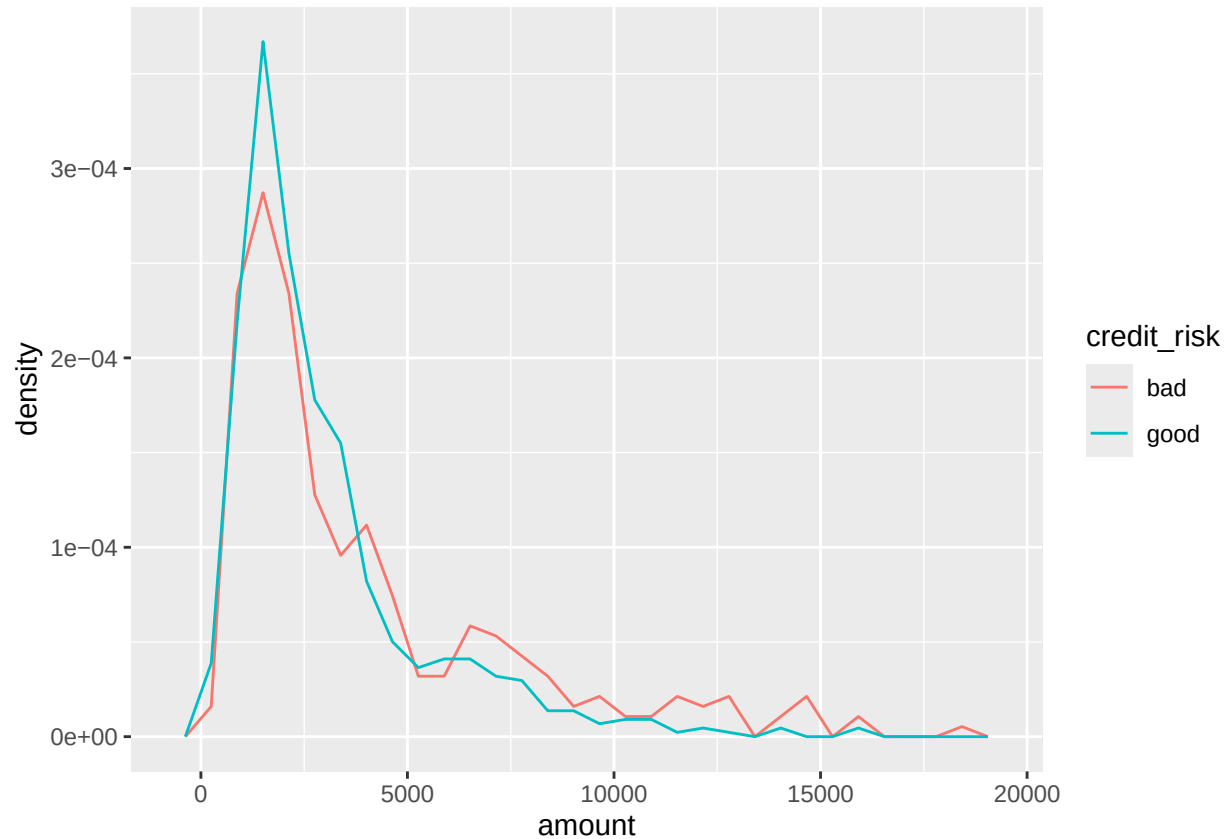
```
> german |>  
+   ggplot(aes(x = amount)) +  
+   geom_freqpoly(aes(y = after_stat(count)))  
`stat_bin()` using `bins = 30`. Pick better value  
`binwidth`.
```



```
> german |>
+   ggplot(aes(x = amount, color = credit_risk)) +
+   geom_freqpoly(aes(y = after_stat(count/sum(count))))
`stat_bin()` using `bins = 30`. Pick better value
`binwidth`.
```



```
> german |>
+   ggplot(aes(x = amount, color = credit_risk)) +
+   geom_freqpoly(aes(y = after_stat(density)))
`stat_bin()` using `bins = 30`. Pick better value
`binwidth`.
```



The `geom_*`'s implicitly calculate some statistical quantities from the data, such as frequency or count tables. These are extracted in the current `ggplot-geom`'s with the relatively new auxiliary function `after_stat`. Until some time ago, however, these statistical variables had to be extracted using poorly documented internal variable names limited by two dots (`..`) on the left and right side of the symbol name. Examples are

```
> german |>
+   ggplot(aes(x = amount)) +
+   geom_freqpoly(aes(y = ..count.. / sum(..count..)))
```

and

```
> german |>
+   ggplot(aes(x = amount, color = credit_risk)) +
+   geom_freqpoly(aes(y = ..density..))
> # ^alternatively: ..count.. (default), ..ncount.., ..ndensity..
```

In the meantime, this procedure, which is still frequently used, has been marked as obsolete (*deprecated*). Accordingly, the last two plot commands generate warning messages such as the following in some old package versions:

```
# Warning: The dot-dot notation (`..count..`) was deprecated in ggplot2 3.4.0.
# ? Please use `after_stat(count)` instead.
```

These warnings have now been disabled again.

7.8 Data set nycflights13

Install the library `nycflights13` with the data, if not yet available.

```
> install.packages("nycflights13", dependencies = TRUE)
```

Loading the library

```
> library(nycflights13)
```

The R package `nycflights13` used in numerous works contains essential information about American domestic flights that took off from the three commercial airports in New York in 2013. Specifically, these are the following airports:

- JFK = John F. Kennedy Airport
- LGA = La Guardia Airport
- EWR = Newark Airport

The geographical positions can also be displayed on an *OpenStreetMap* map using a few R commands, whereby the coordinates longitude (`lon`) and latitude (`lat`) of the three airports were determined manually using the Coordinates converter for WGS84, UTM, CH1903, UTMREF(MGRS), Gauß-Krüger, NAC, W3W³¹.

³¹<https://coordinates-converter.com>

```

> library(OpenStreetMap)
> library(ggimage)
> #library(ggrepel)
> # Coordinates according to https://coordinates-converter.com/en/
> airports_coord <- tribble(
+   ~airport, ~lat, ~lon,
+   "JFK", 40.642948, -73.779373, # JFK = John F. Kennedy Airport
+   "LGA", 40.776927, -73.873966, # LGA = La Guardia Airport, New York
+   "EWR", 40.689064, -74.177255) # EWR = Newark Airport
> airports_coord
# A tibble: 3 x 3
  airport lat lon
  <chr>   <dbl> <dbl>
1 JFK     40.6 -73.8
2 LGA     40.8 -73.9
3 EWR     40.7 -74.2
> lat <- airports_coord |>
+   pull(lat)
> lon <- airports_coord |>
+   pull(lon)
> distance_degree <- 0.1
> # Read OSM map around three New York airports and save it: outcommented
> # airports_new_york <- openmap(
> #   c(min(lat) - distance_degree, min(lon) - distance_degree),
> #   c(max(lat) + distance_degree, max(lon) + distance_degree),
> #   type = "osm")
> # save(airports_new_york, file = "./data/OSM2025/airports_new_york.RData") # Backup copy
> load(file = "./data/OSM2025/airports_new_york.RData") # load saved file
> # Generate quasi Mercator projection
> airports_new_york_mercator <- airports_new_york |>
+   openproj()
> # Plot map with airports
> airports_new_york_mercator |>
+   autoplot() +

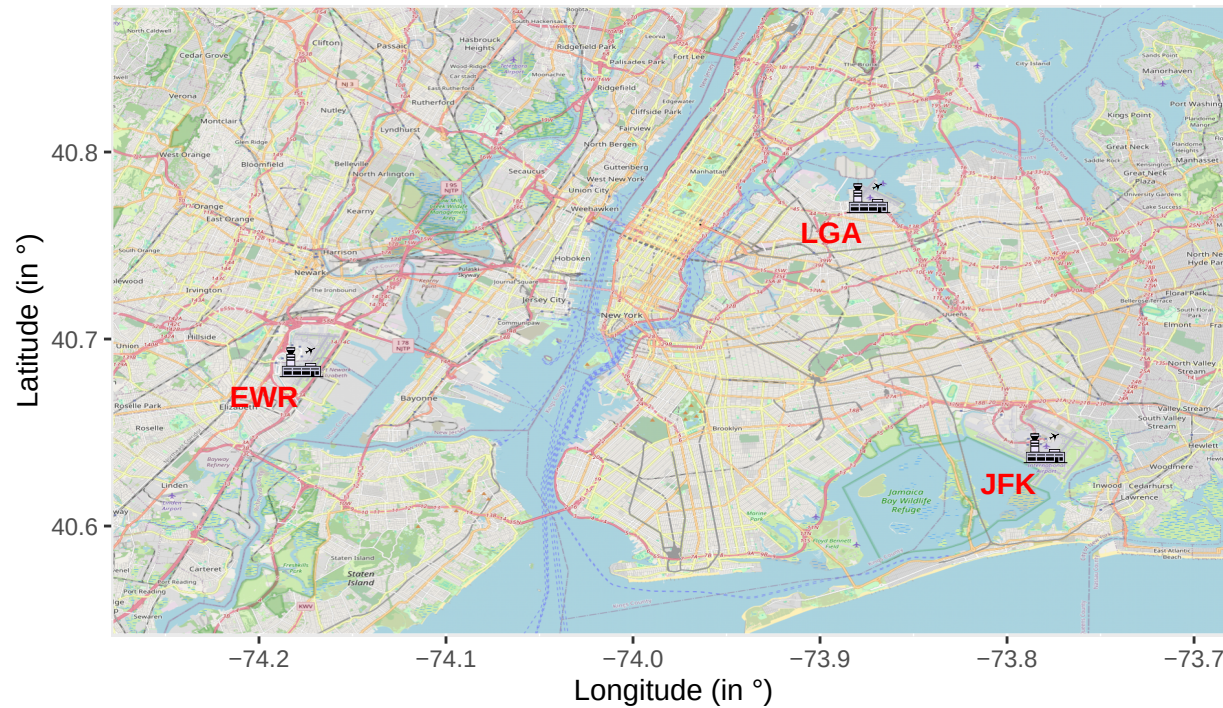
```



```
+ geom_image(data = airports_coord,  
+           aes(x = lon, y = lat),  
+           image = "./images/Airport_symbol.svg.png",  
+           size = 0.07) +  
+ geom_text(data = airports_coord,  
+           aes(x = lon - 0.02, y = lat - 0.02, label = airport),  
+           color = "red",  
+           fontface = "bold",  
+           size = 4) +  
+ labs(title = "Airports New York",  
+       subtitle = "Special Mercator projection",  
+       x = "Longitude (in °)",  
+       y = "Latitude (in °)",  
+       caption = "\u00A9 Ulrich Küsters, October 27, 2025")
```

Airports New York

Special Mercator projection



© Ulrich Küsters, October 27, 2025

In detail, the library contains five non-redundant data sets that can be used as so-called *tibble-data-frame*'s with class `tbl_df` and thus also via class inheritance as *tibbles* and as *dataframes*; see next teaching unit.

- **flights** contains all departures in 2013 from JFK, LGA and EWR and includes 336776 cases and 19 variables.
- **airports** contains all associated destination airports and includes 1458 destination airports and 8 variables.
- **airlines** contains all associated airlines and includes 16 lines and 2 variables.
- **planes** contains all associated airplanes (with identifier) and includes 3322 machines and 9 variables.
- **weather** contains the weather measurement data for all three airports and includes 26115 METAR reports with 15 variables.

METARs are METeorological Aerodrome Reports, see the Wikipedia entry METAR³².

The variable descriptions can be found by calling up the documentation, e.g. by entering `help(package = "nycflights13")` in the console.

The five data records are linked to each other via so-called *keys*. The data records can be linked together using so-called *join* operations. *Join* operations can be realized either with the help of SQL (Structured Query Language) or with the help of the functions implemented in the R library `dplyr` (as part of `tidyverse`). The latter are dealt with in detail in the unit *DSW-3*.

The five tables from `nycflights13` are used in the Data Science courses mainly for the following purposes:

- Demonstration of graphs in `ggplot2`
- Demonstrate the handling of POSIX data and times using the Tidyverse library `lubridate`.
- Demonstrate data preparation and combination of information from multiple tables using Tidyverse libraries `dplyr` and `tidyr`.
- Possible extension of these analyses with **mySQL** and application of comparable functions for the much larger `airlines` dataset (difficult so far due to slow queries on KU-mySQL server).

Note in passing: You can also determine the coordinates of the three airports using the following function calls from the `airports` table in the R package `nycflights13`; latitude and longitudes differ slightly from the coordinates in `airports_coord` used above.

```
> nycflights13::airports |>
+   filter(faa %in% c("JFK", "LGA", "EWR")) |>
+   rename(airport = faa) |>
+   select(airport, lat, lon)
# A tibble: 3 x 3
  airport  lat  lon
  <chr>   <dbl> <dbl>
1 EWR     40.7 -74.2
2 JFK     40.6 -73.8
3 LGA     40.8 -73.9
```

ToDo (own work)

1. *Install* **R**, **RStudio** and the R packages used in this chapter.
2. reproduce all examples

³²<https://de.wikipedia.org/wiki/METAR>

3. inspect the datasets `iris3` and `UCBAdmissions` stored in the R package `datasets` using the above-mentioned summary functions.
4. inspection of the dataset `nycflights13` with `favstats` and `tally`.

Chapter 8

Overview about of ‘graphics’, lattice and ggplot2

In this teaching unit, the most important basics of the three different groups of R graphic packages are covered:

1. The base package **graphics**.
2. The Trellis library **lattice**.
3. The **ggplot2** package. This package is based on the *Grammar of Graphics*. **ggplot2** has a tight connection with the Tidyverse data preparation and manipulation packages **tidyr** and **dplyr**.

The basic R graphics use the drivers of the package **grDevices** package. This base package can be used to generate graphic outputs, including coloring and fonts on various media (**devices**) such as WMF, PNG, PS, EPS, and PDF files, as well as screen windows under various operating systems such as Windows (**windows**) and Linux (**X11**). These driver functions are described here only selectively in connection with specific tasks. Note that **grDevices** is also used by other R graphic libraries for the preparation of static graphics. Examples are the R packages **lattice** and **ggplot2**.

Details on the functions described here can be found in the R documentation of the corresponding packages. In Venables & Ripley (2002) and Murrell (2006) you can find basic explanations of **grDevices**, **graphics** and **lattice**. The second edition of the book by Wickham

(2016) covers `ggplot2` in great detail. On the internet you can find `ggplot2`: Elegant Graphics for Data Analysis (3e)¹. This edition is still under revision and therefore still incomplete. Hillebrand & Nierhoff (2015) limit themselves to a brief overview of `ggplot2`, while Chang (2023) provides easy-to-use recipes for a variety of `ggplot2` graphs.

Note: The R graphs in this manuscript are used for illustration purposes only. So it is not about *beautiful* graphs but about showing how to construct certain graphs. Visually appealing graphics usually require a very careful configuration of all graphic parameters. For publication graphics and company reports, this makes sense, but for the exploratory analysis of data sets, this configuration effort is not justifiable. Rahlf (2018) gives numerous examples of how to create excellent publication graphics using the package in R-base.

¹<https://ggplot2-book.org/>

Chapter 9

Base graphics in R {r-base-graphics}

9.1 Data

To illustrate the fundamental basic functions, some manageable data sets from the R library `datasets` library are used before more extensive data sets like `german` and `nycflights13` are analyzed. Manageable data sets with few variables and cases have the advantage that the calculations behind the graphs and statistics can still be controlled manually.

The R library `datasets` is automatically included in the search path when R is started. Note that the *Tidyverse* libraries as well as other libraries also listed in the search path as they were explicitly loaded by a large range of `library` statements in the header of the *RMarkdown* resp. *bookdown* files.

```
> search()
[1] ".GlobalEnv"          "package:ggimage"
[3] "package:OpenStreetMap" "package:nycflights13"
[5] "package:mosaicData"   "package:ggformula"
[7] "package:Matrix"      "package:lattice"
[9] "package:repmis"      "package:knitr"
[11] "package:summarytools" "package:DiagrammeRsvg"
[13] "package:DiagrammeR"   "package:webshot"
[15] "package:rsvg"         "package:lubridate"
```

```

[17] "package:forcats"      "package:stringr"
[19] "package:dplyr"        "package:purrr"
[21] "package:readr"        "package:tidyr"
[23] "package:tibble"       "package:ggplot2"
[25] "package:tidyverse"    "package:stats"
[27] "package:graphics"     "package:grDevices"
[29] "package:utils"        "package:datasets"
[31] "package:methods"     "Autoloads"
[33] "package:base"

```

The very small data set `mtcars` is sourced from the R package `datasets`. `mtcars` was saved as `adata.frameobject` (class `data.frame`) and contains 32 cases and 11 variables.

The data set `mtcars` describes the characteristics of 32 American passenger cars from the years 1973/74 using the following variables (see the R help under `?mtcars`):

Column	Variable	Meaning
1	mpg	Miles/(US) gallon ¹
2	cyl	Number of cylinders
3	disp	Displacement (cu.in.)
4	hp	Gross horsepower
5	drat	Rear axle ratio (rear axle ratio ²)
6	wt	Weight (1000 lbs)
7	qsec	1/4 mile time (acceleration)
8	vs	V/S (0 is a V-engine and 1 is an in-line engine)
9	am	transmission (0 = automatic, 1 = manual)
10	gear	Number of forward gears
11	carb	Number of carburetors

The following R function calls summarize the most important information:

¹The data set uses American units: 1 gallon = 3.78541 liters; 1 inch = 2.54 cm; 1 cu.in = 1 cubic inch = 16.3871 milliliter = 16.3871 cm³ = 0.0163871 liter; 1 lbs = 1 American pound = 453.592 gr; 1 mile (statute mile, not sea mile) = 1.60934 km.

²By selecting a large rear axle ratio, you can achieve a high torque at low engine speeds. High torque at low engine speeds can save fuel.


```
> str(mtcars) # outputs structure (abbreviated)
'data.frame': 32 obs. of 11 variables:
 $ mpg : num 21 21 22.8 21.4 18.7 18.1 14.3 24.4 22.8 19.2 ...
 $ cyl : num 6 6 4 6 8 6 8 4 4 6 ...
 $ disp: num 160 160 108 258 360 ...
 $ hp : num 110 110 93 110 175 105 245 62 95 123 ...
 $ drat: num 3.9 3.9 3.85 3.08 3.15 2.76 3.21 3.69 3.92 3.92 ...
 $ wt : num 2.62 2.88 2.32 3.21 3.44 ...
 $ qsec: num 16.5 17 18.6 19.4 17 ...
 $ vs : num 0 0 1 1 0 1 0 1 1 1 ...
 $ am : num 1 1 1 0 0 0 0 0 0 0 ...
 $ gear: num 4 4 4 3 3 3 3 4 4 4 ...
 $ carb: num 4 4 1 1 2 1 4 2 2 4 ...
```

```
> dim(mtcars) # dimension = number of rows, number of columns
[1] 32 11
```

```
> nrow(mtcars) # number of rows
[1] 32
```

```
> ncol(mtcars) # Column number
[1] 11
```

```
> head(mtcars) # The header of the data (default 6 obs)
      mpg cyl disp  hp drat   wt  qsec vs am
Mazda RX4         21.0   6  160 110  3.90 2.620 16.46  0  1
Mazda RX4 Wag     21.0   6  160 110  3.90 2.875 17.02  0  1
Datsun 710         22.8   4  108  93  3.85 2.320 18.61  1  1
Hornet 4 Drive     21.4   6  258 110  3.08 3.215 19.44  1  0
Hornet Sportabout 18.7   8  360 175  3.15 3.440 17.02  0  0
Valiant           18.1   6  225 105  2.76 3.460 20.22  1  0
      gear carb
Mazda RX4         4    4
Mazda RX4 Wag     4    4
Datsun 710         4    1
```

```
Hornet 4 Drive      3    1
Hornet Sportabout  3    2
Valiant            3    1
```

```
> tail(mtcars) # The tail of the data (default 6 obs)
```

```
      mpg cyl  disp  hp drat   wt  qsec vs am
Porsche 914-2 26.0  4 120.3  91 4.43 2.140 16.7  0  1
Lotus Europa 30.4  4  95.1 113 3.77 1.513 16.9  1  1
Ford Pantera L 15.8  8 351.0 264 4.22 3.170 14.5  0  1
Ferrari Dino  19.7  6 145.0 175 3.62 2.770 15.5  0  1
Maserati Bora  15.0  8 301.0 335 3.54 3.570 14.6  0  1
Volvo 142E     21.4  4 121.0 109 4.11 2.780 18.6  1  1
```

```
      gear carb
```

```
Porsche 914-2      5    2
Lotus Europa       5    2
Ford Pantera L     5    4
Ferrari Dino       5    6
Maserati Bora      5    8
Volvo 142E         4    2
```

```
> sapply(mtcars, class) # Class/type per variable
```

```
      mpg      cyl      disp      hp      drat      wt
"numeric" "numeric" "numeric" "numeric" "numeric" "numeric"
      qsec      vs      am      gear      carb
"numeric" "numeric" "numeric" "numeric" "numeric"
```

```
> summary(mtcars) # Summary statistics
```

```
      mpg          cyl          disp
Min.   :10.40   Min.   :4.000   Min.    : 71.1
1st Qu.:15.43   1st Qu.:4.000   1st Qu.:120.8
Median :19.20   Median :6.000   Median :196.3
Mean   :20.09   Mean   :6.188   Mean   :230.7
3rd Qu.:22.80   3rd Qu.:8.000   3rd Qu.:326.0
Max.   :33.90   Max.   :8.000   Max.   :472.0

      hp          drat          wt
```

```

Min.    : 52.0   Min.    :2.760   Min.    :1.513
1st Qu.: 96.5   1st Qu.:3.080   1st Qu.:2.581
Median :123.0   Median :3.695   Median :3.325
Mean    :146.7   Mean    :3.597   Mean    :3.217
3rd Qu.:180.0   3rd Qu.:3.920   3rd Qu.:3.610
Max.    :335.0   Max.    :4.930   Max.    :5.424
      qsec      vs      am
Min.    :14.50   Min.    :0.0000   Min.    :0.0000
1st Qu.:16.89   1st Qu.:0.0000   1st Qu.:0.0000
Median :17.71   Median :0.0000   Median :0.0000
Mean    :17.85   Mean    :0.4375   Mean    :0.4062
3rd Qu.:18.90   3rd Qu.:1.0000   3rd Qu.:1.0000
Max.    :22.90   Max.    :1.0000   Max.    :1.0000
      gear      carb
Min.    :3.000   Min.    :1.000
1st Qu.:3.000   1st Qu.:2.000
Median :4.000   Median :2.000
Mean    :3.688   Mean    :2.812
3rd Qu.:4.000   3rd Qu.:4.000
Max.    :5.000   Max.    :8.000

```

Note that all variables have been coded as metric-scaled variables. This definitely makes sense for variables 1 and 3-7. Variables 8 (**vs**) and 9 (**am**), on the other hand, are clearly qualitative variables. Variables 2, 10, and 11 are clearly numerical variables, but the variables have only a few numbers of characteristic values. Therefore, their interpretation differs from that of numerical variables. With some loss, the variables can be coded as nominal scaled variables (**factor**) or, while retaining the order, as ordinal scaled variables (using the function **ordered** with resulting class membership **c("ordered", "factor")** to maintain the order). Variable 3 (**disp**) can be coded as both a numerical as well as an ordinal variable. This variable will therefore be retained as a metric variable.

Changes to the coding will be applied to a copy **my_mtcars** of the data set **mtcars**. As a reminder, all addressing schemes of variables, like positional indices, name indices, etc., can be used in **data.frame**'s. Obviously the direct addressing via variable names makes the most sense. However, instead of using **my_mtcars\$am** you can also address using the position (like in **my_mtcars[[8]]**). Unfortunately, after deletion, insertion, or reordering of variables, a conversion of the indices in subsequent parts of the program is necessary. Therefore, addressing via indices is not useful for maintenance.

```

> my_mtcars <- mtcars

> my_mtcars$vs <- factor(mtcars$vs, levels = c(0,1), labels = c("V-engine", "Series-engine"))
> my_mtcars$am <- factor(mtcars$am, levels = c(0,1), labels = c("automatic", "manual"))

> my_mtcars$cyl <- as.ordered(mtcars[[2]]) # Index addressing not good for maintenance

> my_mtcars["gear"] <- as.ordered(mtcars$gear) # '[' one-element list

> my_mtcars[["carb"]] <- as.ordered(mtcars$carb) # '[' vector

> table(my_mtcars$vs, mtcars$vs) # To check whether the assignment is correct

      0  1
V-engine 18 0
Series-engine 0 14
> table(my_mtcars$am, mtcars$am) # To check whether the assignment is correct

      0  1
automatic 19 0
manual      0 13

```

```

> summary(my_mtcars)
      mpg      cyl      disp      hp
Min.   :10.40  4:11  Min.   : 71.1  Min.   : 52.0
1st Qu.:15.43  6: 7   1st Qu.:120.8  1st Qu.: 96.5
Median :19.20  8:14  Median :196.3  Median :123.0
Mean   :20.09           Mean   :230.7  Mean   :146.7
3rd Qu.:22.80           3rd Qu.:326.0  3rd Qu.:180.0
Max.   :33.90           Max.   :472.0  Max.   :335.0

      drat      wt      qsec
Min.   :2.760  Min.   :1.513  Min.   :14.50
1st Qu.:3.080  1st Qu.:2.581  1st Qu.:16.89
Median :3.695  Median :3.325  Median :17.71
Mean   :3.597  Mean   :3.217  Mean   :17.85
3rd Qu.:3.920  3rd Qu.:3.610  3rd Qu.:18.90

```

```

Max.      :4.930   Max.      :5.424   Max.      :22.90
      vs          am      gear   carb
V-engine   :18     automatic:19     3:15    1: 7
Series-engine:14   manual   :13     4:12    2:10
                        5: 5      3: 3
                        4:10
                        6: 1
                        8: 1

```

A forward reference to the sections that refer to **tidyverse** libraries such as **ggplot2**, **readr** and **dplyr**: In the **ggplot2** graphics library, depending on the function, metrically scaled variables (**numeric**) might sometimes be used as qualitative variables (e.g., with the layer function **facet_wrap**) or as metric variables (e.g., within the layer function **geom_point**). If you want to ensure that a numeric variable is displayed and interpreted as a qualitative variable, it must be converted into a **character**, **ordered** or **factor** variable with the help of a coercing function (e.g., **as.character**). When reading data records, however, there is often trouble with **ordered** or **factor** variables. So in modern libraries such as **readr** and **data.table**, qualitative variables are now predominantly read in as **character** variables and only read in as variables with class membership **ordered** or **factor** if this is required by a statistical procedure (e.g., for a **formula** in a regression function such as **lm**). However, you should note that with large data sets, **factor** variables are stored internally as numerical codes, which usually require less memory space than character string vectors³.

Before a representation of the modern and tidyverse grammar-of-graphics library **ggplot2** for the following reasons^{[5 reasons⁴}, we will first describe some selected, but basic graphics routines from the basic R library **graphics** are described first:

1. Many R libraries use the base library **graphics**.
2. Functions of the **graphics** library are often faster than dedicated high-level graphics libraries such as **ggplot2**.
3. The **ggplot2** functions were primarily designed for exploratory data analysis. Therefore, elementary graphs and graphs consisting of vectors or arrows, for example, can sometimes be easily generated with the basic functions from **graphics**. Doing the same with **ggplot2** might become complex.
4. For publication graphics, one still often uses the basic graphics system due to its greater flexibility, especially since excellent templates for sometimes very complex graphics exist; see e.g. Rahlf (2018)).

³This can be seen as an example in the code sequence `set.seed(123); ch <- sample(c("Hewlett", "Packard", "Workstation"),size=10000, replace=TRUE); fc <- as.factor(ch)` and the memory evaluation statement sequence `object.size(ch); object.size(fc)`. This shows that the **char** vector representation requires twice as much storage space as the corresponding **factor** representation. Due very efficient memory management of strings in more recent versions of R (using a string pool), the difference is no longer huge but still existing.

⁴In the winter term 2025/26, the R crash course was too short to cover the R base graphics. However, basic knowledge is indispensable.

The developers of numerous `ggplot2` extensions (see e.g. `ggplot2` extensions⁵), the last two arguments are becoming increasingly less important.

In the following sections, the most important basic functions from `graphics` are presented in the following sections.

9.2 Histogram

The histogram corresponds to a graphical representation of a count, which can be determined with the following commands, among others.

The function `cut` is used to classify a numeric variable into intervals described by breakpoints defined by the argument vector `break`. Default intervals have the left boundary excluded and the right boundary included. But this can be reversed by the optional parameter `right`.

```
> mpg_cut <- cut(my_mtcars$mpg, breaks = seq(10,35,5))

> table(mpg_cut)
mpg_cut
(10,15] (15,20] (20,25] (25,30] (30,35]
      6      12      8       2       4

> typeof(mpg_cut)
[1] "integer"

> class(mpg_cut)
[1] "factor"

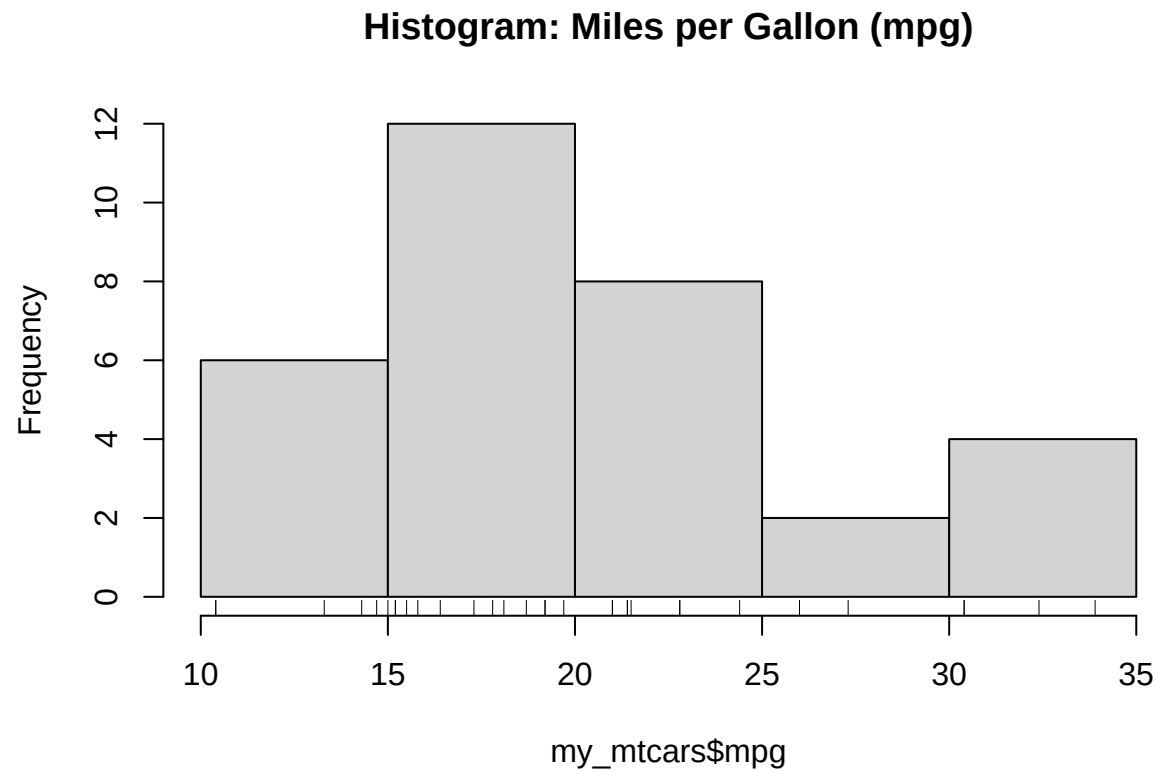
> str(mpg_cut)
Factor w/ 5 levels "(10,15]", "(15,20]", ...: 3 3 3 3 2 2 1 3 3 2 ...
```

This table can be represented graphically with the R function `hist` shown in the next chunk. It is important to note that each interval is an open interval on the left and a closed interval on the right, as can be seen from the description of the `right` parameter in the help (`?hist`). The histogram is supplemented by a `rug`⁶, which marks each observation with a line below the abscissa.

⁵<https://exts.ggplot2.tidyverse.org/>

⁶The English term `rug` means *runner* or *carpet*, as here something like a *rug*.

```
> hist(my_mtcars$mpg, main="Histogram: Miles per Gallon (mpg)")
> rug(my_mtcars$mpg)
```



The problem with `rug` is that different observations with identical variable values cannot be recognized. For example the value 30.4 occurs **twice** in the variable `mpg`, which is not recognizable in the `rug`. The underlying causes can only be determined by counting values.

```
> table(my_mtcars$mpg)
10.4 13.3 14.3 14.7  15 15.2 15.5 15.8 16.4 17.3 17.8 18.1
```

```

      2      1      1      1      1      2      1      1      1      1      1      1
18.7 19.2 19.7    21 21.4 21.5 22.8 24.4    26 27.3 30.4 32.4
      1      2      1      2      2      1      2      1      1      1      2      1
33.9
      1

```

Most graphics functions use default settings that can be changed explicitly. The `hist` function uses 18 parameters (see `?hist`), of which most analysts only use a handful. In addition, graphics can also be customized using basic parameters (`?par`) such as margins; an overview of the most important overview of the most important graphic parameters can be found on the Posit-Cheatsheets page⁷ under the *Contributed Cheatsheets* and the link *How big is your graph?*⁸.

9.3 Density plot

In the following and very simple example, the default setting is replaced, modified and supplemented by the following elements:

- (1) The absolute frequencies are replaced by relative frequencies.
- (2) The interval widths are modified to a grid pattern of 10 to 26 miles per gallon with a width of 2.
- (3) A grid is added.
- (4) A density plot is superimposed on the histogram.

The density plot can be created with `plot(density(my_mtcars$mpg))` even without a frequency diagram. Density plots are created using kernel estimators (see the courses *Angewandte Statistische Methoden* or *Daten und Regressionsanalyse* as well as Küsters & Kalinowski (2001)).

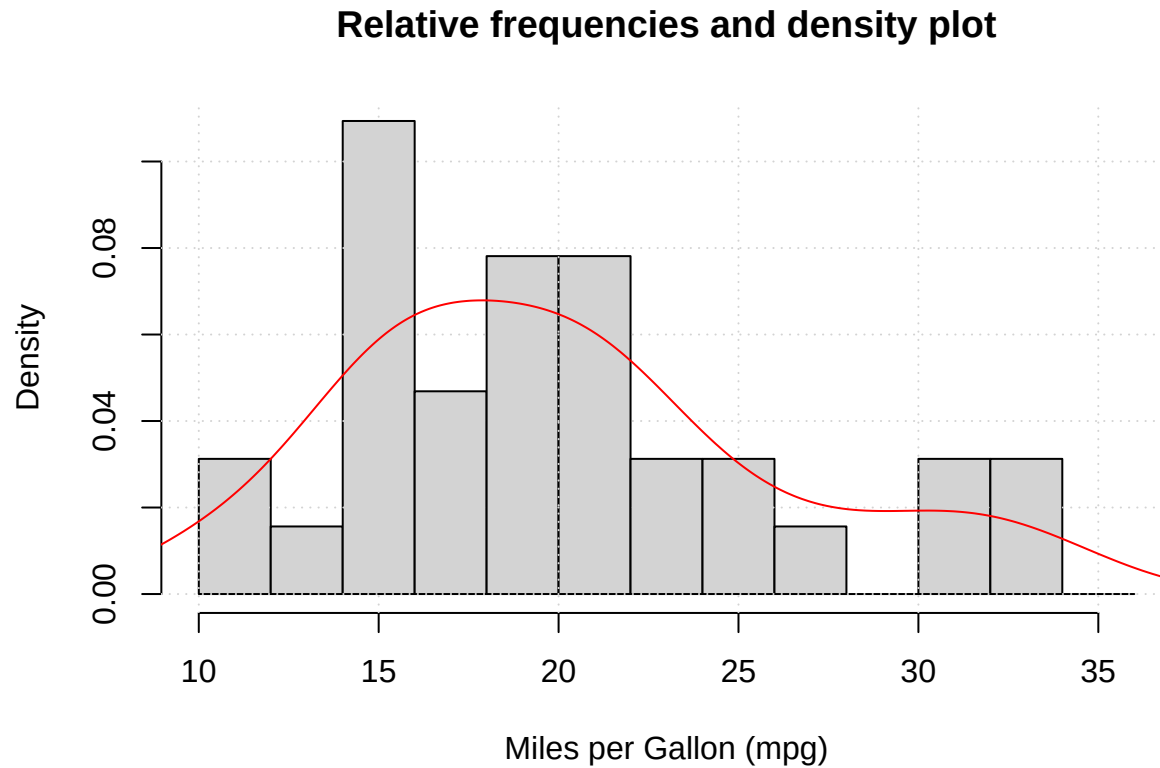
```

> hist(my_mtcars$mpg,
+      prob = TRUE,
+      breaks = seq(10,36,2),
+      main = "Relative frequencies and density plot",
+      xlab = "Miles per Gallon (mpg)",
+      ylab = "Density")
> grid()
> lines(density(my_mtcars$mpg), col = "red")

```

⁷<https://posit.co/resources/cheatsheets/>

⁸<https://rstudio.github.io/cheatsheets/how-big-is-your-graph.pdf>



By varying the arguments of the `density` function (e.g., the parameter `bandwidth` `bw` and the kernel method `kernel`) you can change the density plot considerably. Note the edges of the density plot. These show positive densities for x-values *outside* the value range. This (sometimes desirable) edge effect (also denoted as boundary bias) is later visually “corrected” with the ggplot2 function `geom_density` by clipping the edges to the value range.

9.4 Barplot

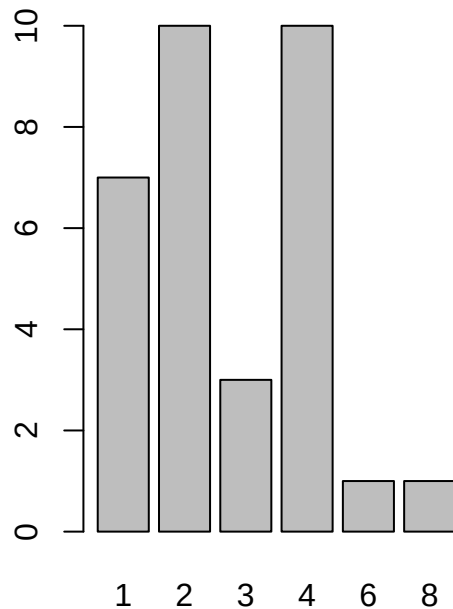
The following R command sequence generates a frequency diagram for two qualitative variables. Here, the variables are `carb` and `am`. Initially at R startup, the default `par(mfrow = c(1,1))` is used. This generates single plots for each window/page. Using the global parameter setting `par(mfrow = c(1,2))` you can position two graphs next to each other. The construction `oldpar <- par()` saves

the previous parameter setting while `par(oldpar)` reset the parameter setting to the state before changing the state by `par(mfrow = c(1,2))`. Details on the `par` options can be found in the chapter *Global graphic parameters with par* and in the R help `?par`.

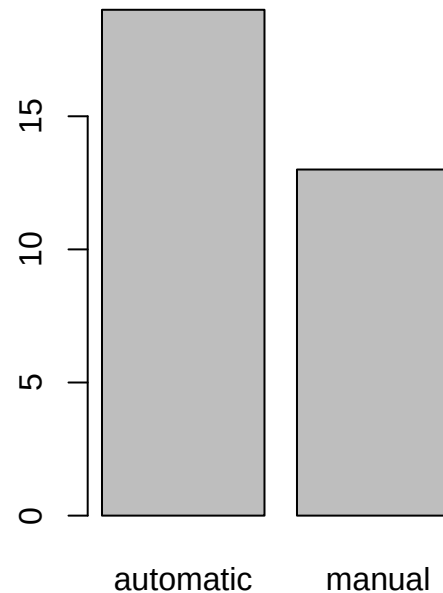
In the R packages `ggplot2` and `knitr`, the functions available to restore the parameters set by the basic command `par` only work to a limited extent. This document was created with the help of the R packages `rmarkdown` and `bookdown` – both use the R package `knitr`. Therefore, R issues several warnings that the parameters stored in `oldpar` such as `cin`, `cra` etc. can no longer be restored by `par(oldpar)`. See the output behind the two following barplots.

```
> oldpar <- par()
> par(mfrow = c(1,2))
> barplot(table(my_mtcars$carb), main = "Barplot of the carburetor count\n(carb)")
> barplot(table(my_mtcars$am), main = "Barplot of the gear\n(am)")
```

**Barplot of the carburetor count
(carb)**



**Barplot of the gear
(am)**



```
> par(oldpar)
Warning in par(oldpar): graphical parameter "cin" cannot be
set
Warning in par(oldpar): graphical parameter "cra" cannot be
set
Warning in par(oldpar): graphical parameter "csi" cannot be
set
Warning in par(oldpar): graphical parameter "cxy" cannot be
set
Warning in par(oldpar): graphical parameter "din" cannot be
set
Warning in par(oldpar): graphical parameter "page" cannot
be set
```

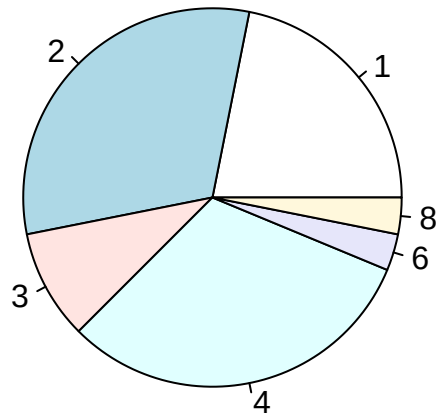
The error messages for `par(oldpar)` in the output indicate that six parameters (e.g., `cra`) are read-only and therefore must not be set; see the help `?par`.

9.5 Pie chart

The following R command creates a pie chart. The use of this and comparable diagrams (e.g. circular gauges, also referred as gauges) are not recommended. Due to the coloring, the frequent misinterpretation and the very unfavorable *ink-to-information* ratio 'pie charts and gauges are rejected by most statistics experts.

```
> pie(table(my_mtcars$carb),
+     main="Pie chart of carb count (carb)")
```

Pie chart of carb count (carb)

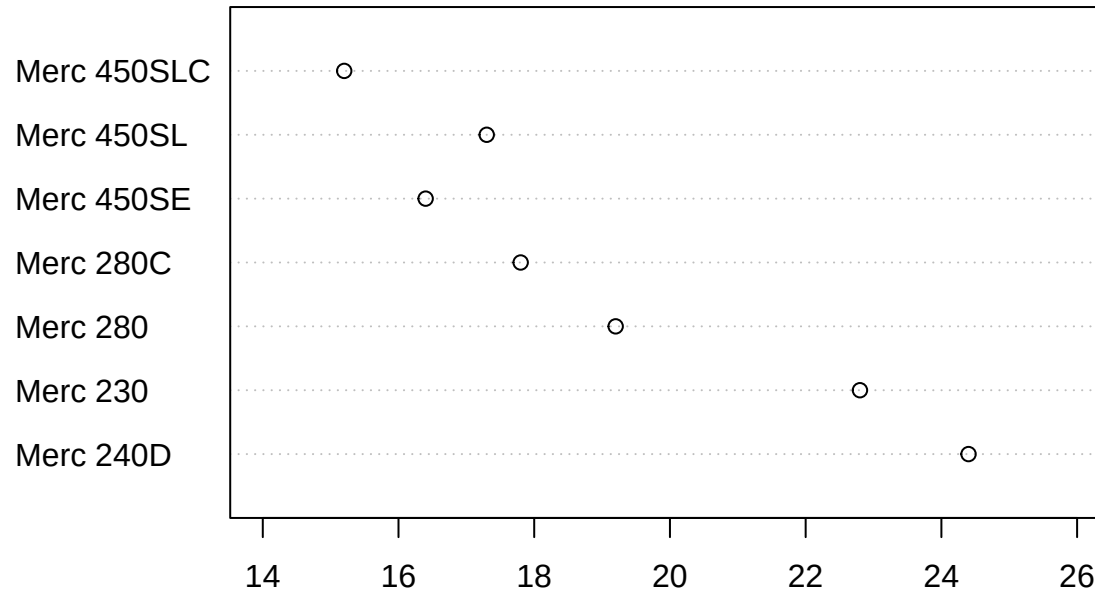


9.6 Dotchart

The following R code first calculates the index positions of all Mercedes cars by using the very simple regular expression “`^Merc`” and the function `str_which` from the package `stringr` (`row.names(my_mtcars)`) as all car names starting with “Merc” are Mercedes vehicles. Second, the variable “mpg” of these cars are plotted in the so-called dot plot. In addition, the value range of the x-axis (abscissa) is extended to the interval `[14,26]`. Dotplots are only suitable for small data sets. However, dot plots are often used for the presentation of a few descriptive key figures and model estimators when numerous variables and/or model variants need to be characterized. This is often the case for large data sets.

```
> merc_idx <- stringr::str_which(row.names(my_mtcars), "^Merc")
> dotchart(my_mtcars$mpg[merc_idx],
+         labels=row.names(mtcars)[merc_idx],
+         main = "Dot Plot Variable \"Miles per Gallon (mpg)\" for Mercedes",
+         xlim = c(14,26))
```

Dot Plot Variable "Miles per Gallon (mpg)" for Mercedes

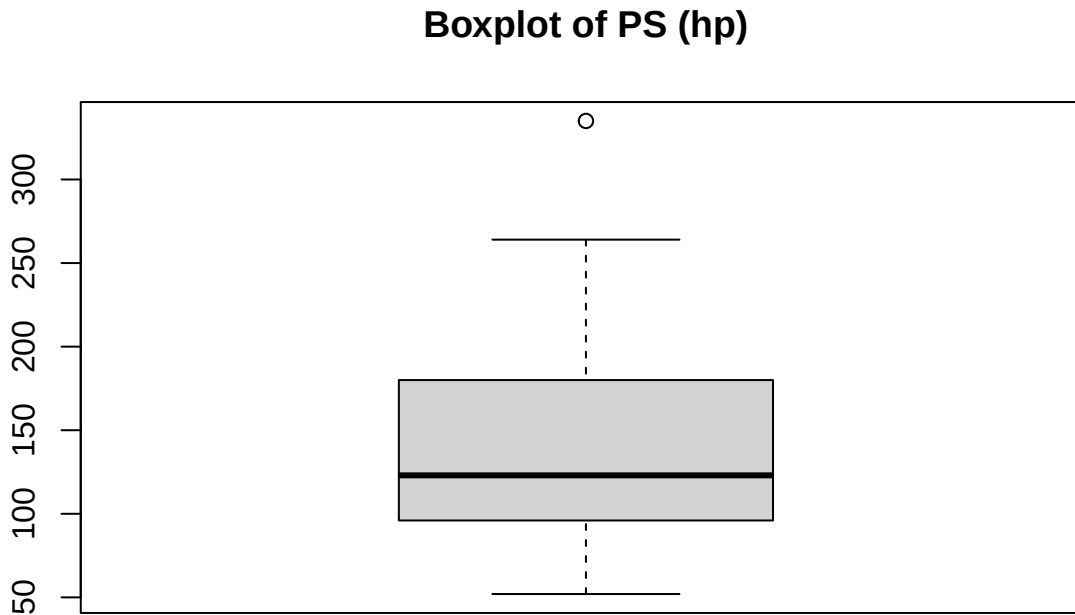


9.7 Boxplots

The boxplot, already familiar from basic studies, is used to quickly visualize the distribution of a variable. Below you will find the summary statistics and the boxplot of the variable `hp`. The asymmetry of the distribution is clearly recognizable, as the lower quartile (1st quartile) is significantly closer to the median (median) than the third quartile (3rd quartile) and the arithmetic mean (mean) is significantly larger than the median (see Fechner's position rule).

```
> summary(my_mtcars$hp)
  Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
  52.0   96.5   123.0   146.7   180.0   335.0

> boxplot(my_mtcars$hp, main = 'Boxplot of PS (hp)' )
```

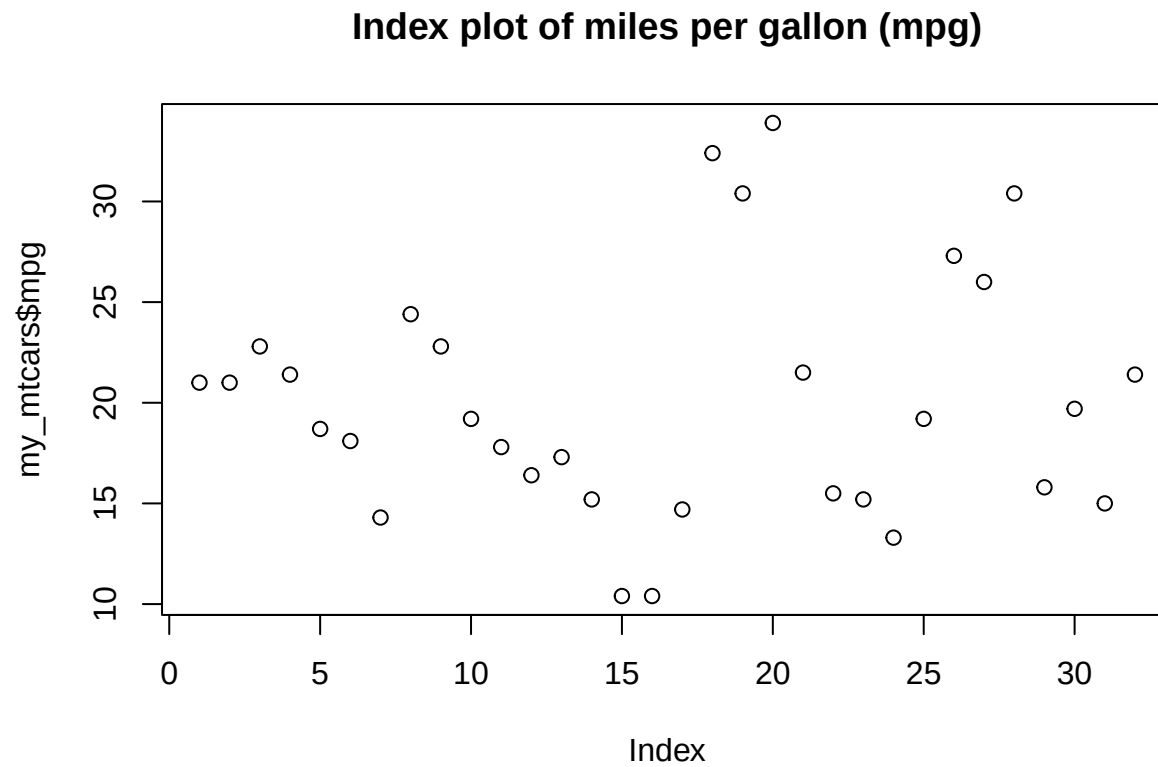


There are numerous box plot variants. A whole series of these variants, such as notched box plots for the assessment of group group differences have been implemented in R; see the help `?boxplot`.

9.8 Index plot

The generic function `plot` generates very different plots depending on the class to which the input argument belongs. If you apply `plot` to a numerical vector, you get a point plot in which the abscissa (x-axis) describes the observation index, i.e., the indices.

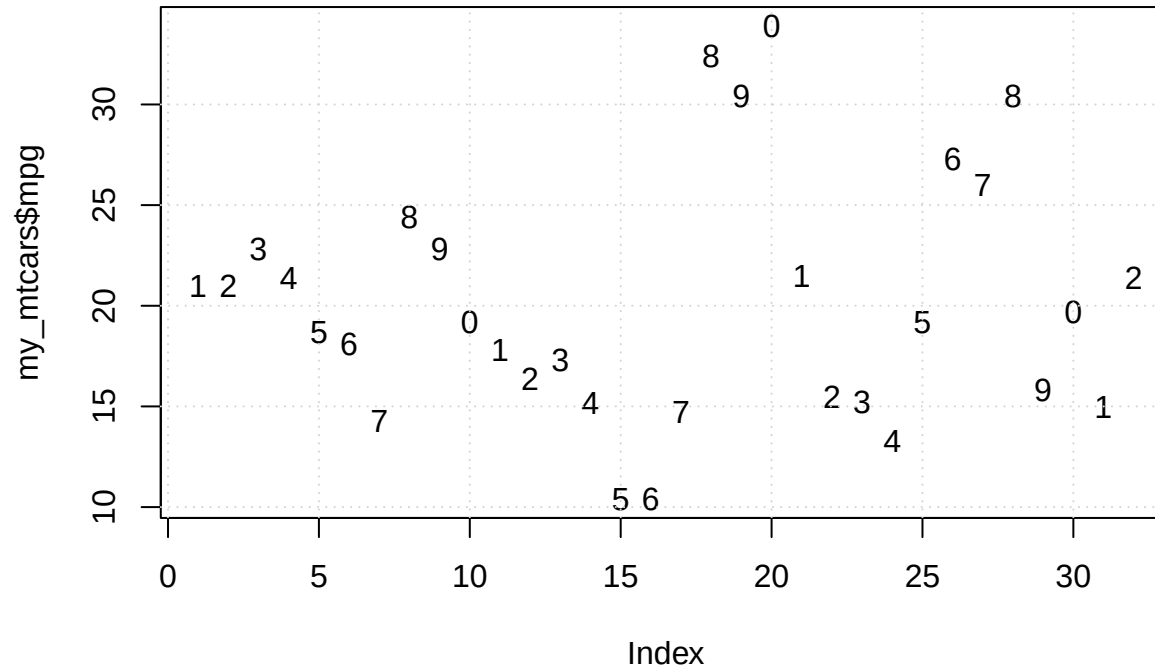
```
> plot(my_mtcars$mpg, main = "Index plot of miles per gallon (mpg)")
```



By overwriting the plot symbol `pch` with `as.character((1:nrow(my_mtcars)) %% 10)` in conjunction with the grid pattern subsequently drawn by `grid()`, the data points can be easily identified.

```
> plot(my_mtcars$mpg,  
+       main = "Index plot of miles per gallon (mpg)",  
+       pch = as.character((1:nrow(my_mtcars)) %% 10))  
> grid()
```

Index plot of miles per gallon (mpg)



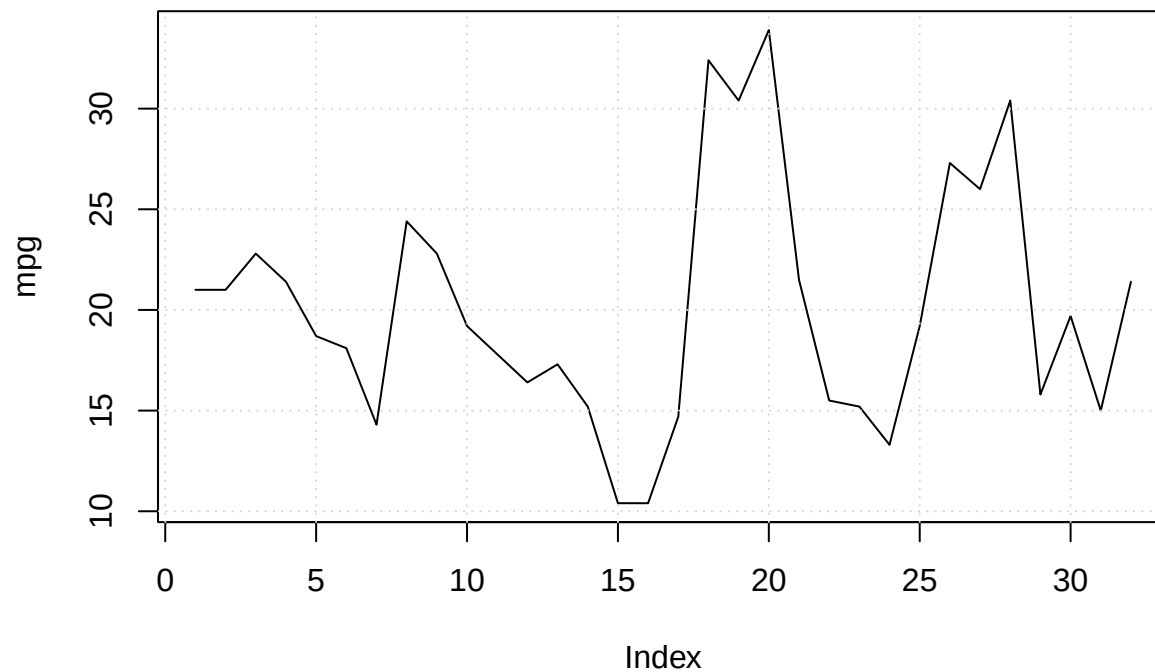
If you want to connect the points, replace the default setting of the line type `type = "p"` in an index plot with `type = "l"`. An example which makes little sense⁹ for the variable `my_mtcars$mpg` is given by the following command.

```
> with(my_mtcars,
+   {
+     plot(mpg,
+         main = "Index plot of miles per gallon (mpg)",
```

⁹Index plots, on the other hand, are often used to display time series where the integer index is usually replaced by a time index (often a combination of year/quarter/month/week/day/hour/minute/second) with the help of R's date and time objects (e.g., data types like `POSIXlt` and `POSIXct` or derived variants used in R packages like `Dateand zoo`). Nowadays, the modern functions from the Tidyverse library `lubridate` are predominantly used for setting and manipulating dates and times.


```
+         type = "l")  
+     grid()  
+ }
```

Index plot of miles per gallon (mpg)



Please note that the access to the variable `mpg` in the dataframe `my_mtcars` either by means of `my_mtcars$mpg` or by `attach(my_mtcars)` is avoided by the use of scoping function `with`.

Index plots, especially if you connect the lines, are useful for ordered data (e.g. time series¹⁰) and for identifying outliers. Otherwise,

¹⁰Procedures for analyzing time series are taught in the two courses *Statistical forecasting methods* and *Time series analysis* are presented. There we used over two decades, among other things object classes `ts` for time series and `mts` for multiple time series Multiple Time Series, since the application of the generic plot function generates appropriately prepared graphs, especially for annual, quarterly and monthly time series. generated for annual, quarterly and monthly

two-dimensional plots are predominantly used. One example is the relationship between performance (`hp`) and consumption (`mpg`).

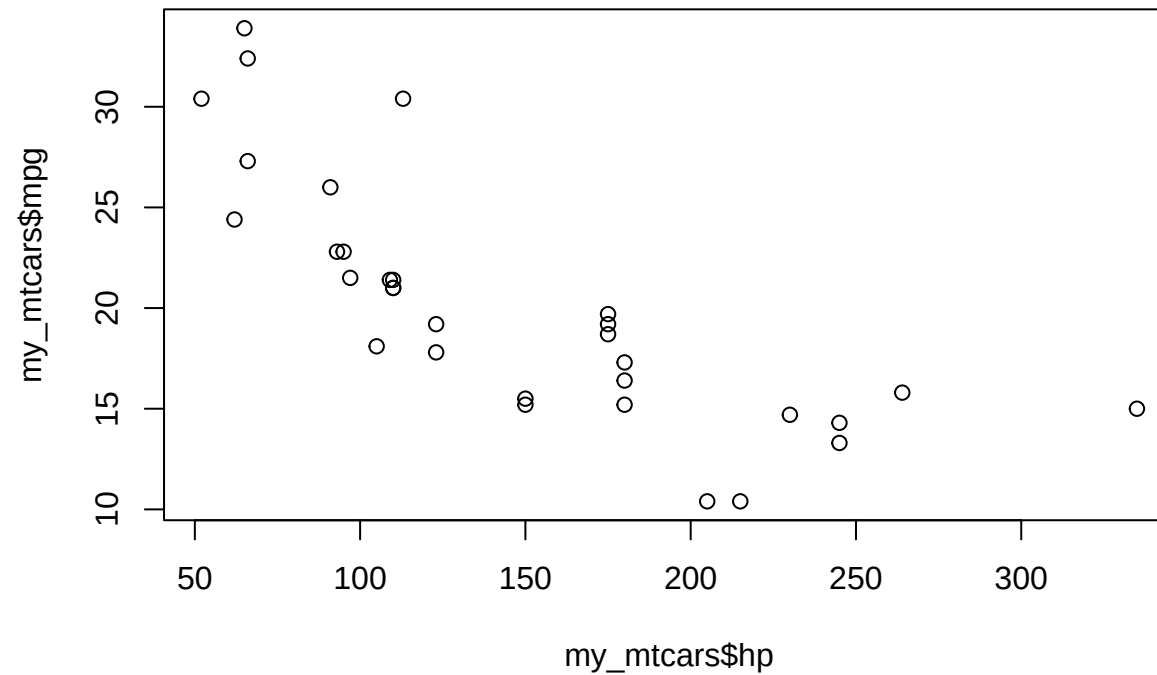
9.9 X-Y plot

If you apply the `plot` function to two numeric vectors of equal length, the result is an X-Y point plot, also known as a scattergram or scatterplot.

The following command generates a simple scattergram between the two variables `hp` (on the abscissa) and `mpg` (on the ordinate) of the data set `my_mtcars` which is a variant of `mtcars`:

```
> plot(x = my_mtcars$hp, y = my_mtcars$mpg)
```

time series. These object classes are not good for object classes for acyclical time series (e.g. days in the year due to the different different number of days, time series with gaps, e.g. due to public public holidays) and time series with a fine granular resolution (e.g. days and times with days and times with different time zones as in the airline data from the `nycflights13` library). Therefore, since WS 2023/24 in the course *Statistical forecasting methods* the object classes `tibble` and `tsibble` from the Tidyverse and Tidyverts worlds instead of the R-base classes `ts` and `mts` are used. These use date and time formats such as `POSIXct` and `POSIXlt` using functions from the R package `lubridate`. If you are dependent on using POSIX representations for (possibly irregular) POSIX representations within the R base system for (possibly irregular) time series system, you can also use the R packages `zoo` and R packages `zoo` and `xts`.

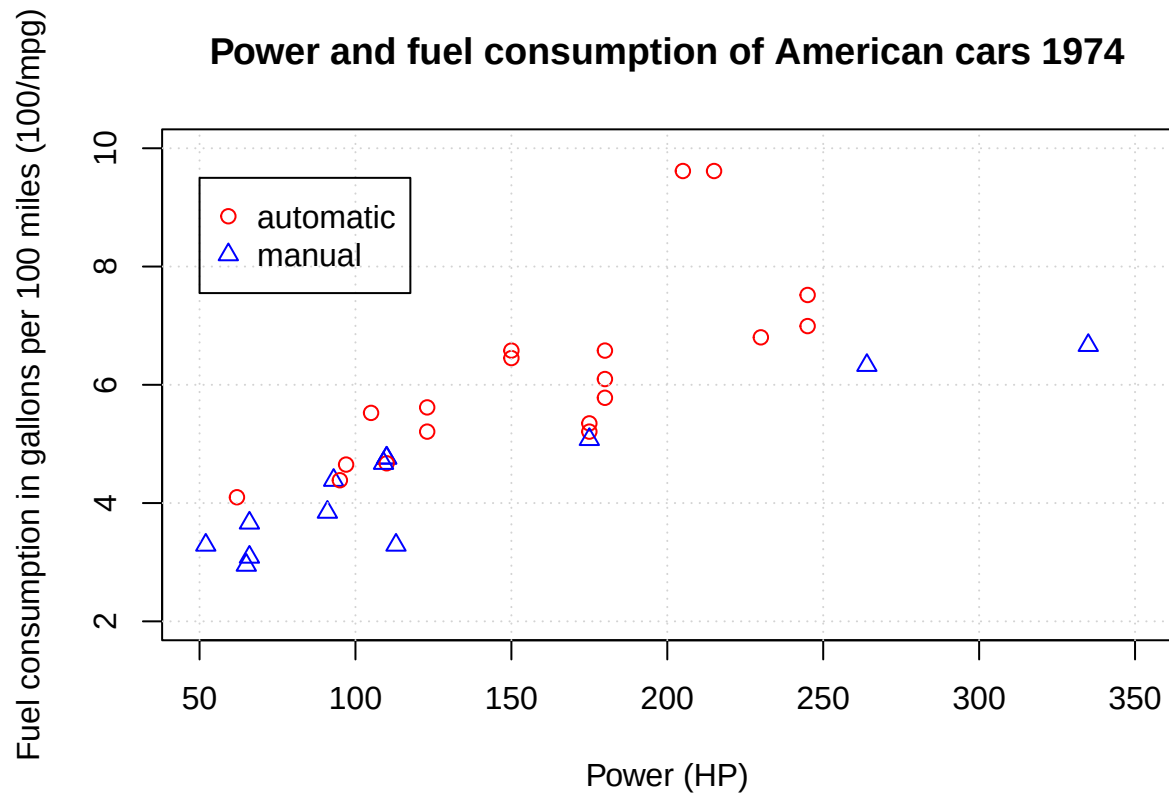


9.10 X-Y plot extended and annotated

The plots created with the R base system **graphics** can be modified and extended with extraordinary flexibility. The X-Y plot shown below provides a first insight into these possibilities.

```
> with(my_mtcars,
+   plot(x = hp,
+       y = 100 / mpg,
+       main = "Power and fuel consumption of American cars 1974",
+       xlim = c(50, 350),
+       ylim = c(2, 10),
+       xlab = "Power (HP)",
```

```
+      ylab = "Fuel consumption in gallons per 100 miles (100/mpg)",  
+      col = ifelse(am == "automatic", "red", "blue"),  
+      pch = ifelse(am == "automatic", 1, 2))  
> grid()  
> legend(50, 9.5,  
+      legend = c("automatic", "manual"),  
+      col = c("red", "blue"),  
+      pch = c(1, 2))
```



In fact, the graphics functions of the R base system can be used to create complex and sophisticated graphics. See for example Rahlf (2014) and Rahlf (2018), where you can also find very useful templates for interesting special cases (such as population pyramids).

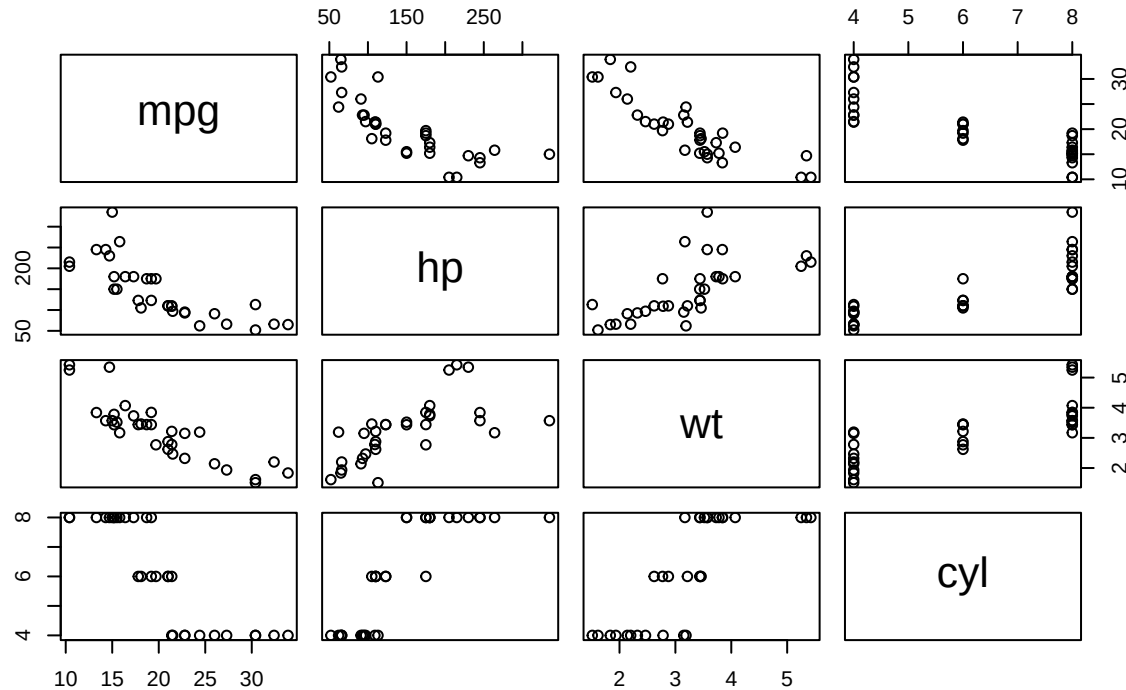
9.11 Tile or panel plot with graphics

Bivariate plots are excellent for identifying details in medium-sized data sets for two variables. However, if you apply the `plot` function to a data frame with more than two numeric variables, you will get all bivariate plots between all possible variable pairs. These panel charts have two characteristics: First, two charts are created for all pairs of variables `X` and `Y`, `plot(X,Y)` and `plot(Y,X)`. This reduces the ability to detect details. Second, the graphing parameters of the `plot` function are applied to all pairs, which limits controllability of individual pairs.

Please note that the following `plot` does not show the modified data set `my_mtcars`, but the original data set `mtcars`, so that the actual number of cylinders can be recognized. When using the factor variable `cyl` in `my_mtcars` only the codes (1 for `cyl==2` etc.) on the scale {1, 2, 3} would be given. Accordingly, care must be taken when using the `plot` function if metric variables have been recoded as factor variables.

```
> plot(mtcars[ , c("mpg", "hp", "wt", "cyl")],  
+      main = "Miles per Gallon (mpg), Horsepower (hp), Weight (wt), Cylinder (cyl)")
```

Miles per Gallon (mpg), Horsepower (hp), Weight (wt), Cylinder (cyl)



9.12 Global graphic parameters with `par`

Numerous graphic parameters can be set globally in R using the function `par` function. These include, in particular, parameters for the arrangement of graphics and for defining margins. You can also set parameters that only affect a specific graphic. The latter are often (as above with `pch` and `col`), but not always, specified via the three-point argument (`...`) locally in the argument of a graphics function.

It should be noted that local and global parameter settings with `par` with abstracting graphics libraries such as `ggplot2` do not usually work, as independent and `ggplot2`-specific parameters must be used for this purpose.

The most important *global* parameters that can be set via `par` are:

- `mfrow` and `mfcpl` to control the number and arrangement of graphics. For example, `mfcpl(c(3,2))` ensures that three lines of graphics are arranged one below the other with two graphic columns next to each other so that six consecutive graphics are placed in the following matrix position order (11, 21, 31, 12, 22, 32).
- `oma` defines the *Outer Margin* in *lines*, `omi` the *Outer Margin* in *inches*¹¹, and `mar` the *Plot Margins* in *lines*.

The somewhat confusing anatomy of the graphics and layout parameters is described in Venables & Ripley (2002) on pages 81 and 85 and in the Cheat sheet How Big is your graph?¹² on the *Cheat-Sheet* page of Posit¹³.

Plot symbols, line types and colors are mainly set *locally* via the three-point argument (`...`). The most important parameters are

- `pch` for plot symbols,
- `lty` for line types and
- `col` for colors.

`lty` and `pch` are defined by numerical parameters. The codes can be found in Venables & Ripley (2002) on p. 74 as well as interactively in the ggplot2 vignette (callable via `vignette('ggplot2-specs')`), since the associated numerical codes are among the few parameters that match in the R libraries `ggplot2` and `graphics`.

The above scatterplot can be modified, for example, by the following R code chunk in which a new margin and new colors are defined:

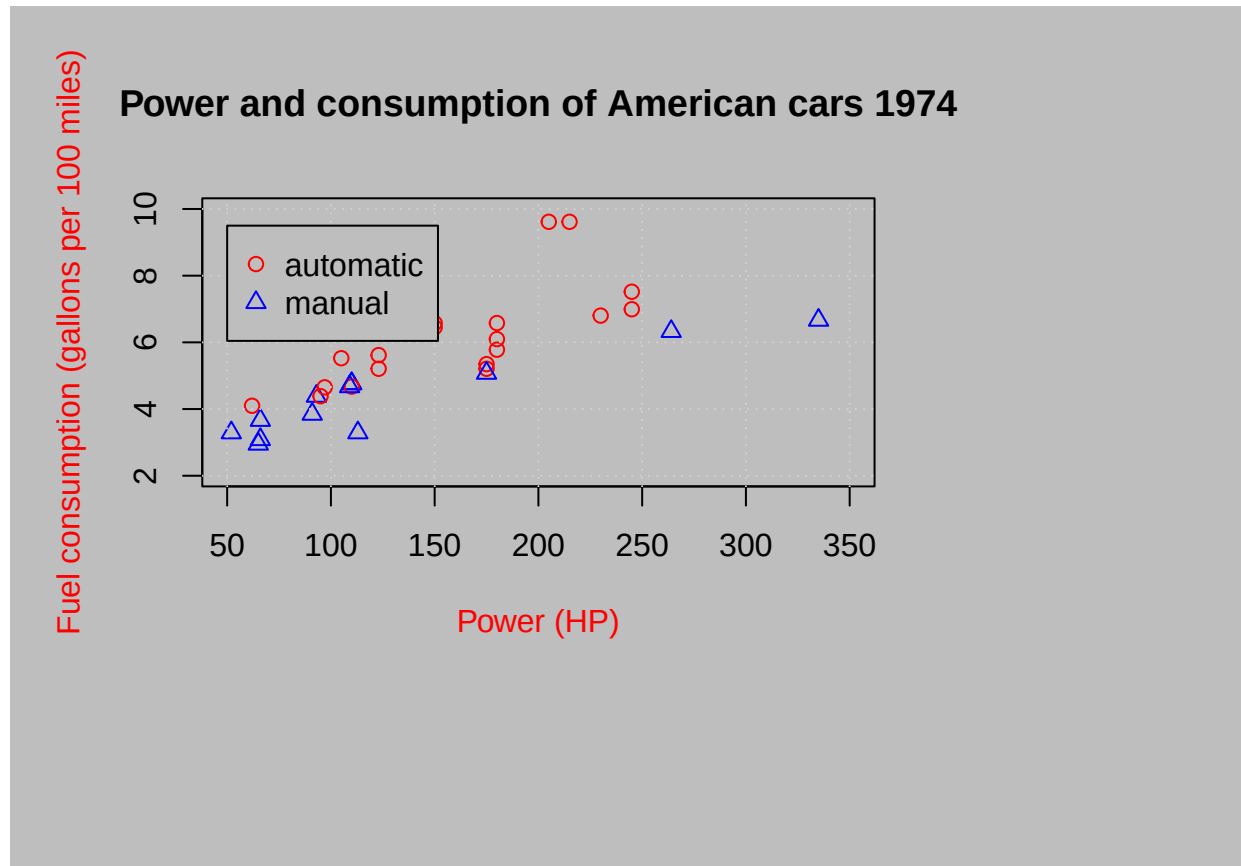
```
> old_par <- par()
> par(bg = "gray",
+     col.lab = "red",
+     mar = c(10, 5, 5, 10))
> with(my_mtcars,
+     plot(x = hp,
+          y = 100 / mpg,
+          main = "Power and consumption of American cars 1974",
+          xlim = c(50, 350),
+          ylim = c(2, 10),
+          xlab = "Power (HP)",
+          ylab = "Fuel consumption (gallons per 100 miles)",
```

¹¹1 inch are 2,54 cm.

¹²<https://rstudio.github.io/cheatsheets/how-big-is-your-graph.pdf>

¹³<https://posit.co/resources/cheatsheets/>

```
+         col = ifelse(am == "automatic", "red", "blue"),  
+         pch = ifelse(am == "automatic", 1, 2)))  
> grid()  
> legend(50, 9.5,  
+        legend = c("automatic", "manual"),  
+        col = c("red", "blue"),  
+        pch = c(1,2))
```

```
> suppressWarnings(par <- par(old_par))
```

The function `suppressWarnings(<function_call>)` is used to suppress the warning, that device dependent physical parameters like `cin`, `cra`, `csi`, `cxy`, `din` and `page` cannot be set.

Modified graphics are necessary for publication graphics and business reports in order to comply with externally set standards¹⁴. For

¹⁴Compared to `par`, `ggplot2` facilitates the definition of parameters, which can be used to predefine your own standards. can be predefined.

interactive data analysis, the default settings are usually sufficient.

Graphics can also be annotated and supplemented, as illustrated above with `grid()` and `rug(<variable>)`.

9.13 Overview of some important functions of the R package `graphics`

Important functions are

- `lines`, with which (x-y) points are connected.
- `points`, which can be used to display (x-y) points separately – also as a supplement to the `lines` graphic.
- `text`, which can be used to plot (*short*) texts at predefined (x-y) points.
- `mtext`, with which you can write short strings at the four margins.
- `abline`, which can be used to plot lines (regression lines, but also horizontal and vertical lines) can be plotted.
- `title`, with which you can enter a graphic title (as an alternative to the `plot` parameter `main`).
- `axis`, with which you can define sides, positions, labels, tick marks etc. can be defined.
- `legend`, with which you can create and position legends.

9.14 X-Y plot with regression line

The following code segments illustrate some graphical functions and parameters using a simulated regression model. The first code segment generates 20 pairs of observations (x_i, y_i) using the model

$$y_i = \beta_0 + x_i\beta_1 + \epsilon_i \text{ for } i = 1, \dots, 20.$$

The components have the following meanings for n observation indices $i = 1, \dots, n$: the numeric regressors $x_i \stackrel{iid}{\sim} N(10, 2^2)$, the independent and normally distributed error terms $\epsilon_i \stackrel{iid}{\sim} N(0, 2^2)$ with expectation 0 and variance 4, the constant (intercept) parameter $\beta_0 = 3$ and the slope parameter $\beta_1 = 2$. The symbol $\stackrel{iid}{\sim}$ means “*independent and identically distributed*”.

As is common with the classical regression model, it is assumed that regressors x_i are assumed to be fixed (not stochastic). For simulation purposes, however, the random number generator `rnorm` with fixed random number generator seed will be used. If we fix x and vary the random term ϵ we could measure the distributional properties of parameter estimates conditional on fixed regressors x_i for $i = 1, \dots, n$ by simulation.

```
> set.seed(123)      # Fix the starting value of the random number generator
> n <- 20             # Sample size
```

```
> x <- rnorm(n, mean = 10, sd = 2) # Simulation of a regressor variable
> e <- rnorm(n, sd = 2)           # Simulation of the error term
> b <- c(3,2)                     # Fixed regression parameters selected
> y <- b[1] + b[2] * x + e        # Generate the model  $y(i) = 3 + 2x[i] + e[i]$ 
```

The first observation is then deterministically overwritten to generate an outlier.

```
> y[1] <- 30 # Generate an outlier (replaces first observation)
> x[1] <- 7
```

Note: The courses *Applied Statistical Methods (ASM)* or *Data and Regression Analysis (DRA)* teach basic knowledge for identify outliers in linear regression models. *ASM* was last held in the 2024/25 winter term. A concise but older summary in German language can be found in Küsters & Kalinowski (2001).

The regression model is estimated with the R functions (`lm`). The resulting object `m` has class membership `lm`. Therefore the estimation results are summarized by the application of the generic function `summary` to the object `m`. There are several other generic functions which can be applied to `lm`-objects, e.g. the regression estimators can be extracted with the generic function `coef` using `coef(<model>)`:

```
> m <- lm(y~x) # Estimate regression model
> summary(m)   # Output of the 'most important' results.
```

Call:

```
lm(formula = y ~ x)
```

Residuals:

Min	1Q	Median	3Q	Max
-3.9025	-2.0189	-0.6985	1.2552	10.3625

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	9.9904	3.6319	2.751	0.013151 *
x	1.3782	0.3497	3.941	0.000958 ***

Signif. codes:

0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

```

Residual standard error: 3.138 on 18 degrees of freedom
Multiple R-squared: 0.4632, Adjusted R-squared: 0.4333
F-statistic: 15.53 on 1 and 18 DF, p-value: 0.0009584
> coef(m)      # Extract parameter estimates for beta[1] and beta[2]
(Intercept)      x
   9.990399    1.378158

```

In the following regression, the first observation is excluded by the use of the `subset` argument of the function `lm`. By doing this, the user-defined outlier's influence on observation $i = 1$ is eliminated.

```

> m1 <- lm(y~x, subset = 2:n)
> summary(m1)

Call:
lm(formula = y ~ x, subset = 2:n)

Residuals:
    Min       1Q   Median       3Q      Max
-2.9980 -1.0966 -0.4872  1.5960  2.3488

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)   4.2849     2.0943   2.046  0.0566 .
x             1.8764     0.1988   9.437 3.6e-08 ***
---
Signif. codes:
  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 1.662 on 17 degrees of freedom
Multiple R-squared: 0.8397, Adjusted R-squared: 0.8303
F-statistic: 89.05 on 1 and 17 DF, p-value: 3.596e-08
> coef(m1)
(Intercept)      x
  4.284925    1.876372

```

The following sequence plots all pairs of observations and supplements them by two regression lines.

- The red regression line for all observations including $i = 1$ (result of the model `m`)
- The blue regression line for all observations with the exception of the first observation (result of the model `m1`), observations $i = 2, \dots, 20$.

In addition, the axis labels (`xlab` and `ylab`), the value range of the abscissa (`xlim`) and the point symbol (`pch`) are modified. Somewhat unusual is the use of the formula interface `plot(y ~ x, ...)` instead of the predominantly used direct specification `plot(x, y, ...)`. However, the formula interface (see `?formula`) works not only for plots with the `graphics` and `lattice` libraries, but also for numerous models (usually regression or classification models like `lm` and `glm`) including variance-analytical designs, as well as in some machine learning methods (e.g., `nnet`).

With the help of a partial substitution of expressions via the `bquote` function, simple formula labeling can also be done. The source code below illustrates this as an example with the labels of the abscissa and the ordinate (the two x-y labels). Furthermore, text is placed separately at position (10, 16). Simple syntax examples can be found in the `demo("plotmath")`, whereby numerical estimation results with rather simple formulas can be mixed¹⁵.

The graph is supplemented by marking the outlier with `points` by means of a modified symbol `pch=3` (cross) in oversize (`cex=3`), supplementary texts, and a red, directed arrow (`arrow`) using the arguments origin (`x0, y0`) and target (`x1, y1`), as well as the angle (`angle`) and length (`length`) of the arrowhead.

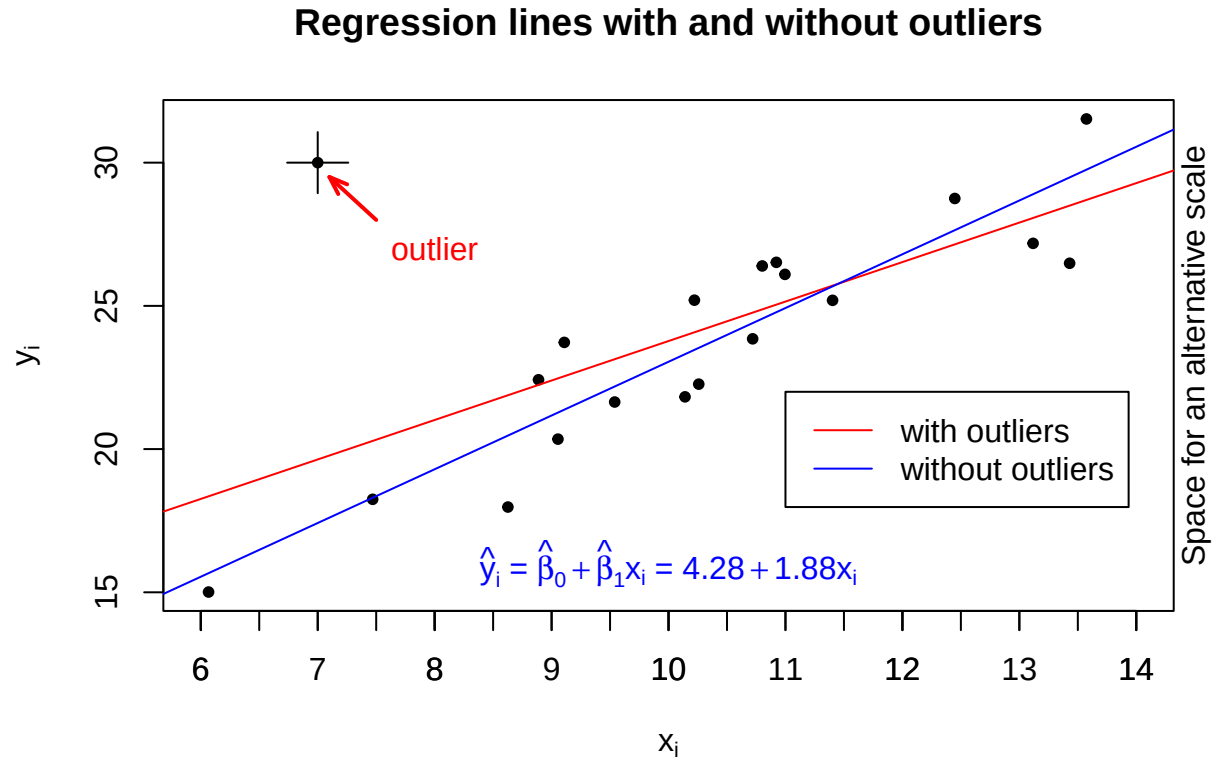
```
> plot(y ~ x,                      # 'formula' alternative to plot(x, y, ...)
+      xlim = c(6, 14),
+      pch = 20,                    # pch=20 means point
+      xlab = bquote(x[i]),
+      ylab = bquote(y[i])) # Formula label
> abline(m, col = "red")
> abline(a = coef(m1)[1],
+       b = coef(m1)[2],
+       col = "blue")
> title("Regression lines with and without outliers")
> legend(11,22,c("with outliers", "without outliers"),
+       lty = 1,
+       col = c("red", "blue"))
> points(x[1], y[1], pch = 3, cex = 3)
```

¹⁵With the help of the R package last updated in November 2022 `latex2exp`, formulas can also be translated to a limited extent using the *LaTeX* syntax into `plotmath` expressions.

```

> text(x[1]+1, y[1] - 3, "outlier", col = "red")
> arrows(x0 = x[1] + 0.5, x1 = x[1] + 0.1,
+        y0 = y[1] - 2, y1 = y[1] - 0.5,
+        length = 0.1,
+        angle = 20,
+        col = "red",
+        lwd = 2)
> mtext("Space for an alternative scale", 4)
> axis(1, at = 6:14)
> axis(1, at = seq(6, 14, 0.5), labels = FALSE)
> b0 <- round(coef(m1)[1], 2)
> b1 <- round(coef(m1)[2], 2)
> text(10,16,
+      bquote(paste(hat(y)[i],
+                    ' = ',
+                    hat(beta)[0] + hat(beta)[1] * x[i], # Math formula
+                    ' = ',
+                    .(b0) + .(b1) * x[i])),          # Numerical estimates
+      col = "blue")

```

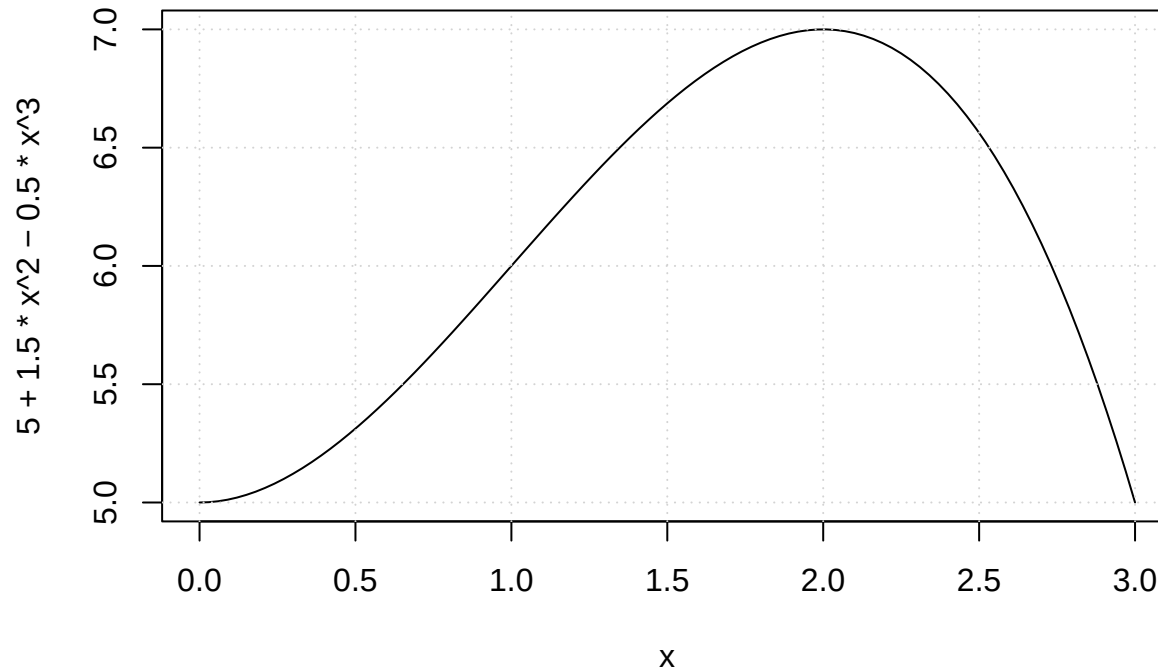


With creative talent, the R package `graphics` you can used to construct very appealing publication graphics. Otherwise I recommend to taking a cue from Rahlf (2014) or Rahlf (2018).

9.15 Plotting functions in graphics

With the help of the function `curve` defined in the R package `graphics` you can plot functions of a one-dimensional variable. The function must be defined by an expression. The easiest way is to embed the function directly in the function `curve`, as in the following example for a polynomial of the third order. The expression `expr` contains the scalar function $f(x) = 5 + 1.5x^2 - 0.5x^3$ which is dependent on x and which is plotted for the interval `(from, to)` with axes augmented by a grid. The default setting of the variable name is `'x'`, but this can be changed.

```
> curve(expr = 5 + 1.5*x^2 - 0.5*x^3, from = 0, to = 3, axes = TRUE)
> grid()
```



`curve` is often used to get a quick impression of a univariate functional curve.

9.16 Excursus: Generating functions with R

In the field of machine learning, AI, and also prognostics, there is always a need for functions that generate functions¹⁶. The necessary programming technique is illustrated here using an example in which three trigonometric functions of the form

¹⁶Do not confuse the term *generating functions* in R with the term *generating function* in probability theory and statistics. The former denote R functions which return R functions, while the latter (often denoted as *moment generating functions*) are mathematical functions which belong uniquely to a probability or density function. These functions are used to determine central and non-central moments of distributions like expectation, variance, skewness etc.

$$f(x|\alpha, \beta, \gamma) = \alpha \cdot \sin(x \cdot \beta + \gamma)$$

with flexible parameterization for α , β and γ are plotted with the function `curve`. Here α is the amplitude, β the frequency, and γ the phase. The values β and γ are specified in radians, not necessarily normalized to the range $[0, 2\pi]$. The following R program shows parameterized R meta function with three parameters (α, β, γ) . Each call of the meta function returns a function that only depends on *one* argument x .

```
> make_my_sin_function <- function(amplitude = 1, phase = 0, freq = 1) {
+   myf <- function(x) {
+     return(amplitude * sin(x * freq + phase))
+   }
+   return(myf)
+ }
```

Three subsequent function calls of `make_my_sin_function` define *three* differently parameterized variants of the sine function. Note that each function call contains three numeric arguments as arguments. However, no data is returned, but an executable R function with a single argument x ; the three parameters α , β , and γ are “hidden” in the local environment of the returned function.

```
> my_5sin_f1 <- make_my_sin_function(5)
> my_3sin_f2 <- make_my_sin_function(3, 0, 2)
> my_2cos_f1 <- make_my_sin_function(2, pi / 2)
```

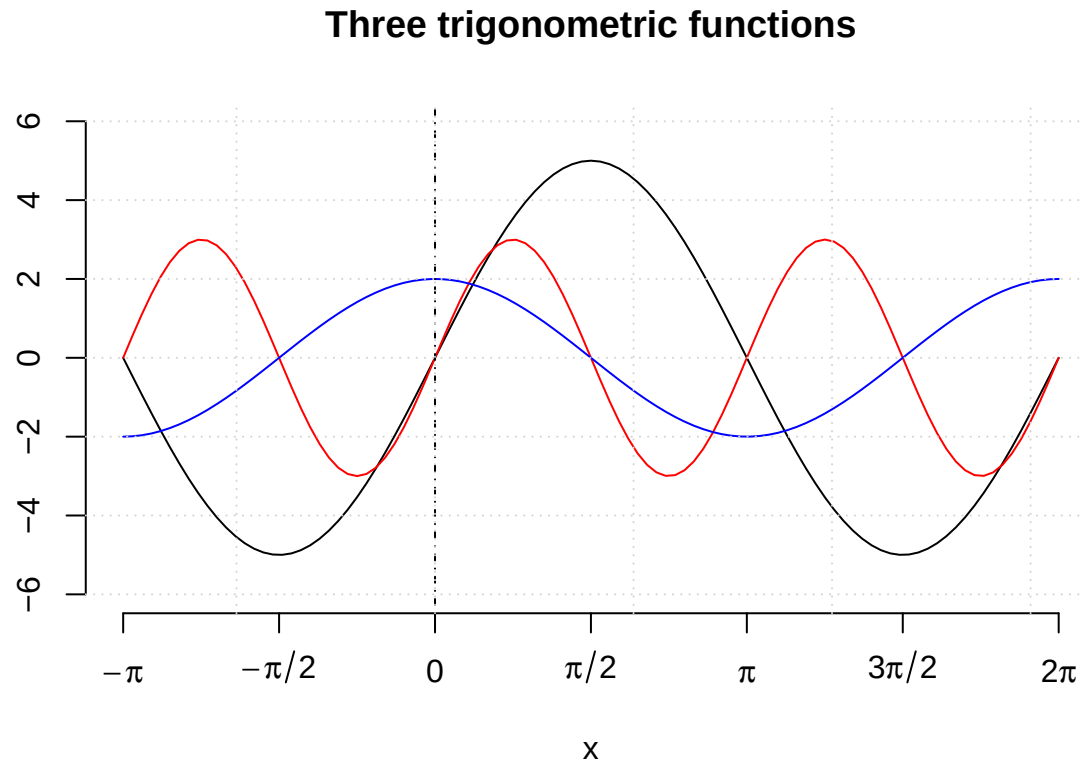
The `curve` function defined in the R package `graphics` takes as its first argument an expression that contains a function dependent on x only¹⁷. Additionally, the arguments include the value range (`from` and `to`) as well as two additional graphic parameters¹⁸.

```
> curve(expr = my_5sin_f1(x),
+       from = -pi,
+       to = 2 * pi,
+       axes = FALSE,
+       ylim = c(-6, 6),
```

¹⁷Functions, expressions, and data in R are so-called *first-order objects*, i.e., R functions can use functions and data as arguments as well as returns. In classical programming languages, on the other hand, there is often a strict separation between data and function, whereby functions only take data as arguments and also return only data.

¹⁸By the way, the switch `add` can be used to specify whether an existing plot is supplemented (`add = TRUE`) or whether a new curve is created (`add = FALSE`); the latter is the default setting. The function `axis`, applied to the abscissa (first parameter 1), illustrates an X-axis scale especially suitable for trigonometric functions which use radians as arguments.

```
+      ylab = "")
> axis(1,
+      at = seq(-pi, 2 * pi, pi / 2),
+      labels = expression(-pi, -pi / 2, 0, pi / 2, pi, 3 * pi / 2, 2 * pi))
> axis(2, seq(-6, 6, 2), ylab = "")
> curve(expr = my_3sin_f2(x), from = -pi, to = 2 * pi, add = TRUE, col = "red")
> curve(expr = my_2cos_f1(x), from = -pi, to = 2 * pi, add = TRUE, col = "blue")
> abline(v = 0, lty = 4)
> grid()
> title("Three trigonometric functions")
```



Trigonometric functions are frequently used in applied statistics and econometrics. Examples are the representations of the roots of ARMA model polynomials, the forecasting functions of ARMA models as well as the Fourier and spectral analysis of time series.

A common technique was used in the above definition. The function¹⁹ `make_my_sin_function` takes up to three numerical parameters and returns a function in which the transferred parameters are stored locally in an environment.

First you generate a function with two arguments `amplitude` and `phase` while the default `freq = 1` is used as the third argument.

```
> my_2cos_f1 <- make_my_sin_function(2,pi/2)
```

But printing the result by

```
> print(my_2cos_f1)
function (x)
{
  return(amplitude * sin(x * freq + phase))
}
<bytecode: 0x00000218afc9f4d8>
<environment: 0x00000218afc03820>
```

does not reveal anything about the internal (hidden) arguments.

But the values of the hidden arguments used in the definition call can be determined by two steps. In a first step all names contained in local environment of the function `my_2cos_f1` are determined:

```
> ls(envir = environment(my_2cos_f1))
[1] "amplitude" "freq"      "myf"       "phase"
```

But the variable assignments stored internally in the environment are still not obtained with this call. In a second step the data stored in the local environment `environment(my_2cos_f1)` of the function `my_2cos_f1` can be determined with the access functions like `get` and `mget`. The value of a variable (like `phase`) bound to the environment can be read with the `get` function, which accepts a defined environment as an argument²⁰:

¹⁹Each function consists of (1) the **formals** (i.e. the arguments), (2) the **body** and (3) the **environment**. Please note that for two function types, the so-called **primitives** (e.g. `[]`) and the **builtin's** (e.g. `sum`) there are no output of the body of the function (**body**). The reason for this is that these functions were not implemented in R, but within the R interpreter as *primitives* or via the C or Fortran libraries integrated therein.

²⁰Using `get("<varname>")` without a second argument determined the value within the current environment. For example `get("b1")` in the current interpreter session gives the value of the slope β_1 in the simulation experiment above, i.e., 1.88.

```
> get("phase", envir = environment(my_2cos_f1))
[1] 1.570796
```

The values of *all* the objects stored in the local environment of the function objects (including the function named locally with `myf` and returned via `return`) can be read out with `mget`. Here the return has a list structure:

```
> mget(ls(envir = environment(my_2cos_f1)), envir = environment(my_2cos_f1))
$amplitude
[1] 2

$freq
[1] 1

$myf
function (x)
{
  return(amplitude * sin(x * freq + phase))
}
<bytecode: 0x00000218afc9f4d8>
<environment: 0x00000218afc03820>

$phase
[1] 1.570796
```

With `assign` you can change a local variable, as you can see from the following example:

```
> get("amplitude", envir = environment(my_2cos_f1))
[1] 2
> my_2cos_f1(pi)
[1] -2
> assign("amplitude", 20, envir = environment(my_2cos_f1))
> get("amplitude", envir = environment(my_2cos_f1))
[1] 20
> my_2cos_f1(pi)
[1] -20
```

R treats data as well as functions and expressions as *first-order-objects*. This property makes it possible to program functions that not only handle data, but also functions

1. both as arguments and
2. return them as results.

Both properties were illustrated above with the functions `curve` and `make_my_sin_function` illustrated above. Using this approach numerous AI algorithms can be implemented directly.

9.17 Notes on the genesis of the R graphics functions

From today's perspective, configuring the graphics parameters in the R base system can be described as cumbersome. This can be explained by the history of S, the predecessor of R, as well as by the initial spread of the uncontrolled growth of open source development of R since the late 1990s. The statistical and graphical programming environment S was originally developed in the 1970s and 1980s at the world-renowned Bell-Labs²¹, which are also famous for numerous other innovations. At that time, it was common for graphics systems to use a classic paper-and-pencil technique (plotter approach). This approach, which is still used in the R base system, is rather old-fashioned, but cannot always be avoided, especially for highly customized graphics. In addition, the base graphics system is very fast.

For standardized graphics, the R graphics library is based on `grid` (not to be confused with the grid function `graphics::grid`). The R library `lattice` is based on the *Trellis* approach developed in the 80s for S by the Bell-Labs. `lattice` uses the formula interface of the R library originally developed for linear and generalized regression models (`lm` and `glm`). This is similar to the `tally` and `favstats` functions used in the introduction. Some `lattice` graphics are briefly discussed in the next chapter.

9.18 Excursus: `attach` and `detach`

In interactive analyses, you can reduce the amount of typing by using `attach`. By attaching a data set (often a `data.frame`, a `ts` or a `mts` object) with `attach` (e.g. with `attach(my_mtcars)`) inserts all variables of the object into the search path immediately after the global environment. However, this increases the **risk of name conflicts**. Therefore this procedure should only be used in exceptional cases. In my opinion the use of `attach` should be restricted to two cases: First for small interactive demonstrations in the lecture hall. Second to test R idioms in a separate R interpreter environment. Using `detach` the search path can be restored to the previous state. Example:

²¹https://en.wikipedia.org/wiki/Bell_Labs

```

> test_df <- data.frame(v1 = 1:4, v2 = letters[11:14])
> test_df # Test data
  v1 v2
1  1  k
2  2  l
3  3  m
4  4  n
>
> if(exists("v1")) cat("v1 = ",v1,"\n") else cat("v1 does not exist")
v1 does not exist
>
> # Check whether v1 can be reached in the search path
> search()
[1] ".GlobalEnv"          "package:ggimage"
[3] "package:OpenStreetMap" "package:nycflights13"
[5] "package:mosaicData"   "package:ggformula"
[7] "package:Matrix"       "package:lattice"
[9] "package:repmis"       "package:knitr"
[11] "package:summarytools" "package:DiagrammeRsvg"
[13] "package:DiagrammeR"   "package:webshot"
[15] "package:rsvg"         "package:lubridate"
[17] "package:forcats"      "package:stringr"
[19] "package:dplyr"        "package:purrr"
[21] "package:readr"        "package:tidyr"
[23] "package:tibble"       "package:ggplot2"
[25] "package:tidyverse"    "package:stats"
[27] "package:graphics"     "package:grDevices"
[29] "package:utils"        "package:datasets"
[31] "package:methods"     "Autoloads"
[33] "package:base"
> attach(test_df) # Include the variables from test_df
> search() # Search path, 'test_df' in second place
[1] ".GlobalEnv"          "test_df"
[3] "package:ggimage"     "package:OpenStreetMap"

```

```

[5] "package:nycflights13" "package:mosaicData"
[7] "package:ggformula"   "package:Matrix"
[9] "package:lattice"     "package:repmis"
[11] "package:knitr"        "package:summarytools"
[13] "package:DiagrammeRsvg" "package:DiagrammeR"
[15] "package:webshot"      "package:rsvg"
[17] "package:lubridate"    "package:forcats"
[19] "package:stringr"      "package:dplyr"
[21] "package:purrr"        "package:readr"
[23] "package:tidyr"        "package:tibble"
[25] "package:ggplot2"      "package:tidyverse"
[27] "package:stats"        "package:graphics"
[29] "package:grDevices"    "package:utils"
[31] "package:datasets"     "package:methods"
[33] "Autoloads"            "package:base"
>
> if(exists("v1")) cat("v1 = ",v1,"\n") else cat("v1 does not exist")
v1 = 1 2 3 4
>
> detach(test_df) # Remove integration
>
> if(exists("v1")) cat("v1 = ",v1,"\n") else cat("v1 does not exist")
v1 does not exist
>
> search()
[1] ".GlobalEnv" "package:ggimage"
[3] "package:OpenStreetMap" "package:nycflights13"
[5] "package:mosaicData" "package:ggformula"
[7] "package:Matrix" "package:lattice"
[9] "package:repmis" "package:knitr"
[11] "package:summarytools" "package:DiagrammeRsvg"
[13] "package:DiagrammeR" "package:webshot"
[15] "package:rsvg" "package:lubridate"
[17] "package:forcats" "package:stringr"

```

```
[19] "package:dplyr"      "package:purrr"
[21] "package:readr"      "package:tidyr"
[23] "package:tibble"     "package:ggplot2"
[25] "package:tidyverse"  "package:stats"
[27] "package:graphics"   "package:grDevices"
[29] "package:utils"      "package:datasets"
[31] "package:methods"    "Autoloads"
[33] "package:base"
> rm(test_df) # Delete test data
```


Chapter 10

Lattice graphics in R

10.1 Overview

The graphics of the R library `lattice` abstract from the *paper and pencil* *Philosophy* of the 70s and 80s and implement the Trellis the Trellis graphics described in detail in Cleveland (1993). In only a few selected graphic maps are described in this section described in this section, as most of the species can be plotted using the `ggplot2` package due to internal consistency due to the *Grammar-of-Graphs* developed by @Wilkinson2004 in the 90s. of Graphics* developed by @Wilkinson2004 in the 1990s. can be programmed. Due to the panel function, however, there are still some (now few) applications that require the use of the trellis developed in the R package `lattice` package is simpler. However complementary R libraries are increasingly being developed with which the Trellis the Trellis functionality in the `ggplot2`/tidyverse world. world. See, for example, the page [ggplot2 extensions](<https://exts.ggplot2.tidyverse.org/>), on which a whole `ggplot2` extensions can be found¹.

10.2 Basic structure of the `lattice` functions

The central feature of the `lattice` plots is the use of the `formula` interface, as used in regression models (e.g. `lm` and `glm`) to represent dependent and independent variables. is used. All functions have the structure

¹On the gallery² there are currently (April 19, 2025) a total of 148 registered extensions, of which only 18 explicit are integrated in `ggplot2`. are included.

```
> plotFunction(y ~ x | a * b, dataset)
```

on. The formula `y ~ x` describes which variables in an individual individual subplot on the ordinate (Y-axis) and abscissa (X-axis) axis, while the dash (`|`) indicates for which combinations individual for which combinations individual plots are created. If `a` and `b` are categorical, the x-y plots are created for all combinations of the of the values of the variables `a` and `b`. Sometimes the syntax `~<var>`, however, only one variable (possibly conditional) is included³.

By omitting the conditioning variables `a * b` and conditioning (`|`) you can create bivariate plots. Univariate and plots conditioned on `a` and `b`, if applicable, are created using the syntax `~x | a * b`⁴. For different graphic maps there are different functions such as

- `histogram(~ x | a * b)` for histograms (`x` metric),
- `barchart(~ x | a * b)` for bar charts (`x` often `factor` or `character` variable),
- `dotplot(~ x | a * b)` for dotplots,
- `bwplot(~ x | a * b)` for box plots, also referred to as box-whisker plots (`x` metric),
- `densityplot(~ x | a * b)` for density estimators (`x` metric) and
- `xyplot(y ~ x | a * b)` for two-dimensional plots (scattergrams, `y` and `x` usually metric).

If the variables `x`, `y`, `a` and `b` are not in the search path, but only available in a dataframe (e.g. `my_df`), you can use the above function calls with a `dataframe`. function calls with a `data` argument, e.g. `histogram(~ x | a * b, data = my_df)`.

The X-Y plots of two or more variables from a `data.frame` can be can be generated with the help of `spiom(dataframe)`. Furthermore there are in `lattice` - similar to the R package `graphics`- various QQ plots and variants of three-dimensional plots.

Lattice plots are only available after loading the R package `lattice` or by prefixing the package name with the format `lattice::<fkt_name>` (as for example in `lattice::bwplot(...)` available⁵.

10.3 lattice-examples ‘univariate’

A simple and purely univariate example is the reproduction of the already created above with the means of the basic library `graphics`. histogram created above. The English designation of the ordinate axis by “Percent of Total” is due to the use of an English language language setting during the installation of R, but can be replaced by `ylab = "Percent of Total"` as an additional graphic parameter as an additional graph parameter; see `?histogram`.

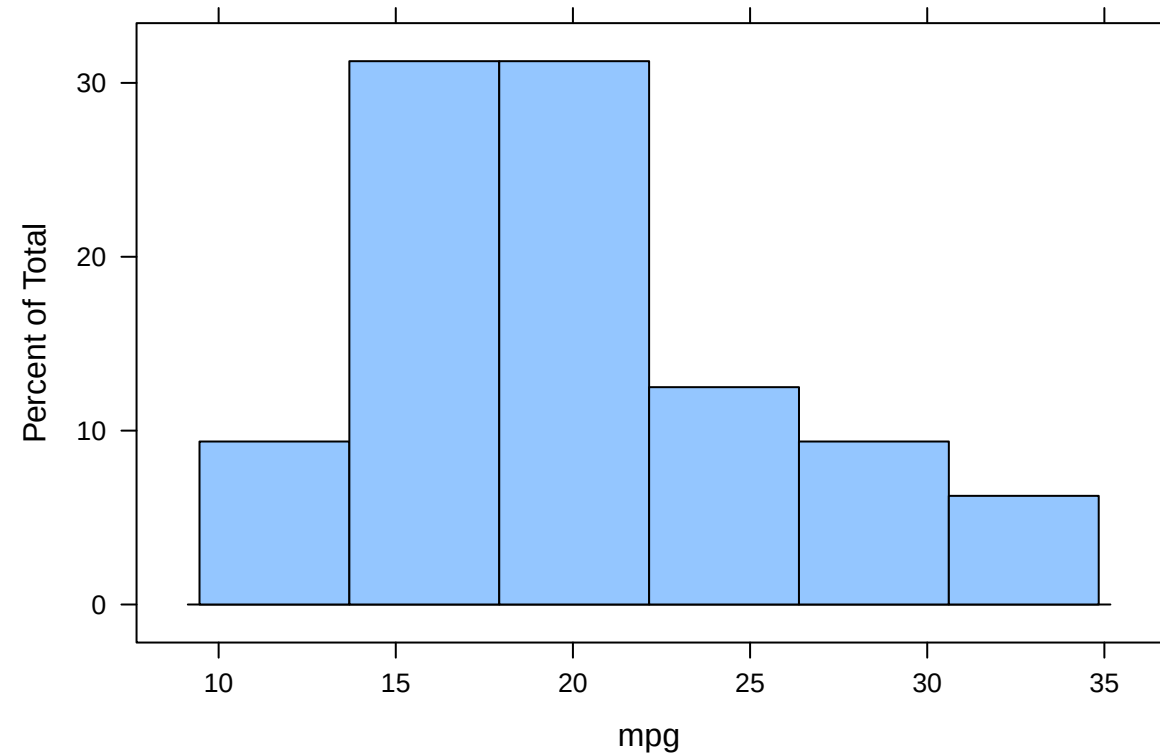
³The tilde operator `~` in the `formula` interface can be used monadically and dyadically; see the help `?tilde`.

⁴The tilde operator `~` in the `formula` interface can be used monadically and dyadically; see the help `?tilde`.

⁵See the help `?'::'`.

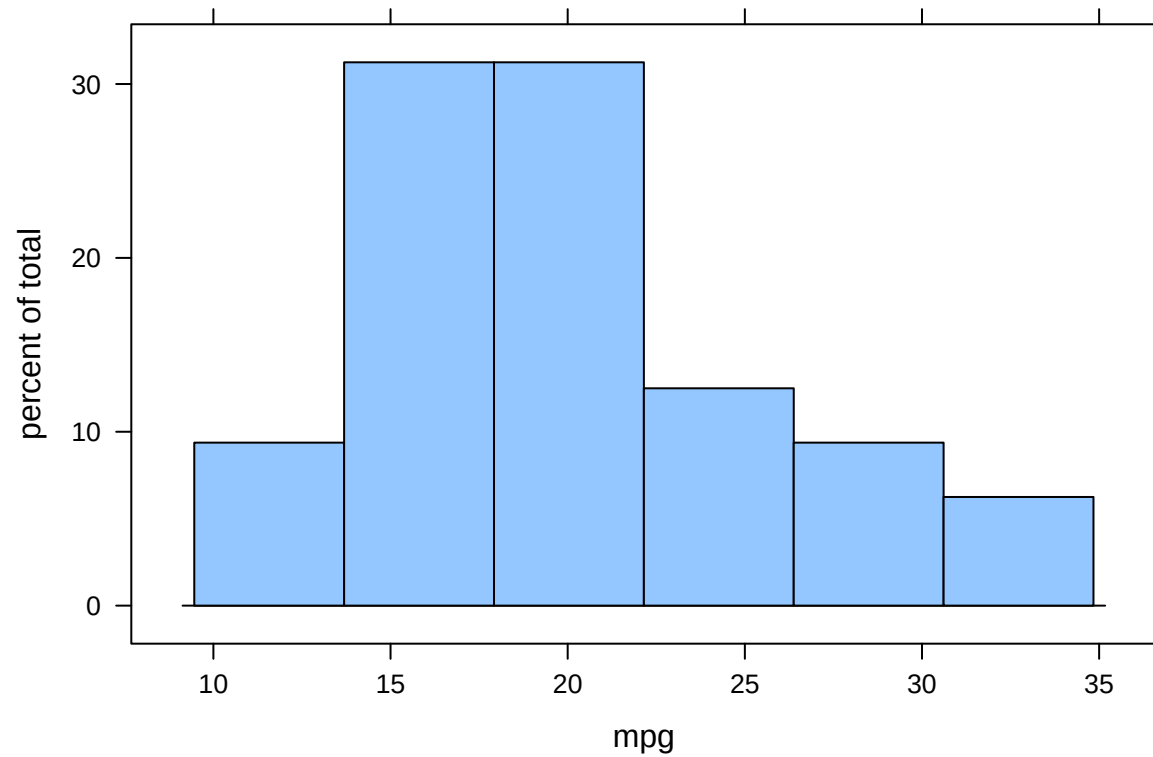
Note: With the help of the function call `Sys.setenv(LANGUAGE='en')` the language setting can be set to English, with `Sys.setenv(LANGUAGE='de')` to German. Up to R version 3 German-language installation of R, the character string “Prozent der total”⁶ was used as the default setting.

```
> library(lattice)
> histogram(~mpg, data = my_mtcars)
```



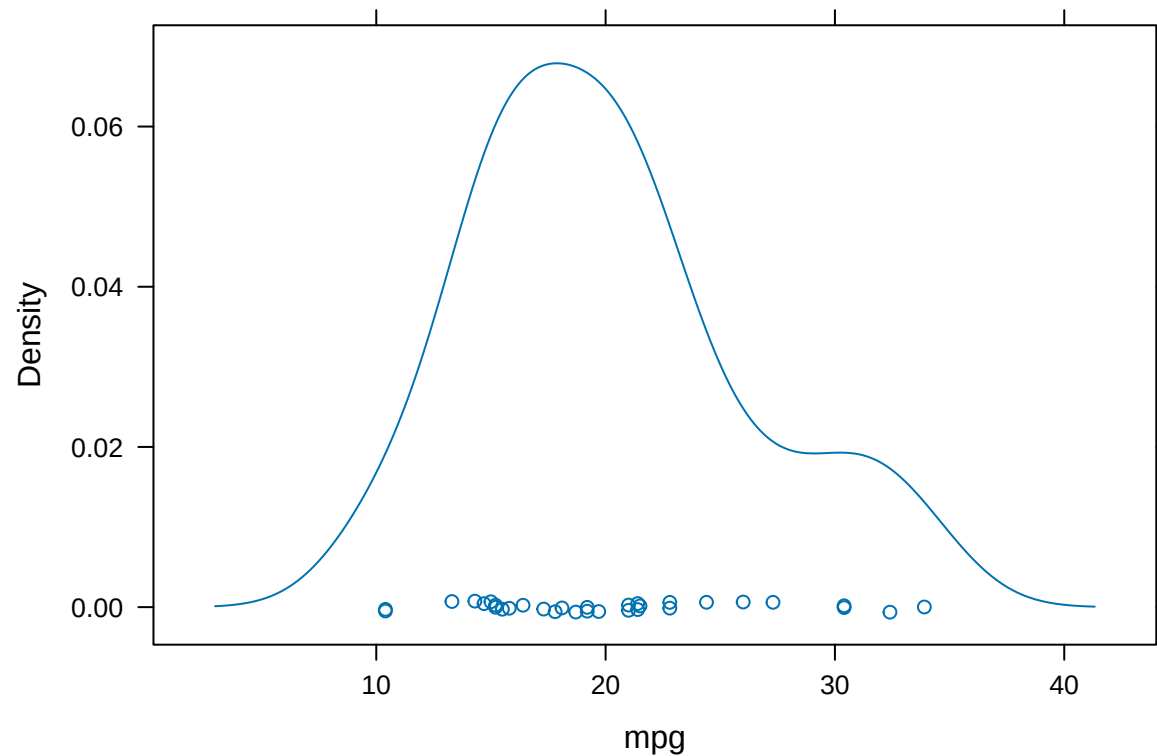
```
> histogram(~mpg, data = my_mtcars, ylab = "percent of total")
```

⁶The label defaults “Percent of Total” and “Percent of Total” are suboptimal, as the data can be both a population and a sample. population as well as a sample.



The following command produces the corresponding density estimate, which, by default, displays the individual observations as jittered points. (jittered) points by default. With the routines from `graphics` these observation points explicitly (and without jittering) `withrug` must be added. Again, the kernel functions and bandwidths of the kernel estimators as well as other parameters of the density function estimation can be controlled by additional parameters.

```
> densityplot(~mpg, data = my_mtcars)
```



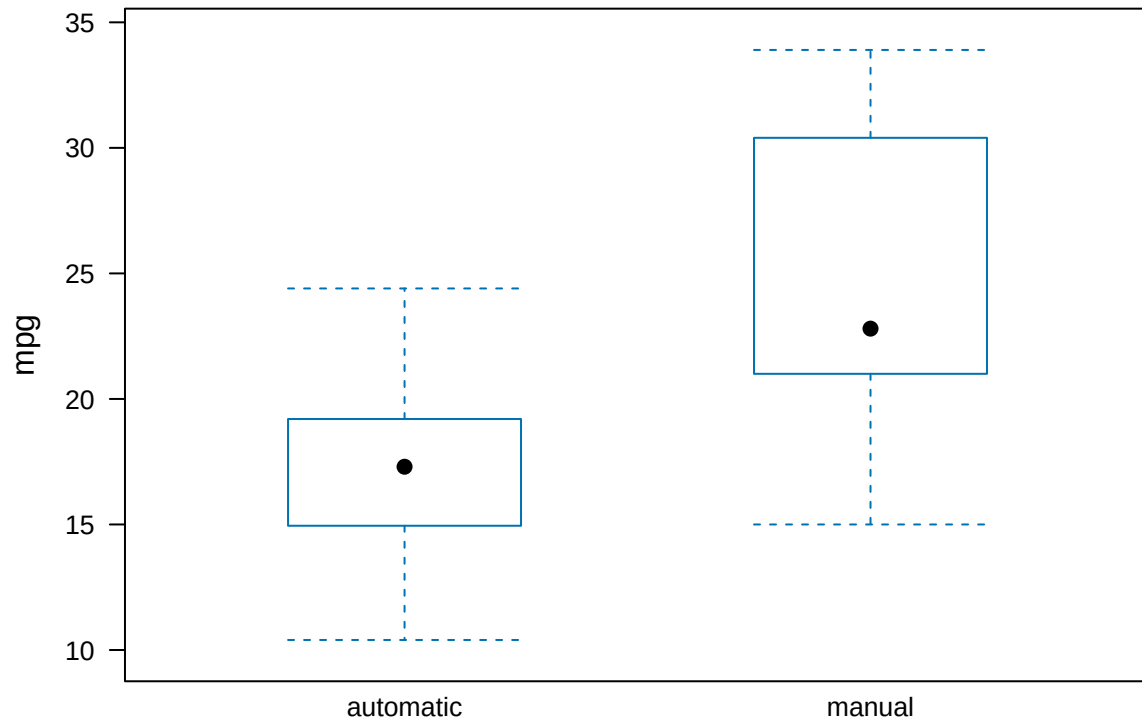
In this plot appears the ‘locale’⁷ dependent ordinate axis label “Density”, which can be overwritten by `ylab`. Clearly visible are the additionally drawn and scattered with the help of a random number generator ^{22]} individual observation points are clearly visible; this shows that `lattice` plots have a more abstract level than the `rug` plots supplemented by graphics’ plots. Note also that the density plot covers a larger range of values than the range of observations actually available in the observations actually available in the data set – this will be This is not the case with the corresponding functions of the `ggplot2` library discussed in the next section.

⁷Information on local settings, encodings etc. can be found in the Appendix “A1: Link list for R software, libraries, help and Introductions” to this course “Data Science Tools SS 2025”.

10.4 `lattice`-examples ‘bivariat’

Next, the first graphs differentiated according to groups are shown next. An example are the two box plots of `mpg`, conditioned to the two values ‘automatic’ and ‘manual’ of the variable `am`.

```
> bwplot(mpg ~ am, data = my_mtcars)
```



Here, too, you can see (similar to the above graph “Performance and consumption of American cars 1974”) that cars with automatic transmission have a consumption (corresponding to a lower number of miles per gallon) compared to manual transmissions – nowadays consumption hardly differs any more.

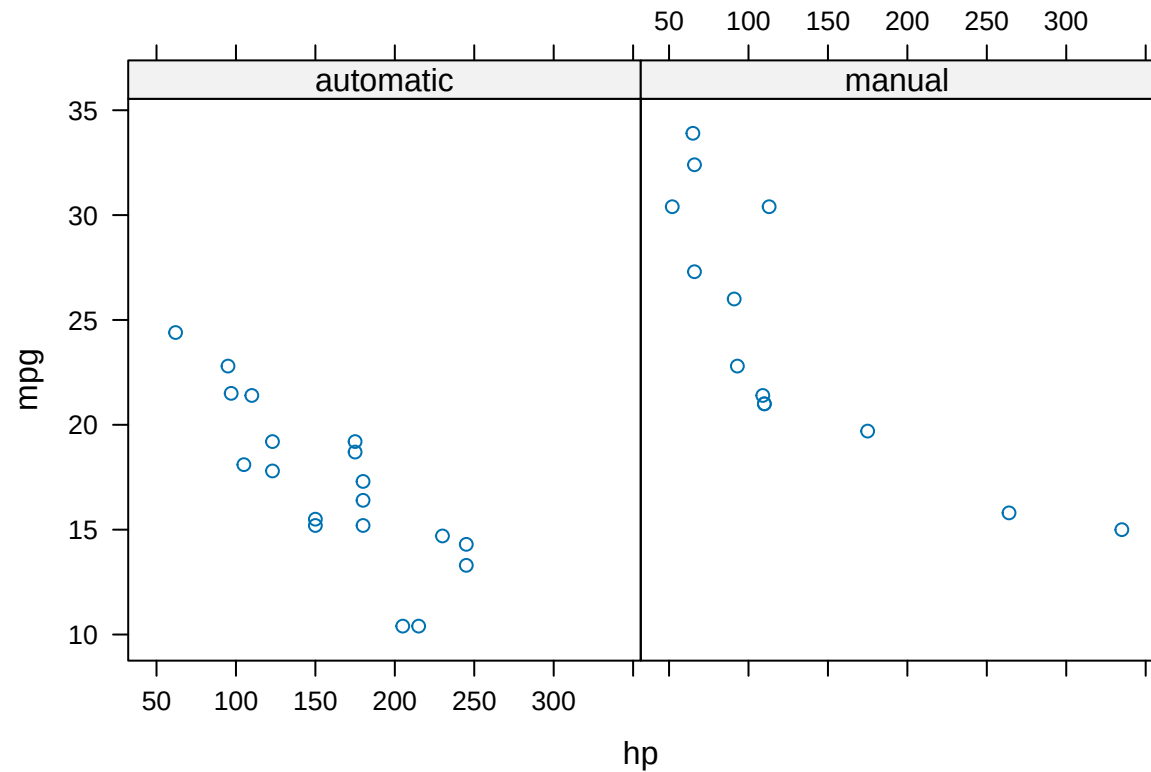
Exercise: As an exercise, you can generate the two plots with notches by adding the argument `notch = TRUE` in the function call. The exact interpretation is not trivial and requires a precise study of the underlying which are based on asymptotic theory. See `boxplot.stats`

and the literature cited there.

10.5 lattice examples with conditioning variables

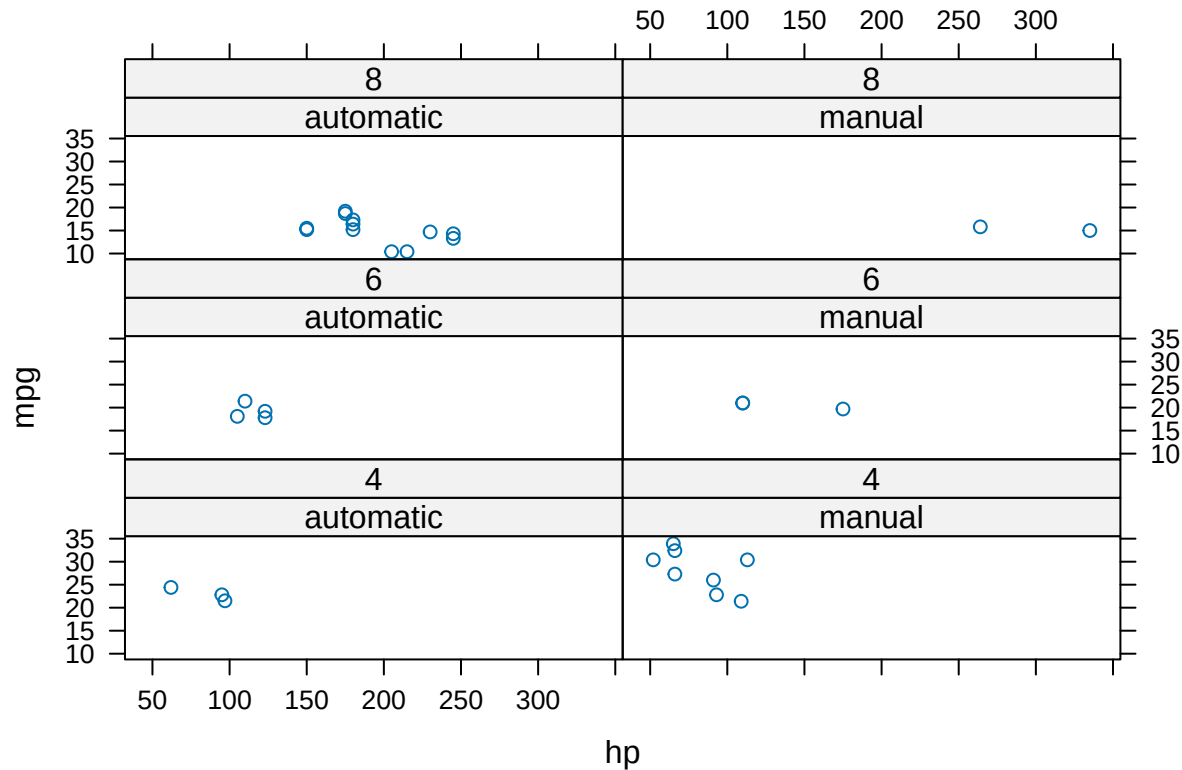
The following scattergram shows the negative correlation between `mpg` as Y-variable and `hp` as X-variable, differentiated according to the two circuit types. The more powerful the engine, the lower the number of miles number of miles that can be achieved per gallon. This corresponds to an increase in consumption increases with power, whereby in this data set the consumption of automatic vehicles is higher than that of manual transmission vehicles. Note that the scales of the X and Y axes in both subplots coincide.

```
> xyplot(mpg ~ hp | am, data = my_mtcars)
```



If you want to condition the X-Y plots between `mpg` and `hp` on the two variables `am` and `cyl`, this can be realized with the following call. the following call.

```
> xyplot(mpg ~ hp | am * cyl, data = my_mtcars)
```



It is not only the small number of observations per combination that is problematic. For example, in the plot of the cell

```
am == "manual" and cyl == 6
```

only **two** observations, although this combination includes **three** observations belong to this combination, but two of these observations overlap.

Such overlaps can only be analyzed with the help of a table, such as the one generated, for example, with the `tally` function contained in the R package `mosaic`. function contained in the R package⁸.

```
> mosaic::tally(am ~ cyl, data = my_mtcars)
      cyl
am      4  6  8
automatic 3  4 12
manual    8  3  2
> my_mtcars[my_mtcars$am == "manual" & my_mtcars$cyl == 6, c("mpg", "hp")]
      mpg hp
Mazda RX4    21.0 110
Mazda RX4 Wag 21.0 110
Ferrari Dino  19.7 175
```

For some data sets, you can avoid this display problem with ‘jittering’ can be avoided by extracting the observation values from the function `jitter` from the R package `base` and additionally using the argument `jitter.data=TRUE`, which is poorly documented in `lattice`:

```
> xyplot(jitter(mpg) ~ jitter(hp) | am * cyl, data = my_mtcars, jitter.data = TRUE)
```

However, this does not work in this case, as the double combination combination `mpg==21.0` and `hp==110` is still only included once in the subplot in the position (2,2), as the extent of the *dither* with the help of a random number generator is insufficient. Compare, for example, the original values with the dithered values, which only become apparent when an additional argument is set manually (here `factor=20`) when calling `jitter` from the original values from the original values:

```
> set.seed(123)
> head(with(my_mtcars, cbind(mpg, jitter(mpg), jitter(mpg, factor = 20))))
      mpg
[1,] 21.0 20.99150 21.15256
[2,] 21.0 21.01153 21.23637
```

⁸Numerous functions are available within R that can be used to create univariate and bivariate tables. Within the system, the `table` function is particularly suitable for this purpose. In the book by Baumer et al. (2017), however, the function the `tally` function from the R package `mosaic` is used for tabulation. `mosaic` is used for tabulation. `Mosaic` expects a data transfer via the argument `data`, which cannot be used directly for the piping principle of the *tidyverse*; piping can only be used indirectly through the placeholder through the placeholder “`data=.`” see the next chapter 3. Chapter 3. However, in recent years, on the basis of tidyverse modeling tidyverse modeling, suitable tabulation functions tabulation functions have also been developed in recent years on the basis of tidyverse modeling; an important representative is, for example `tabyl` from the R package `janitor`. Outside of these three packages other powerful tabulation functions exist, some of which some of which generate visually very appealing tables.

```
[3,] 22.8 22.79636 22.41969  
[4,] 21.4 21.41532 21.38224  
[5,] 18.7 18.71762 18.90677  
[6,] 18.1 18.08182 17.87313
```

So you have to be careful and check the data manually, e.g. with the manually, e.g. with the above table function `mosaic::tally(am ~ cyl, data = my_mtcars)` and subsequent case selection using

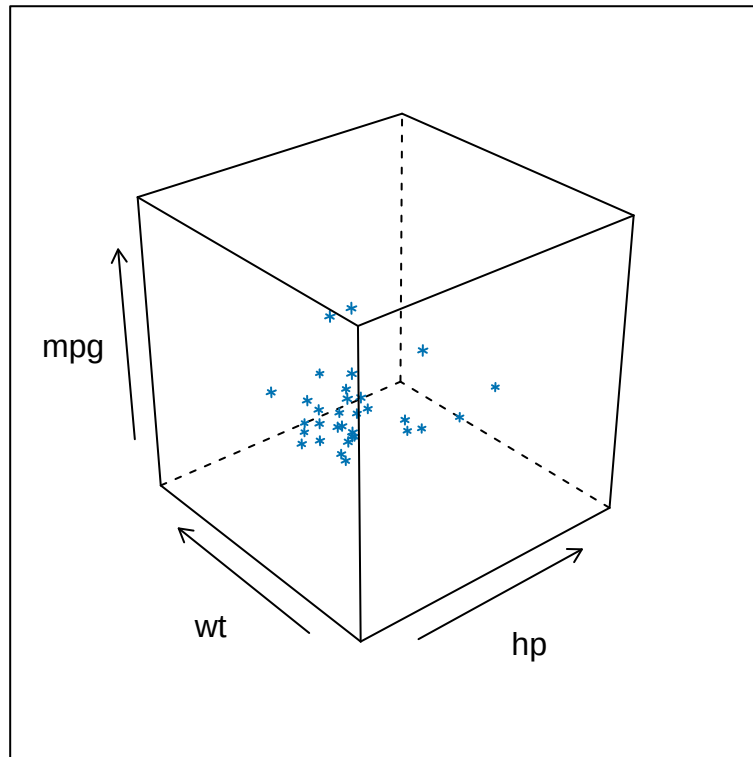
```
> my_mtcars[my_mtcars$am == "manual" & my_mtcars$cyl == 6, c("mpg", "hp")]
```

The libraries `tibble`, `tidyr` and `dplyr` of the *Tidyverse* universe provide a number of functions that can be used to to carry out such checks more easily and clearly; see “Chapter 3: Basic features of `tibble`, `dplyr` and `tidyr`”.

10.6 lattice examples for three variables

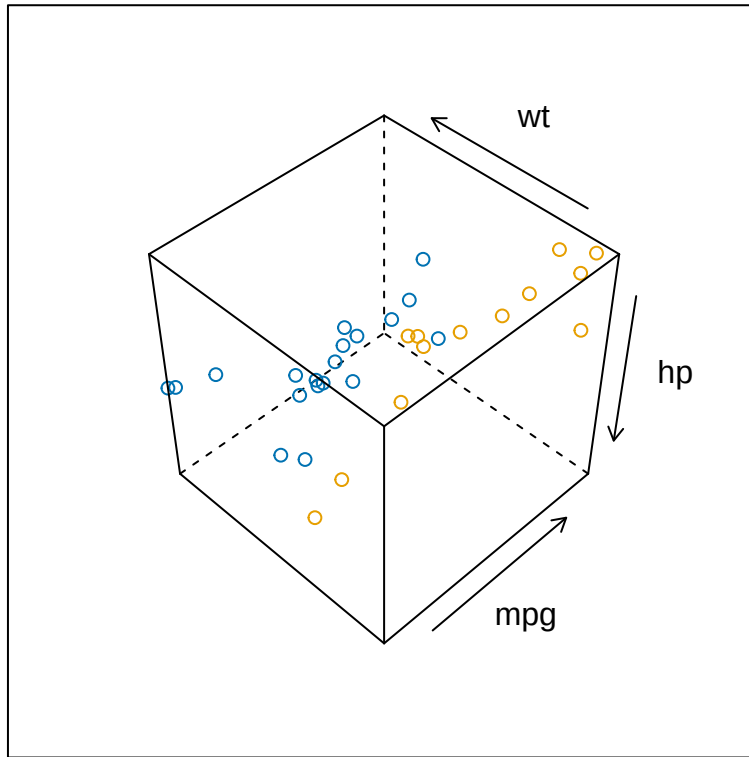
Three-dimensional representations are relatively easy to create, e.g. a three-dimensional cloud of all observation points of the three variables `mpg`, `hp` and `wt`. Without dynamic rotation, however, the exact positions are difficult to recognize.

```
> cloud(mpg ~ hp * wt, data = my_mtcars)
```



The `lattice` function `cloud` contains a number of parameters, some of which are parameters, some of which are difficult to define, including the rotation parameter `screen`, which can be used to change the perspective. Groups can also be created, can also be defined, e.g. with the help of the `groups` parameter. Both options are illustrated with the following example, in which vehicles with manual transmission are marked by red circles and cars with automatic transmission by blue circles.

```
> cloud(mpg ~ hp * wt,  
+       data = my_mtcars,  
+       groups = am,  
+       screen = list(x = 45, y = 135, z = 90))
```



As you can see from the example, it is difficult to determine an insightful perspective. Most of the time (especially with metric data) with dimension reduction methods such as principal component analysis such as principal component analysis and singular value decomposition. These methods are presented in a further teaching unit.

Chapter 11

Dynamic 3D graphs with plotly

Dynamic graphs have been around since the 80s. The system developed at system *XGobi* developed at Bell Labs was embedded in **S** in 1991 and later integrated into **R** via the `rggobi` package. Detailed information about *Xgobi*, which was developed at Bell-Labs developed can be found at [Xgobi](http://lib.stat.cmu.edu/general/XGobi/)¹, Information about the successor *GGobi* can be found at ggobi.org². For both systems exist(ed) interfaces to R, although both interfaces are no longer maintained³. A still very useful, because conceptual overview of the use of 3D graphics in *rggobi* can be found in Wickham et al. (2008).

In the meantime, the `plotly` library developed outside of `lattice` is frequently used in R. developed outside of `plotly`, which makes it very easy to generate dynamic generate dynamic graphs. The following graph, which can only be output via RMarkdown in HTML documents, represents the dynamic variant of the `ccloud` graphic mentioned above. Within RStudio the plot is output via the view panel.

```
> plotly::plot_ly(  
+   my_mtcars,  
+   x = ~hp, y = ~wt, z = ~mpg,
```

¹<http://lib.stat.cmu.edu/general/XGobi/>

²<http://ggobi.org/>

³In my experience, RGGobi tends to work stably on Linux systems, but under Windows 8.1 and R 3.5.1 the system often crashed in 2018. Systems like RGGobi can analyze data sets dynamically using stretching, compression, rotation as well as symbol and color markings. Dynamic graphs using these projection techniques may be suitable for data sets with 3 to 6 variables, but often fail with larger quantities of variables due to the limitation of visualization to three-dimensional spaces and point clouds. Most data science applications comprise several hundred variables hundreds of variables, which means that although dynamic graphics are available, their use limited due to the need to select variables.

```
+ type = "scatter3d",  
+ mode = "markers",  
+ text = rownames(my_mtcars),  
+ color = ~am,  
+ colors = c("red", "blue"))
```

This function call is illustrated interactively in the course and also used as a dynamic graphic in the HTML output. In the PDF output, however, this dynamic graph is suppressed, as it is very difficult to use in PDF files. The `plot_ly` argument `text = rownames(my_mtcars)` makes it possible, by the way, that you can use the mouse or trackball to display not only the observation coordinates as well as the case name. If you only need the observation index, the `text` argument can be replaced by `text = 1:nrow(my_mtcars)`.

Details on the numerous variants of the dynamic `plotly` graphs can be found on *Plotly R Open Source Graphing Libraries*⁴. Details about the above-mentioned specific specified 3D plot in *3D Scatter Plots in R*⁵. In addition to R, Plotly is also available for numerous other programming languages in addition to R, including Python, Matlab and Julia, see the *Plotly Open Source Graphing Libraries*⁶. Among them you will find a specialized interface for `ggplot2`.

Personal note: The easiest way to rotate dynamic graphics easiest is the use of an ergonomic *trackball*. I use a Kensington SlimBlade trackball⁷ as a mouse replacement since many years. General information can be found in Wikipedia: Trackball (German)⁸ and Trackball (English)⁹.

⁴<https://plotly.com/r/>

⁵<https://plotly.com/r/3d-scatter-plots/>

⁶<https://plotly.com/graphing-libraries/>

⁷<https://www.kensington.com/de-de/p/produkte/eingabeger%C3%A4te/trackballs/kensington-slimblade-pro-trackball/>

⁸<https://de.wikipedia.org/wiki/Trackball>

⁹<https://en.wikipedia.org/wiki/Trackball>

Chapter 12

Appendix A: List of links for R software, libraries, help, and introductions

12.1 Basic software

The following basic software is used as the basis for this course.

- **R-Project:** The R-Project¹ website contains all important information about R, including important announcements, links to further sources, documentation, software, etc.
- **CRAN:** The CRAN (Comprehensive R Archive Network²) primarily contains links to downloadable versions of R for Windows, macOS, and Linux, as well as R libraries. The current version for Windows (approx. 88 MB) can be found at R version 4.5.1. on Windows³. CRAN also provides task views, which give an overview of the most important R libraries for specific topics.
- **RStudio:** The software development environment RStudio⁴ (IDE=Integrated Development Environment) is offered in various

¹<https://www.r-project.org>

²<https://cran.r-project.org>

³<https://cran.r-project.org/bin/windows/base/R-4.5.1-win.exe>

⁴<https://posit.co/download/rstudio-desktop/>

versions by the commercial company Posit⁵ (formerly RStudio). The Windows 10/11 version, which is freely available under the AGPL-v3 license, is RStudio Desktop 2025.09.1+401 - Windows 10/11 (64-bit)⁶, which is approximately 288 MB in size.

Alternative IDEs:

- An alternative to RStudio is XEmacs⁷ or GNU Emacs⁸ in conjunction with ESS⁹ (ESS=Emacs Speaks Statistics); This system is still the leading development environment for more than a decade and can be used on Linux, Windows, and macOS.
- A second alternative is to use the IDE Eclipse¹⁰ in conjunction with StatET for R¹¹; this is particularly interesting for developers who, for example, develop with C++ or Java and use R as an additional system.
- A third and modern alternative is the Positron IDE¹² developed by Posit, which is based on VS Code¹³. This Data Science Code Editor shows its strengths in editing R- and/or Python-based data science code and in documenting it. Positron can be found for a large variety of operating system under Download Positron¹⁴. Note that Positron can be extended with other tools, as the system is based on VS Code.

It makes sense to use Emacs and Eclipse especially if you are very familiar with one of these IDEs but have little experience with RStudio. Until around 2010, Emacs and XEmacs were **the** development environments for Unix and Linux systems¹⁵.

Positron is still in the middle of development. Compared to the very mature and stable RStudio, however, it still lacks a number of tools¹⁶ that are useful not only for data analysis but also for developing algorithms for computer-aided statistics and machine learning. This means that Positron still lags behind RStudio, but that could change in the near future.

⁵<https://posit.co/>

⁶<https://download1.rstudio.org/electron/windows/RStudio-2025.09.1-401.exe>

⁷<https://www.xemacs.org/>

⁸<http://www.gnu.org/software/emacs/>

⁹<https://ess.r-project.org/>

¹⁰<https://www.eclipse.org/>

¹¹<https://projects.eclipse.org/projects/science.statet>

¹²<https://posit.co/products/ide/positron/>

¹³<https://code.visualstudio.com/>

¹⁴<https://positron.posit.co/download.html>

¹⁵In Emacs and XEmacs, you can program macros very well using LISP dialects. That is why these IDEs are still popular in Unix and Linux environments. Since around 2015, however, RStudio has been superior to Emacs dialects in almost all areas of software development related to R.

¹⁶For example, the help for R in RStudio is very sophisticated, while in Positron, comparable help for Python is significantly more limited in comparison.

12.2 Manuals and help

- **Manuals:** R-Project¹⁷, in particular R-Manuals¹⁸ and CRAN¹⁹
- **Reference cards:** The *cheat sheets* from Posit Cheatsheets²⁰ for both the RStudio IDE/GUI and libraries and general R commands; the German-language sheets listed on the page are somewhat outdated, but some are more compact.
- **The Big Book of R:** Links to numerous R-related books have been collected by Oscar Baruffa; see the Big Book of R²¹. I do not share all comments, but this URL is often updated.
- **Tidyverse Universe:** At tidyverse²² you will find documentation for numerous and largely consistent R libraries that are particularly suitable for data science. The system essentially consists of the following libraries:
 - ggplot2²³ for graphics based on the *Grammar of Graphics*,
 - tibble²⁴ for clean structuring of *dataframes*,
 - magrittr²⁵ for *piping* data,
 - readr²⁶ for reading data in text format,
 - readxl²⁷ for importing data in Excel format,
 - dplyr²⁸ for preparing and manipulating data,
 - tidyr²⁹ for organizing data in a tidy way,
 - purrr³⁰ for functional programming (keyword *apply* family),

¹⁷<https://www.r-project.org/>

¹⁸<https://cran.r-project.org/manuals.html>

¹⁹<https://cran.r-project.org/>

²⁰<https://posit.co/resources/cheatsheets/>

²¹<https://www.bigbookofr.com/>

²²<https://www.tidyverse.org/>

²³<http://ggplot2.tidyverse.org/>

²⁴<http://tibble.tidyverse.org/>

²⁵<http://magrittr.tidyverse.org/>

²⁶<http://readr.tidyverse.org/>

²⁷<http://readxl.tidyverse.org/>

²⁸<http://dplyr.tidyverse.org/>

²⁹<http://tidyr.tidyverse.org/>

³⁰<http://purrr.tidyverse.org/>

- `stringr`³¹ as a consistent system for processing character strings,
- `forcats`³² as a consistent system for processing factor variables for nominal and ordinal scaled variables,
- `lubridate`³³ as a consistent system for processing dates and times (for times, you can also use the R package `hms`³⁴),
- `modelr`³⁵ for using pipes in modeling (tends to be more suitable for teaching purposes),
- `broom`³⁶, which can be used to map results from model estimators in `Tibble`’s and, if necessary, supplement them, and
- `reprex`³⁷ for documenting problems on GitHub “How to write a reproducible example”³⁸ and Stackoverflow³⁹ – the latter currently (October 27, 2025) contains over 510,000 questions⁴⁰.

Notes:

The basic library system of R, which is installed as standard with the R interpreter, provides a range of functions for the above tasks, but has evolved over time, particularly in relation to data frames and string processing, and is therefore not consistently coordinated. The *tidyverse*, on the other hand, is characterized by a largely clean and coordinated organization of functions, which in some cases also reduces or eliminates R inconsistencies.

- The *tidyverse*⁴³ is still under development, so changes to functionality and syntax are to be expected at any time.
- The latest development status can usually be found on GitHub. These can be found in the sections “Development Version” on each Tidyverse package site, e.g., `dplyr-package`⁴⁴.

³¹<http://stringr.tidyverse.org/>

³²<http://forcats.tidyverse.org/>

³³<http://lubridate.tidyverse.org/>

³⁴<https://cran.r-project.org/web/packages/hms/index.html>

³⁵<https://cran.r-project.org/web/packages/modelr/index.html>

³⁶<https://broom.tidymodels.org/>

³⁷<http://reprex.tidyverse.org/>

³⁸<https://gist.github.com/hadley/270442>

³⁹<https://stackoverflow.com/questions/tagged/r>

⁴⁰Not much has changed here in about a year. The reason for this is that nowadays, many users no longer post explicit questions on help sites such as Stackoverflow, but rather use the assistance provided by LLM⁴¹’s such as Copilot⁴², etc. This could backfire in the medium term, as LLM’s often have to obtain the necessary information from the internet. However, if these sources are no longer available, it may also become difficult for LLMs to provide up-to-date assistance. In addition, LLMs frequently fabricate and fantasize, resulting in incorrect or inaccurate answers.

⁴³<https://www.tidyverse.org/>

⁴⁴<https://dplyr.tidyverse.org/>

- In addition to the most important *Tidyverse* libraries, you can find other useful libraries on the R Packages⁴⁵ page presented by Posit.
- In addition, there are several useful libraries for special tasks when creating R packages:
 - devtools⁴⁶ for creating packages and installing development versions of packages.
 - roxygen2⁴⁷ for documenting packages. Note that RStudio support the development and documentation of packages, see section Writing Packages⁴⁸ in the RStudio User Guide⁴⁹.
 - packrat⁵⁰ for encapsulating project-specific R libraries. This approach is now replaced by the R package renv⁵¹, which can be easily used in RStudio, see the help⁵² in the RStudio User Guide⁵³.
- The parameters of the functions in the *ggplot2* library are sometimes difficult to find. The R function call `vignette("ggplot2-specs")` is helpful here. However, this is only available after loading *ggplot2* with `library(ggplot2)`.
- At Tidymodels⁵⁴ you will find an extensive collection of R packages that can be used to estimate and evaluate statistical models and machine learning methods within the framework of the *Tidyverse* philosophy. The methods contained therein tend to be more suitable for the analysis and prediction of cross-sectional data.
- Tidyverts⁵⁵ is an extension of Tidyverse for the administration, cleaning, preparation, exploration, modeling, prediction, and evaluation of time series (longitudinal data).
- At mlr⁵⁶ and its successor mlr3⁵⁷, which is still under development, you will find an excellent competitor to the *Tidymodels* libraries, which also contain numerous statistical models and machine learning methods⁵⁸.

⁴⁵<https://posit.co/products/open-source/rpackages/>

⁴⁶<https://github.com/r-lib/devtools>

⁴⁷<https://cran.r-project.org/web/packages/roxygen2/vignettes/roxygen2.html>

⁴⁸<https://docs.posit.co/ide/user/ide/guide/pkg-devel/writing-packages.html>

⁴⁹<https://docs.posit.co/ide/user/>

⁵⁰<http://rstudio.github.io/packrat/>

⁵¹<https://rstudio.github.io/renv/index.html>

⁵²<https://docs.posit.co/ide/user/ide/guide/environments/r/renv.html>

⁵³<https://docs.posit.co/ide/user/>

⁵⁴<https://www.tidymodels.org>

⁵⁵<https://tidyverts.org/>

⁵⁶<https://mlr.mlr-org.com/>

⁵⁷<https://mlr3.mlr-org.com/>

⁵⁸In previous courses on *Data Science Methods*, the R package *mlr* was mainly used.

- `mosaic`⁵⁹ is an R library created as part of the educational MOSAIC project⁶⁰, which provides a range of very easy-to-use, consistent evaluation functions that can also be used in conjunction with *tidyverse*, such as `favstats`, `tally`, and `qdata`: *mosaic* functions can be used either with the help of the formula interface *ggformula* based on *ggplot2* or with the help of *lattice* graphics, both of which are explained in the following chapters.
- Under Getting Help with R⁶¹, you will find an overview page of the internal and external help systems available for R.
- Under RDocumentation⁶², you will find a search engine that allows you to search all packages stored on CRAN and Bioconductor, but no longer Github. Bioconductor packages are particularly interesting for biometricians, epidemiologists, and medical statisticians.
- At R-Seek⁶³ you will find an old-fashioned, but efficient general search engine for **R**.
- Help for R: A language and environment for statistical computing and graphics⁶⁴ is an R-specific search engine that can also be accessed from R using the R function `RSiteSearch("<searchterm>")` or via Rgui using the menu ‘help->search.r-project.org’. However, the number of entries can be overwhelming. For example, `RSiteSearch("knitr")` returns over 13,100 scored Internet references to the `knitr` library; there are nearly twice as many for `ggplot2`.
- You can find numerous informative articles at R-bloggers⁶⁵.
- You can find numerous questions and answers about **R** at Stackoverflow⁶⁶. Note: Before posting a question, you should (a) check carefully whether the problem has already been solved and (b) document the problem in detail, e.g. using the *Tidyverse* library `reprex`⁶⁷.

12.3 Supplementary online information on textbooks and libraries

The following pages mainly contain supplementary information.

⁵⁹<https://cran.r-project.org/web/packages/mosaic/index.html>

⁶⁰<http://www.mosaic-web.org/>

⁶¹<https://www.r-project.org/help.html>

⁶²<https://www.rdocumentation.org>

⁶³<https://rseek.org/>

⁶⁴<http://search.r-project.org/>

⁶⁵<https://www.r-bloggers.com/>

⁶⁶<https://stackoverflow.com/questions/tagged/r>

⁶⁷<http://reprex.tidyverse.org/>

- The HTML version of Wickham, H., Centinkaya-Rundel, and Golemund, G. (2023): *R for Data Science - Import, Tidy, Transform, Visualize, and Model Data*. O'Reilly. Second edition. can be found at R for Data Science⁶⁸.
- The HTML version of the first edition of Wickham, H. (2014): *Advanced R* can be found at Adv-R⁶⁹. The second, more comprehensive edition published 2019 can be found at Advanced R, 2nd ed⁷⁰. And at Advanced R Solutions⁷¹ you can find the solutions to the exercises in the second edition of *Advanced R* created by Grosser, Bumann, and Wickham (2022). All three books are available in print from Chapman and Hall.
- The HTML version of the recently published second edition of Wickham, H. and Bryan, J. (2023): *R Packages*. O'Reilly can be found at R-Packages⁷². This book explains how to create and document R packages.
- On R-Packages⁷³ you can find the HTML version of the recently published second edition of Wickham, H. and Bryan, J. (2023): *R Packages*. O'Reilly. This book explains how to create and document R packages.
- On ggplots-YHatQ⁷⁴, a Python variant of ggplot2 was found with some clear examples that can be easily translated back into R. (Please do not use +\ in R, only + as a layer connection.) Unfortunately, this server was no longer active in the recent past nor currently (October 31, 2023; April 6, 2025, October 28, 2025) not active.
- At ggplot2 extensions⁷⁵, you will find numerous extensions (151 in total on October 28, 2025) of *ggplot2*. Also useful is ggExtra⁷⁶, which allows you to extend two-dimensional plots with histograms of the two marginal distributions.
- R-based dynamic web applications can be created most easily with the help of Shiny⁷⁷. This environment can be run *locally* on a computer within RStudio without installing any additional software. However, use on websites requires the operation of a Shiny server, such as the one provided by RStudio at Rpubs⁷⁸; a rather slow but currently free service. You can also install and run Shiny on your own server.
- Project and version management is easiest to implement using the project definition functions integrated in RStudio (Using

⁶⁸<https://r4ds.hadley.nz/>

⁶⁹<http://adv-r.had.co.nz>

⁷⁰<https://adv-r.hadley.nz/>

⁷¹<https://advanced-r-solutions.rbind.io/>

⁷²<http://r-pkgs.had.co.nz/>

⁷³<http://r-pkgs.had.co.nz/>

⁷⁴<http://ggplot.yhathq.com/>

⁷⁵<https://exts.ggplot2.tidyverse.org/>

⁷⁶<https://github.com/daattali/ggExtra>

⁷⁷<https://shiny.posit.co/r/>

⁷⁸<http://rpubs.com/>

Projects⁷⁹ and Version Control⁸⁰). This uses external version control software such as Git (<https://git-scm.com/>).

- The R-Forge page `bigmemory`⁸¹ provides a useful, albeit outdated, overview of the CRAN-available R packages `bigalgebra`, `biganalytics`, `bigmemory`, `bigtabulate`, and `bigvideo`, which can be used to easily process large data sets that no longer fit into RAM. Note in particular the package `biglm`, which can be used to estimate linear and generalized linear models where the data no longer fits into RAM.
- Large Language Model support can also be integrated in R using a variety of R packages like `mall`⁸² and `ellmer`⁸³. RStudio supports the direct of Microsoft Copilot⁸⁴ use of GitHub Copilot⁸⁵. Using an API key a large variety of LLM chatbots can be called via the POST method directly from R using packages like `httr`⁸⁶ for internet communication and `jsonlite`⁸⁷ for parsing the results returned in JSON files.
- The website of Pairach Piboonrungrroj⁸⁸ provides an overview of coding standards in R. The following standards are commonly used:
 - Google’s R Style Guide⁸⁹,
 - Hadley Wickham’s Style Guide⁹⁰ and, in particular,
 - Hadley Wickham’s tidyverse style guide⁹¹ as an updated version.
- SQL databases can be used within R.
 - The easiest way is to use the within-memory relational database management system SQLite⁹² with the R library RSQLite⁹³. SQLite has the advantage that you do not need to install a database system.

⁷⁹<https://support.posit.co/hc/en-us/articles/200526207-Using-Projects>

⁸⁰<https://docs.posit.co/ide/user/ide/guide/tools/version-control.html>

⁸¹https://r-forge.r-project.org/R/?group_id=556

⁸²<https://cran.r-project.org/web/packages/mall/index.html>

⁸³<https://ellmer.tidyverse.org/>

⁸⁴<https://copilot.microsoft.com/>

⁸⁵<https://docs.posit.co/ide/user/ide/guide/tools/copilot.html>

⁸⁶<https://cran.r-project.org/web/packages/httr/index.html>

⁸⁷<https://cran.r-project.org/web/packages/jsonlite/index.html>

⁸⁸<https://pairach.com/2012/06/06/r-style-guide/>

⁸⁹<https://google.github.io/styleguide/Rguide.xml>

⁹⁰<http://adv-r.had.co.nz/Style.html>

⁹¹<http://style.tidyverse.org/>

⁹²<https://www.sqlite.org>

⁹³<https://cran.r-project.org/web/packages/RSQLite/index.html>

- MySQL⁹⁴ is also easy to use and suitable for more complex purposes and larger amounts of data when used with the R library RMySQL⁹⁵. Please note that to install MySQL free of charge, you must use the community version, e.g., the MySQL Installer for Windows (<https://dev.mysql.com/downloads/mysql/>). In the R community, MySQL is now increasingly being replaced by MariaDB (<https://mariadb.org/>). Note that RMySQL nowadays also uses the RMariaDB driver; see the entry “RMariaDB⁹⁶ 1.3.4 (2025-02-24)” in the DBI-description⁹⁷. The primary textbook used in the course, *Modern Data Science with R*, still uses MySQL in both the first and second editions (Baumer et al. (2017), Baumer et al. (2021)).
- The R package DBI⁹⁸ provides an SQL interface that allows generic access to various databases by simply changing the database connection. According to the documentation, this is currently possible with over 30 database systems. These include SQLite, MySQL, MariaDB, and PostGres databases. This works for table generation as well as table queries and combinations. The vignette Introduction to DBI⁹⁹ provides an overview of how to use it.
- With the help of the *Tidyverse* library *dplyr* and the SQL backend library dbplyr¹⁰⁰, you can also indirectly perform SQL queries (but only limited table or database generation steps) using the *dplyr* syntax; see also Using dplyr to query databases directly instead of using SQL¹⁰¹. The RODBC¹⁰² package also allows you to access Microsoft-based systems such as MS SQL Server, Access, and Excel.
- In 2015, Microsoft acquired the company Revolution Analytics¹⁰³. Revolution Analytics specialized in scalable R solutions. As part of its integration into the Microsoft Azure architecture and the associated solutions for machine learning, the now defunct Microsoft R Application Network (MRAN) was also set up, whose pages included an open source version called Microsoft R Open and a page with descriptions of the R packages. The latter also allowed visualization of package dependencies, e.g., for dplyr, see the “Dependencies Graph” there, which is reproduced here for the 2021 status as an example below this paragraph. Such graphs are not very clear. MRAN contained significantly fewer packages than CRAN and Github combined. A current description of how to use R in Azure using very simple Jupyter notebooks can be found in the web page Interactive R development¹⁰⁴.

⁹⁴<https://www.mysql.com/>

⁹⁵<https://CRAN.R-project.org/package=RMySQL>

⁹⁶<https://rmariadb.r-dbi.org>

⁹⁷<https://github.com/r-dbi/backends#readme>

⁹⁸<https://cran.r-project.org/package=DBI>

⁹⁹<https://cran.r-project.org/web/packages/DBI/vignettes/DBI.html>

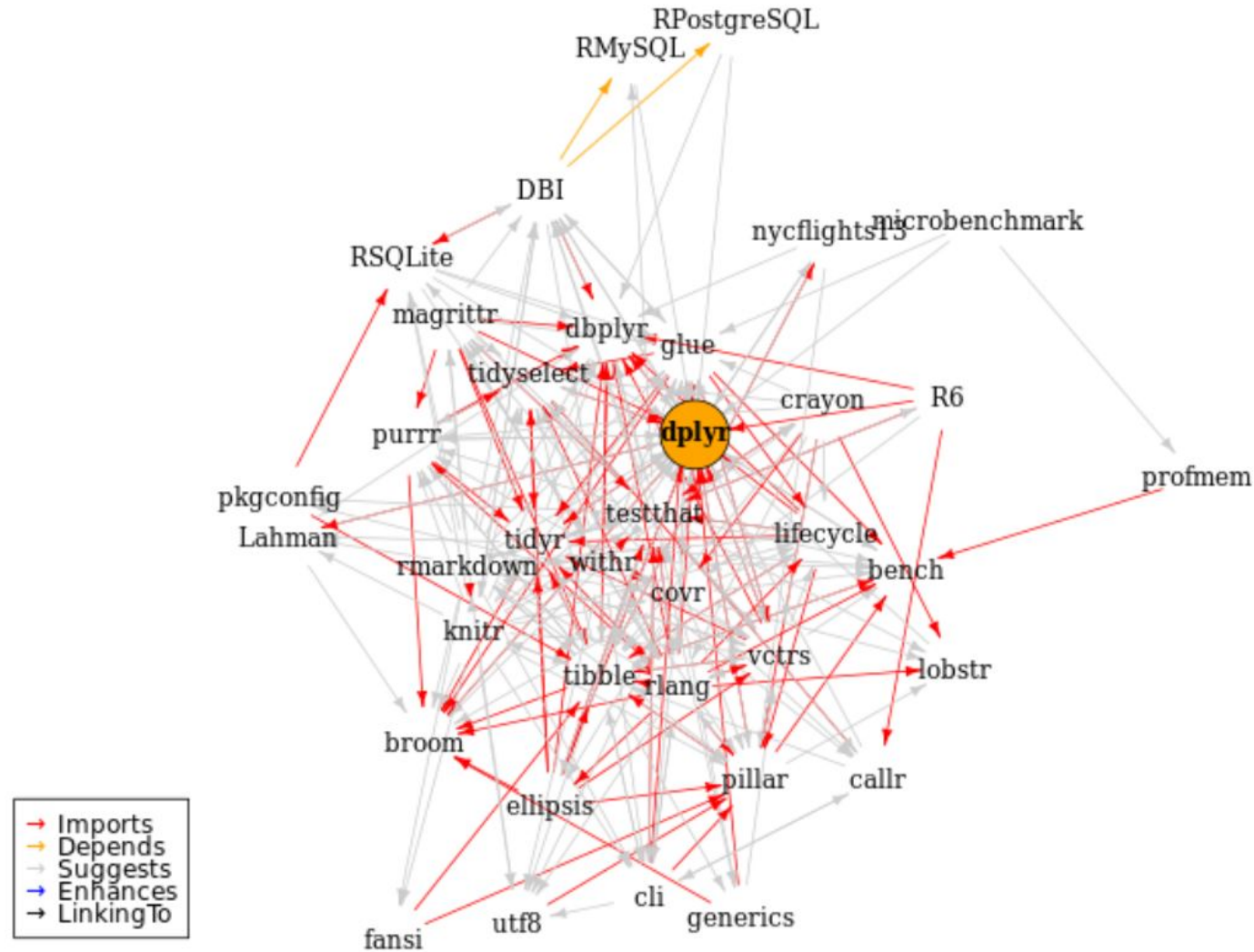
¹⁰⁰<https://dbplyr.tidyverse.org/>

¹⁰¹<https://blog.exploratory.io/using-dplyr-to-query-databases-directly-instead-of-using-sql-fed43f059ed6>

¹⁰²<https://cran.r-project.org/web/packages/RODBC/RODBC.pdf>

¹⁰³https://en.wikipedia.org/wiki/Revolution_Analytics

¹⁰⁴<https://learn.microsoft.com/en-us/azure/machine-learning/how-to-r-interactive-development>



12.4 Documentation of R programs and data analyses

There are a number of methods available for documentation; here I will limit myself to tools that are integrated into **R/RStudio** or tools in which R programs can be embedded.

- Reproducible results can be created most easily with the help of RStudio using the R library `knitr`¹⁰⁵ and R Markdown¹⁰⁶ or the R Markdown successor Quarto¹⁰⁷¹⁰⁸. These documents integrate text with R code segments (known as chunks) and their outputs (text and graphics). The outputs can be HTML, PDF, and Word documents using numerous output formats (static documents, interactive documents, dashboards, and presentations); see the R Markdown Gallery¹¹⁰ for more information. Please note that TeX¹¹¹ and LaTeX¹¹² must be installed to generate PDF documents. Recommended options for Windows include MiKTeX¹¹³ or the TinyTeX¹¹⁴ system, which is a slimmed-down version of TeX Live. A relatively detailed summary of the most important R Markdown commands can be found in Using R Markdown for Class Reports¹¹⁵. The introduction Getting used to R, RStudio, and R Markdown¹¹⁶ is limited to the most important commands, some of which are illustrated by (silent?) YouTube videos.
- A relatively new documentation environment that can be used for other programming languages such as Python, Julia, and Observable (a JavaScript extension) in addition to R is Quarto¹¹⁷, which is essentially a modernized, albeit very similar, version of RMarkdown¹¹⁸. With this environment, you can create articles, presentations, books, websites, blogs, and, for about 1.5 years now, knowledge repositories via Confluence, similar to RMarkdown. The most important Quarto commands can be found in the cheat sheet Publish and Share with Quarto :: Cheatsheet.¹¹⁹
- **Note:** RMarkdown, bookdown, etc. are now in maintenance mode. For new articles, reports, presentations, dashboard, website including blogs as well as interactive docs, I strongly recommend using Quarto.

¹⁰⁵<https://yihui.name/knitr/>

¹⁰⁶<https://rmarkdown.rstudio.com/>

¹⁰⁷<https://quarto.org/>

¹⁰⁸Most course materials, including these supplementary DST scripts, were created with RMarkdown. The difference between RMarkdown and Quarto is not huge, but semi-automatic conversion using the R package `converttr`¹⁰⁹ does not work without considerable manual post-processing.

¹¹⁰<https://rmarkdown.rstudio.com/gallery.html>

¹¹¹<https://en.wikipedia.org/wiki/TeX>

¹¹²<https://en.wikipedia.org/wiki/LaTeX>

¹¹³<https://miktex.org>

¹¹⁴<https://yihui.name/tinytex/>

¹¹⁵<http://www.stat.cmu.edu/~cshalizi/rmarkdown/>

¹¹⁶<https://ismayc.github.io/rbasics-book/index.html>

¹¹⁷<https://quarto.org/>

¹¹⁸The similarity implies that a lot of information about *RMarkdown* and *knitr* available in books, articles, and on websites can be applied to Quarto in almost the same way.

¹¹⁹<https://rstudio.github.io/cheatsheets/html/quarto.html>

- A competitor to R Markdown is the use of the WYSIWYM¹²⁰-based¹²¹ documentation system LyX (The Document Processor)¹²³, which is not very easy to install for integration with R chunks using the R package `knitr`; see Using knitr with LyX¹²⁴. Please note that the LyX bundle also installs MiKTeX on older installation images; this is no longer the case with the current LyX 2.4.4 installer, so a TeX/LaTeX system must be installed beforehand.
- **Important note:** When installing MiKTeX or TeXLive, you should always enable the automatic post-installation of TeX and LaTeX packages from the Internet.

12.5 Relevant IT standards

When using R, you will repeatedly encounter standards that standardize character sets, local definitions of dates, times, time zones, number notation including currencies, etc. This is particularly important when using source code, reading files (especially those containing non-ASCII characters¹²⁵ such as umlauts), and entering dates and times in text form. A brief overview can be found under the entry `Locales`¹²⁷ in the Tidyverse library `readr`¹²⁸. These IT standards are particularly important if you are working with German terms and expressions rather than English or American ones.

Locales are based on the following standards, among others:

- Language codes according to ISO 639¹²⁹, where the ISO 639-1¹³⁰ replaces the two-letter alpha-2 code for *German* with the code `de` (see List of ISO 639-1 codes¹³¹) and the three-letter ISO 639-2¹³² by `deu` or `ger` (according to the List of ISO 639-2 codes¹³³). Language codes define, among other things, the names of days of the week and months, as well as number formats.
- Character codes were previously often defined using the ISO 8859¹³⁴ family of standards – these can still be found occasionally on

¹²⁰<https://en.wikipedia.org/wiki/WYSIWYM>

¹²¹WYSIWYM stands for *what-you-see-is-what-you-mean* and, as a tool for structured documentation, is the opposite of WYSIWYG¹²² systems (*what-you-see-is-what-you-get*) such as MS Word and LibreOffice.

¹²³<http://www.lyx.org/>

¹²⁴<https://yihui.name/knitr/demo/lyx/>

¹²⁵ASCII = American Standard Code for Information Interchange, see, for example, ASCII¹²⁶.

¹²⁷<http://readr.tidyverse.org/articles/locales.html>

¹²⁸<http://readr.tidyverse.org>

¹²⁹https://en.wikipedia.org/wiki/ISO_639

¹³⁰https://en.wikipedia.org/wiki/ISO_639-1

¹³¹https://en.wikipedia.org/wiki/List_of_ISO_639-1_codes

¹³²https://en.wikipedia.org/wiki/ISO_639-2

¹³³https://en.wikipedia.org/wiki/List_of_ISO_639-2_codes

¹³⁴https://de.wikipedia.org/wiki/ISO_8859

web servers and quite frequently in legacy systems. Nowadays, the¹³⁵ UTF-8¹³⁶ is now used, which can be used to encode virtually all characters and symbols in written languages – R Studio also generally uses UTF-8, whereby the character code can be specified within RStudio in the global options, the project options, and for a specific R or RMarkdown source file. UTF-8 is used on Linux and macOS, but Windows internally uses a compromise between UTF-8 and UTF-32. R and RStudio now also use UTF-8 in the standard installation on Windows. However, you often have to import files (e.g., older R programs or data sets) using one of the following standards:

- The so-called ISO 8859-1¹³⁷ standard (also known as **Latin-1**) or
- the Windows-1252 Western European Standard¹³⁸ based on it.
- A very comprehensive Unicode table can be found at UT¹³⁹.
- In RStudio, you can set or take into account the encoding in the following menus:
 - Tools → Global options → Code → Saving → Default text encoding (the default setting used to be UTF-8, now it is)
 - Tools → Project Options → Code Editing → Text encoding (default setting for this project is UTF-8)
 - File → Save with Encoding → Choose Encoding
 - File → Reopen with Encoding → Choose Encoding.
 - The last commands are very useful if you suddenly see strange symbols in a foreign file that are obviously being used as substitutes for characters that are not available in the local language. You can then test whether the file is readable after all by re-reading it with different character sets.
 - The options we commonly use are:
 - * UTF-8 (now system default)
 - * ISO-8859-1 (formerly system default); also known as **Latin-1** or **latin1**
 - * WINDOWS-1252
 - * ASCII

¹³⁵UTF-8 is the abbreviation for UCS Transformation Format, where UCS is an abbreviation for Universal Character Set.

¹³⁶<https://en.wikipedia.org/wiki/UTF-8>

¹³⁷https://en.wikipedia.org/wiki/ISO_8859-1

¹³⁸<https://en.wikipedia.org/wiki/Windows-1252>

¹³⁹<https://unicode-table.com>

- * If umlauts and special characters are not displayed correctly in a source file (usually R or R Markdown), try opening the file with a different encoding and, if necessary, save it with a new encoding (preferably UTF-8).
- * Encodings cause problems time and again, especially when importing data with character strings and with database interfaces; for the latter, see UTF8 encoding using RMySQL¹⁴⁰ and Change MySQL default character set to UTF-8 in my.cnf¹⁴¹¹⁴². An overview of encodings can be found on the website What every programmer absolutely, positively needs to know about encodings and character sets to work with text¹⁴³ by David Zentgraf. The help commands `?Encoding` and `?locales` provide information in R about setting and querying encodings and locales.
- The *locales* also affect date and time specifications, whereby the UTC¹⁴⁴ as Coordinated Universal Time is essentially the successor to Greenwich Mean Time¹⁴⁵¹⁴⁶ as a spatially and organizationally comprehensive, unambiguous date and time specification. However, spatially and organizationally determined local deviations from this universal time are problematic; time specifications in documents and files often refer to local time (see tz database¹⁴⁷), where spatial deviations due to location and organizational regional deviations due to Daylight Saving Time¹⁴⁸¹⁴⁹ must be taken into account in addition to different spellings. Special attention must be paid to the time zone (Time Zone TZ¹⁵⁰), especially when using designations, e.g., “East Time” means something different in the US than in Canada. The only correct use is the TZ according to the official List of tz database time zones¹⁵¹ in accordance with the IANA Time Zone Database¹⁵²¹⁵³.
- Date and time specifications in R often refer to POSIX¹⁵⁴ standards¹⁵⁵. POSIX standards were previously common in the UNIX/LINUX world, but have now become established outside this system environment. Since R was originally developed under Linux¹⁵⁶, R uses POSIX for date and time specifications in both the *Base* system and in numerous libraries. These are sometimes

¹⁴⁰<https://stackoverflow.com/questions/38347348/utf8-encoding-using-rmysql>

¹⁴¹<https://stackoverflow.com/questions/3513773/change-mysql-default-character-set-to-utf-8-in-my-cnf>

¹⁴²In MySQL for Windows, the file `my.cnf` is called `my.ini`; it is located in the user-specific directory, e.g. `[user]\MySQL\MySQL Server 5.7\my.ini`.

¹⁴³<http://kunststube.net/encoding/>

¹⁴⁴https://en.wikipedia.org/wiki/Coordinated_Universal_Time

¹⁴⁵https://en.wikipedia.org/wiki/Greenwich_Mean_Time

¹⁴⁶Greenwich Mean Time is often abbreviated as GMT.

¹⁴⁷https://en.wikipedia.org/wiki/Tz_database

¹⁴⁸https://en.wikipedia.org/wiki/Daylight_saving_time

¹⁴⁹DST here is the abbreviation for ‘Daylight Saving Time’, which is what is known in Germany as ‘summer time’ and ‘winter time.’ Do not confuse the abbreviation for ‘Daylight Saving Time’ with the same 3-letter abbreviation DST for ‘Data Science Tools’.

¹⁵⁰https://en.wikipedia.org/wiki/Time_zone

¹⁵¹https://en.wikipedia.org/wiki/List_of_tz_database_time_zones

¹⁵²<https://www.iana.org/time-zones>

¹⁵³IANA stands for ‘Internet Assigned Numbers Authority’.

¹⁵⁴<https://en.wikipedia.org/wiki/POSIX>

¹⁵⁵POSIX is an abbreviation for ‘Portable Operating System Interface’.

¹⁵⁶<https://en.wikipedia.org/wiki/Linux>

colloquially referred to as UNIX time¹⁵⁷, although there are subtle differences. In the Microsoft world, POSIX standards have long been implemented through toolkits such as Cygwin¹⁵⁸. The corresponding Cygwin installation setup routine can be found on the Cygwin homepage¹⁵⁹. However, the RTools 4.5¹⁶⁰ mentioned above already contains the programs required for compiling R and R libraries. The POSIX date specifies the number of days since January 1, 1970 (January 1, 1970 = 1), while POSIX time, which is implemented in R as a `POSIXct` object, indicates the number of seconds since January 1, 1970, at 00:01 (January 1, 1970, at 00:01 = 1). Dates or times before January 1, 1970, are encoded with zero and negative numbers. An overview of classic encoding in R can be found in Dates and Times in R¹⁶¹. However, it is more convenient to use the functions implemented in the *Tidyverse* library `lubridate`¹⁶², as these abstract implementation details such as `POSIXct`. The standard ISO 8601¹⁶³ defines formats such as YYYY-MM-DD and MM/DD/YYYY for date and time specifications; this standard therefore refers to the notation resp. representation (including time zone information), not to the implementation. A note on output: The R base function `strptime` can be used to convert between POSIX objects (`POSIXct` and `POSIXlt`) and string representations; The format specifications available there with the help of `strptime` are also used in the `ggplot2` function `scale_x_date` to specify the date and time output in graphs.

- Color specifications usually refer to the RGB¹⁶⁴ color space or the CMYK¹⁶⁵ color model. Since users usually have little knowledge of color models, it often makes sense in R to work with the default settings first.
 - If you need a specific color code, you can find it in the R Color Charts of the Stower Institute at `ColorChart.pdf`¹⁶⁶¹⁶⁷
 - At `ColorBrewer 2.0`¹⁶⁸ you will find an interactive web application that is particularly useful for designing maps, but also offers several color scales that can be used to represent (a) sequential, (b) divergent, and (c) unordered qualitative attributes. With the `palette_explorer` function from the R package `tmertools`, you can also inspect the corresponding color scales for the three scale categories *sequential* vs. *categorical* vs. *divergent* and select a suitable Brewer palette. Unfortunately, the R package `tmertools` in its current version on CRAN for R 4.5.1 does not longer contains the function `palette_explorer`. And the replacement package `c4a`, which is available on GitHub, cannot yet be installed

¹⁵⁷https://en.wikipedia.org/wiki/Unix_time

¹⁵⁸<https://en.wikipedia.org/wiki/Cygwin>

¹⁵⁹<https://www.cygwin.com/>

¹⁶⁰<https://cran.r-project.org/bin/windows/Rtools/rtools45/rtools.html>

¹⁶¹<https://www.stat.berkeley.edu/~s133/dates.html>

¹⁶²<https://lubridate.tidyverse.org/>

¹⁶³https://en.wikipedia.org/wiki/ISO_8601

¹⁶⁴https://en.wikipedia.org/wiki/RGB_color_model

¹⁶⁵https://en.wikipedia.org/wiki/CMYK_color_model

¹⁶⁶<https://www.uvm.edu/~ngotelli/Rscripts/ColorChart.pdf>

¹⁶⁷The original Stower Institute document was no longer available at <http://research.stowers-institute.org/efg/R/Color/Chart>. This link has therefore been replaced by a copy that is still available externally.

¹⁶⁸<http://colorbrewer2.org/#type=sequential&scheme=BuGn&n=3>

without errors under Windows. However, you can still install an outdated version of `tmertools` under R 4.5.1 using the installation command `remotes::install_version("tmertools", version = "3.1", force = TRUE)` and call it with `tmertools::palette_explorer()`. You can also create another subdirectory (e.g. by `dir.create("~/R/oldlibs", recursive = TRUE)`), install the older `tmertools` version using `remotes::install_version("tmertools", version = "3.1", lib = "~/R/oldlibs")` and load this version explicitly by `library(tmertools, lib.loc = "~/R/oldlibs")` before invoking `palette_explorer()`.

- ‘When designing colors, it is important to consider not only red-green color blindness (see the tick mark box on the ColorBrewer 2.0 homepage) but also the transformation to grayscale¹⁶⁹. On ColorHexa¹⁷⁰, you can convert a color value (e.g., `#b2d2e9`¹⁷¹) or a color (e.g., ‘dark blue’¹⁷²) and receive the corresponding color information.
- A brief but useful summary of the most important information about colors and their use in R can be found in the R color cheatsheet¹⁷³; however, some of the links provided there do not work or result in security warnings.

¹⁶⁹Unfortunately, introductory seminars, seminars, bachelor’s and master’s theses, and sometimes even doctoral theses often contain graphics that are no longer legible when printed in black and white or grayscale or when copied.

¹⁷⁰<https://www.colorhexa.com>

¹⁷¹<https://www.colorhexa.com/b2d2e9>

¹⁷²<https://www.colorhexa.com/00008b>

¹⁷³<https://www.nceas.ucsb.edu/sites/default/files/2020-04/colorPaletteCheatsheet.pdf>

Chapter 13

References

- Abelson, H., Sussman, G. J., & Sussman, J. (1979). *Structure and interpretation of computer programs*. MIT Press.
- Badia, A. (2020). *SQL for data science – data cleaning, wrangling and analytics with relational databases*. Springer.
- Baumer, B. S., Kaplan, D. T., & Horton, N. J. (2017). *Modern data science with r*. CRC-Press.
- Baumer, B. S., Kaplan, D. T., & Horton, N. J. (2021). *Modern data science with r* (2nd ed.). CRC-Press.
- Brown, J. A., Pakin, S., & Polivka, R. (1988). *APL2 at a glance*. Prentice Hall.
- Chang, W. (2023). *R graphics cookbook – practical recipes for visualizing data*. CRC-Press.
- Cleveland, W. S. (1993). *Visualizing data*. Hobart Press.
- Gandrud, C. (2020). *Reproducible research with r and RStudio*. CRC-Press.
- Grömping, U. (2019). *South german credit data: Correcting a widely used data set* (4). Beuth Hochschule für Technik Berlin.
https://doi.org/http://www1.beuth-hochschule.de/FB_II/reports/Report-2019-004.pdf
- Grosser, M., Bumann, H., & Wickham, H. (2022). *Advanced r solutions*. CRC-Press.
- Hillebrand, J., & Nierhoff, M. H. (2015). *Mastering RStudio - develop, communicate, and collaborate with r*. PACKT.
- Horton, N. J., & Kleinman, K. (2015). *Using r and RStudio for data management, statistical analysis and graphics* (2nd ed.). CRC Press.
- Iverson, K. E. (1962). *A programming language*. Wiley.
- Kofler, M. (2023). *Linux - das umfassende handbuch*. Rheinwerk Computing.
- Küstern, U., & Kalinowski, C. (2001). Traditionelle verfahren der multivariaten statistik. In H. Hippner, U. Küstern, M. Meyer, & K. D. Wilde (eds.), *Handbuch data mining im marketing - knowledge discovery in marketing data bases* (pp. 131–192). Gabler - Vieweg.

- Murrell, P. (2006). *R graphics*. CRC Press.
- Ohri, A. (2018). *Python for r users – a data science approach*. Wiley.
- Rahlf, T. (2014). *Datendesign mit r: 100 visualisierungsbeispiele*. Open Source Press.
- Rahlf, T. (2018). *Datenvisualisierung mit r - 111 beispiele*. Springer.
- Venables, W. N., & Ripley, B. D. (2002). *Modern applied statistics with s* (4th ed.). Springer.
- Wickham, H. (2016). *ggplot2 - elegant graphics for data analysis*. Springer.
- Wickham, H. (2019). *Advanced r* (2nd ed.). CRC-Press.
- Wickham, H., Cetinkaya-Rundel, M., & Grolemund, G. (2023). *R for data science - import, tidy, transform, visualize, and model data* (2nd ed.). O'Reilly.
- Wickham, H., & Grolemund, G. (2016). *R for data science - import, tidy, transform, visualize, and model data*. O'Reilly.
- Wickham, H., & Grolemund, G. (2018). *R für data science - daten importieren, bereinigen, umformen, modellieren und visualisieren*. O'Reilly Deutschland.
- Wickham, H., Lawrence, M., Lang, D. T., & Swayne, D. F. (2008). An introduction to rggobi. *R News*, 8(2), 3–7.
- Zhang, N. (2021). *A tour of data science – learn r and python in parallel*. CRC.
- Zhao, Y., & Cen, Y. (2014). *Data mining applications with r*. Academic Press.